Principia Softwarica: The Plan 9 Shell `rc`

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
Tom Duff

March 10, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **7**
- 1.1 Motivations . . . 7
- 1.2 The Plan 9 shell: `rc` . . . 8
- 1.3 Other shells . . . 8
- 1.4 Getting started . . . 9
- 1.5 Requirements . . . 10
- 1.6 About this document . . . 11
- 1.7 Copyright . . . 11
- 1.8 Acknowledgments . . . 11

2 Overview **12**
- 2.1 Essential shell features . . . 12
  - 2.1.1 Running commands . . . 12
  - 2.1.2 Redirections . . . 13
  - 2.1.3 Pipes . . . 13
  - 2.1.4 Storing commands in a script . . . 13
  - 2.1.5 A shell as a scripting language . . . 14
- 2.2 `rc` command-line interface . . . 14
- 2.3 `boot.rc` . . . 14
  - 2.3.1 `#!` . . . 15
  - 2.3.2 Initialization script . . . 16
  - 2.3.3 Variables . . . 16
  - 2.3.4 `$path` . . . 16
  - 2.3.5 Comments . . . 16
  - 2.3.6 Quoting and escaping . . . 17
  - 2.3.7 The environment . . . 17
  - 2.3.8 Builtins . . . 17
  - 2.3.9 Other features . . . 18
- 2.4 Code organization . . . 18
- 2.5 Software architecture . . . 19
  - 2.5.1 The command-line user interface . . . 19
  - 2.5.2 `rc`'s components . . . 21
  - 2.5.3 Trace of a simple command: `ls /` . . . 23
- 2.6 Book structure . . . 24

3 Core Data Structures **25**
- 3.1 `Token` . . . 25
- 3.2 Abstract syntax `Tree` . . . 25
- 3.3 Byte`Code` and `codebuf` . . . 27
- 3.4 `Thread` and `runq`eue . . . 28

3.5 Words and lists of words . . . 29
3.5.1 Words . . . 29
3.5.2 List of sequence of words . . . 30
3.6 Variables . . . 30

4 main() . . . 33
4.1 Overview . . . 33
4.2 Command-line arguments processing . . . 33
4.2.1 Login mode: rc -l . . . 34
4.2.2 Interactive mode: rc -i . . . 34
4.3 Initialization . . . 34
4.4 Bootstrapping bytecodes (simplified) . . . 34
4.5 Setting runq . . . 35
4.6 Setting runq->argv . . . 35
4.7 Bytecode interpreter loop . . . 36
4.8 Reading commands: Xrdcmds() . . . 36

5 Input . . . 38
5.1 Overview . . . 38
5.2 Reading a character: getnext() . . . 38
5.2.1 End-of-file management . . . 38
5.2.2 Multiple lines commands and escaped newlines . . . 39
5.2.3 Displaying the prompt . . . 39
5.3 Looking ahead: nextc() and advance() . . . 40

6 Lexing . . . 42
6.1 yylex() . . . 42
6.2 Spaces and comments ('#') . . . 43
6.3 Newlines . . . 43
6.4 Operators ('&', '&&', '|', '||', '<', '>', '$', ...) . . . 44
6.5 Quoted strings ('...') . . . 46
6.6 Keywords and identifiers (if, for, while, switch, fn, ...) . . . 47
6.7 Array subscript (<arr>(<n>)) . . . 49

7 Parsing . . . 50
7.1 Overview . . . 50
7.2 Simple commands (<cmd> <arg1>...<argn>) . . . 51
7.3 Operators . . . 52
7.3.1 Sequences (';', '&') . . . 52
7.3.2 Logical operators ('&&', '||', '!') . . . 52
7.3.3 String matching ('~') . . . 53
7.3.4 Pipe ('|') . . . 53
7.3.5 Redirections ('>', '<') . . . 53
7.4 Control flow statements (if, if not, while, switch, for) . . . 54
7.5 Functions (fn) . . . 54
7.6 Variables (<x> = ...) . . . 54
7.7 Lists ((...)) . . . 55

8 Bytecode Generation and Interpretation **56**
8.1 Overview . . . 56
8.1.1 `emitxxx()` . . . 56
8.1.2 `compile()` . . . 56
8.1.3 `outcode()` . . . 57
8.1.4 `argv` management . . . 57
8.1.5 Process status management . . . 58
8.1.6 Subprocesses management . . . 59
8.2 Simple commands . . . 60
8.2.1 Bytecode generation . . . 60
8.2.2 `Xsimple()` . . . 61
8.2.3 Fork . . . 62
8.2.4 Exec . . . 62
8.2.5 `$path` management . . . 63
8.2.6 Wait . . . 64
8.2.7 Fork optimization . . . 65
8.3 Operators . . . 65
8.3.1 Basic sequence . . . 65
8.3.2 Logical operators . . . 65
8.3.3 String matching . . . 67
8.3.4 Redirection . . . 68
8.3.5 Pipe . . . 72
8.3.6 Asynchronous execution . . . 75
8.4 Control flow statements . . . 76
8.4.1 `if` . . . 76
8.4.2 `while` . . . 78
8.4.3 `for` . . . 78
8.4.4 `switch` . . . 79
8.4.5 Blocks: `'{...}'` . . . 81
8.5 Functions . . . 81
8.5.1 Function definitions (`fn <foo> ... )` . . . 82
8.5.2 Function uses (`<foo>(...)`) . . . 83
8.6 Variables . . . 84
8.6.1 Variable definitions (`<x>=...`) . . . 84
8.6.2 Variable uses (`$<x>`) . . . 85
8.6.3 Special variables . . . 89

9 Builtins **90**
9.1 Overview . . . 90
9.2 `$ cd` . . . 91
9.3 `$ exit` . . . 93
9.4 `$ .` . . . 93
9.5 `$ eval` . . . 95
9.6 `$ wait` . . . 96

10 Environment **97**
10.1 `Updenv()` . . . 97
10.2 `Vinit()` . . . 98

11 Signals **100**

12 Initialization **103**
12.1 Actual bootstrapping code . . . 103
12.2 Intitialization script and `rc -m /path/to/rcmain` . . . 103
12.3 Actual environment . . . 104

13 Globbing **105**
13.1 Lexing globbing characters . . . 105
13.2 Expanding globbing characters . . . 105
13.3 `glob()` . . . 106
13.4 `match()` . . . 109

14 Advanced Topics TODO **114**
14.1 Reading commands from a string: `rc -c` . . . 114
14.2 Failing fast: `rc -e` . . . 114
14.3 Unicode . . . 114
14.4 Advanced constructs . . . 115
14.4.1 Subshell: `@ <cmd>` . . . 115
14.4.2 Here documents: `<< <HERE>` . . . 116
14.4.3 Read-write redirections: `<> <file>` . . . 119
14.4.4 General redirections: `>[2] <file>` . . . 120
14.4.5 Advanced dup: `>[<fd0>=<fd1>]`, `<>[<fd0>=<fd1>]`, `<[<fd0>=<fd1>]` . . . 121
14.4.6 Advanced pipes: `|[<fd>]`, `|[<fd0>=<fd1>]` . . . 122
14.4.7 Command output as a file: `'<{<cmd>}'` . . . 122
14.4.8 Command substitution: `` `{<cmd>} `` . . . 123
14.4.9 Stringification of variables: `$"<foo>` . . . 125
14.5 Advanced builtins . . . 126
14.5.1 `$ exec` . . . 126
14.5.2 `$ whatis` . . . 126
14.5.3 `$ rfork` . . . 128
14.5.4 `$ flag` . . . 129
14.5.5 `$ shift` . . . 129
14.5.6 `$ finit` . . . 130

15 Conclusion **132**

A Debugging **133**
A.1 AST dumper . . . 133
A.2 Bytecode generator trace: `rc -r` . . . 135
A.3 Printing subprocesses status: `rc -s` . . . 137
A.4 Printing commands: `rc -x` . . . 137
A.5 Printing characters: `rc -v` . . . 137
A.6 Printing all characters: `rc -V` . . . 137

B Error Management **138**

C Utilities **140**
C.1 Memory management . . . 140
C.2 Command-line arguments . . . 140
C.3 Buffered IO . . . 145
C.4 Format . . . 148
C.5 String conversions . . . 151

C.6 Misc 151

D Examples of rc scripts TODO **153**
D.1 /rc/lib/rcmain 153
D.2 /home/pad/lib/profile 153

Glossary **154**

Index **155**

References **163**

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a shell.

1.1 Motivations

Why a shell? Because I think you are a better programmer if you fully understand how things work under the hood, and the shell is the central piece of the *command-line user interface* (CLI). The shell is a thin layer around the kernel (hence its name) allowing you to run commands in a terminal.

Most users now use a *graphical user interface* (GUI) to execute programs (e.g., macOS, Microsoft Windows, X Window), but most programmers still spend a significant portion of their time in a shell to run compilation commands, editors, debuggers, or *scripts* to automate repetitive tasks. Integrated Development Environments (IDEs) can handle some of those use cases, but the shell still reigns when a programmer needs more flexibility. The power of pipes, redirections, variables, and basic control flow constructs allows sometimes in one command-line to perform tasks that would require hundreds of lines in a regular programming language[1].

There are very few books explaining how a shell works. I can cite *Advanced UNIX Programming* [Roc04], but it explains only the code of a mini-shell. This is a pity because the implementation of a real shell covers many interesting topics (e.g., programming language design, compilation, interpretation, system programming, etc.) as you will see soon in this book.

Here are a few questions I hope this book will answer:

- What is the difference between a shell and a terminal? What is the difference between a shell and a login program?
- What happens when the user type `ls` in a terminal? What is the trace of such a command through the different layers of the software stack?
- What are the main features of a shell?
- How are implemented redirections and pipes? What are the system calls involved?
- Why `ls` and `rm` are regular programs but not `cd`? Why `cd` has to be a shell builtin? What is a shell builtin?
- How does `C-c`, which interrupts a process, work? Which process receives the signal after an interruption? The shell or the command ran from the shell?

[1] For example, [BKM86] contrasts writing a program to count words in Pascal and in a shell.

1.2 The Plan 9 shell: rc

I will explain in this book the code of the Plan 9 shell rc [Duf90][2] (for Run Commands), which contains about 6700 lines of code (LOC). rc is written mostly in C, with its parser using also Yacc [Joh79].

Like for most books in Principia Softwarica, I chose a Plan 9 program because those programs are simple, small, elegant, open source, and they form together a coherent set. Moreover, rc is arguably simpler to use and to understand than sh [Bou79], the UNIX shell, or any of its derivatives (e.g., bash [Ram94], the most popular shell under Linux). For example, by treating the content of any variables uniformly as a list of strings, rc does not need the extra operator $@ used in sh ($* is enough). Moreover, The syntax of rc, specified formally and succintely by a small grammar, is also easier to learn than the (unspecified) syntax of sh, partly because rc is inspired by the syntax of C with its curly braces (sh, instead, is using multiple keywords inspired by Algol).

rc itself lacks many of the interactive features found in other shells (e.g., bash) such as filename completion, command-line editing, job control, etc. This is partly because under Plan 9, the terminal of the windowing system rio is providing instead those features (see the WINDOWS book [Pad16b]).

1.3 Other shells

Here are a few shells that I considered for this book, but which I ultimately discarded:

- The first UNIX shell, originally called sh, was written by Ken Thompson, the original author of the UNIX kernel. sh started as an assembly program in UNIX V1[3] and finished as a C program in UNIX V6[4]. Ken Thompson's shell introduced the pipe ('|'), redirections ('>' and '<'), as well as wildcard matching[5] ('*'). The syntax of those features remained the same in all subsequent shells. Ken Thompson's shell contains only 899 LOC, but it is just a basic command interpreter, not a full scripting language like rc.

- The Bourne shell [Bou79], also called sh, superseded Ken Thompson's shell (which was renamed osh) in UNIX V7[6]. It was written by Stephen Bourne and contains 4145 LOC of C. It is arguably the most famous shell because it defined first the main features that we expect now from a shell: the ability to run commands with pipes and redirections, but also the use of variables and control flow constructs to write scripts. The Bourne shell is both an interactive command interpreter and a full programming language. Most subsequent shells tried to remain backward compatible with the Bourne shell.

  The syntax of the Bourne shell was inspired by Algol with pair of keywords (e.g., begin/end, do/done, case/esac) instead of the curly braces of C[7]. The code of sh is smaller than the code of rc, but it is also harder to understand. as in rc, but instead a complex recursive descent parser.

- Bash [Ram94, New95][8] (for Bourne-Again Shell) is an open-source shell similar to the Bourne shell (hence its name) from the GNU project[9]. It is the default shell in most Linux distributions. It provides many interactive features not found in the Bourne shell such as filename completion, command-line editing, interactive history, job control, etc. Many of those features were partly inspired by the C shell [Joy86], an older shell originating in BSD UNIX using a syntax more similar to C.

[2] See http://plan9.bell-labs.com/magic/man2html/1/rc for its manual page.
[3] http://minnie.tuhs.org/cgi-bin/utree.pl?file=V1/sh.s
[4] http://minnie.tuhs.org/cgi-bin/utree.pl?file=V6/usr/source/s2/sh.c
[5] Also known as globbing.
[6] http://minnie.tuhs.org/cgi-bin/utree.pl?file=V7/usr/src/cmd/sh/
[7] Stephen Bourne was such as fan of Algol that the C source code of sh itself looks like Algol, thanks to macros such as THEN, BEGIN, END, etc.
[8] https://www.gnu.org/software/bash/
[9] http://www.gnu.org

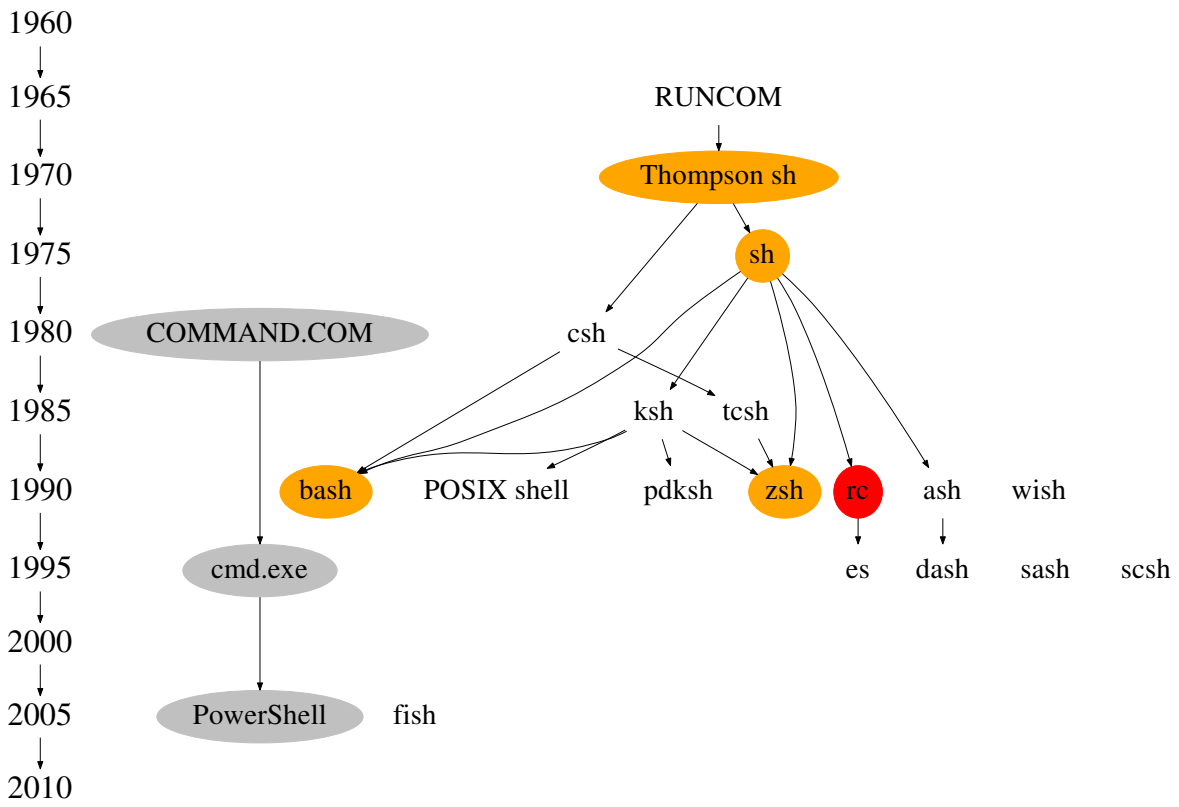

Figure 1.1: Shells timeline

Bash relies on the GNU Readline library[10] for many of those interactive features (`rc` relies instead on `rio`'s terminal to provide similar features). However, the codebase of Bash is very large with more than 100 000 LOC (not including the code of the Readline library, which would add another 33 000 LOC), which is more than an order of magnitude more code than in `rc`. Bash's grammar file `parse.y` contains alone 6268 LOC.

- The Z shell [Kid05][11] is another open-source shell popular among advanced Linux users. It is extensible through plugins and themes[12]. It contains most of the features found in other shells (e.g., Bash, the C Shell, the Korn Shell [BK89, Kor94]) and introduced many new interactive features such as programmable completion, recursive wildcarding with `**/*` (eliminating the need for the program `find`), and more. However, all those features come at a price: the code of the Z shell contains more than 145 000 LOC (not including the tests).

Figure 1.1 presents a timeline of major UNIX shells (and a few non-UNIX shells). I think `rc` represents the best compromise for this book: it implements the essential features of a shell while still having a small and understandable codebase (6700 LOC).

1.4 Getting started

To play with `rc`, you will first need to install the Plan 9 fork used in Principia Softwarica (see http://www.principia-softwarica.org/wiki/Install). Once installed, you do not need to do anything to run `rc` because

[10] https://tiswww.case.edu/php/chet/readline/rltop.html
[11] http://www.zsh.org/
[12] See https://github.com/robbyrussell/oh-my-zsh for a large repository of such contributions.

it is the program started by default by the kernel (see the KERNEL book [Pad14]). Once the kernel finished to boot, you should see a percent sign, called the *prompt*, after which you can type any commands as in the following:

```
1   % ls /
2   bin
3   boot
4   ...
5   srv
6   % rc -help
7   Usage: rc [-SsrdiIlxepvV] [-c arg] [-m command] [file [arg...]]
8   % rc
9   % prompt='$ '
10  $ ls /
11  bin
12  boot
13  ...
14  srv
15  $ exit
16  %
```

Line 1 runs the program `ls` to list the content of the root directory. Line 6 and Line 8 show that `rc` is a regular program (just like `ls` at Line 1): you can run a shell program under a shell. Line 8 and Line 9 seem to indicate that typing `rc` has no effect, but the percent sign shown at the beginning of Line 9 is in fact displayed by the `rc` process started by Line 8, not the original `rc` process started by the kernel. Line 9 modifies the *special variable* `prompt` (see Section 8.6.3 for a list of those special variables) to better differentiate the two shell processes. Indeed, Line 10 shows that `'$ '` is the new prompt replacing the percent sign. Finally, Line 15 shows the use of the *builtin* `exit` (see Chapter 9 for more information on builtins) to exit from the shell. Doing so goes back to the preceding shell process (the one launched by the kernel) with the original percent prompt at Line 16.

1.5 Requirements

Because most of this book is made of C source code, you will need a good knowledge of the C programming language [KR88] to understand it. To understand Chapter 7, you will also need to know Yacc [BLM92].

Note that this book is not an introduction to the shell or to shell scripting. I assume you are already familiar with at least one shell, for instance a derivative of `sh` such as `bash`, and so are familiar with concepts such as a pipe, a redirection, what `#!` means, or what a script is. If not, I suggest you to read either the book introducing the UNIX programming environment [KP84], or the original `sh` tutorial [Bou79], or any more recent books on shell scripting [Mic08, New95, Kid05].

A shell is a thin layer on top of a kernel, and so a shell relies heavily on the services offered by the kernel: system calls (e.g., `rfork()` `exec()`, `wait()`, `chdir()`), but also device files (e.g., `/dev/cons` to read and write characters on the terminal). Thus, it can be useful to know how the Plan 9 kernel works (see the KERNEL book [Pad14]), or at least be familiar with sytem programming under UNIX [Roc04, Ste94], to fully understand some of the code in this book.

A shell is also a full programming language, and as you will see soon, `rc` is internally both a compiler and a bytecode interpreter. I assume you also have a basic understanding of how a compiler works, and for example that you know what a lexer or parser is (see the COMPILER book [Pad16a]).

If, while reading this book, you have specific questions on the syntax and interface of `rc`, I suggest you to consult the man page of `rc` at `docs/man/1/rc` in my Plan 9 repository.

Note that the `shells/docs/` directory in my Plan 9 repository contains documents describing either `rc` [Duf00] or `sh` [Bou15]. Those documents are useful to understand some of the design decisions presented in this book, especially how and why `rc` differs from `sh`.

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the author of `rc`, Tom Duff, who wrote in some sense most of this book.

Chapter 2

Overview

Before showing the source code of `rc` in the following chapters, I first give an overview in this chapter of the general features of a shell. I also quickly describe the command-line interface of `rc` and the specific language supported by `rc`. I also define terms, explain how the code is organized, and more generally give the background necessary to understand the code I will show later.

2.1 Essential shell features

The following sections will explain the essential features of a shell. A shell is a strange beast: it must be both an interactive command interpreter, allowing to run commands easily on one line in a terminal, and a proper programming language, allowing to write complex script in a file.

2.1.1 Running commands

The first job of a shell is to allow the user to run commands[1]. Most shells offer a minimalist syntax for running commands, so we can type those commands quickly. Indeed, contrast the shell command `ls /` with the equivalent C program:

⟨`lsroot.c` 12⟩≡
```
#include <u.h>
#include <libc.h>

void main(){
  char* args[] = {"/"};
  exec("/bin/ls", args);
}
```

Here are a few notes on the minimalist syntax used by shells:

- *No Parenthesis:* to call a program, you do not need any parenthesis; just type the program name next to its arguments.
- *No quotes:* to pass arguments to the program, you usually do not need any quotes or commas; just type the arguments separated by space. For example, `foo`, `--help`, `42`, `/a/b/c` are all valid arguments[2]. The only time you need to enclose an argument in a quote is when you want to use one of the special character used by the shell (e.g., `'#'`, `'$'`, `'&'`, etc. with `rc`), also known as *meta-characters*.

[1] The very first shell, which was written for the CTSS operating system in 1964, was called `RUNCOM`[Pou00].

[2] You can even sometimes avoid to specify fully all the arguments by using shortcuts, for example, wildcards as explained in Chapter 13.

- *No full path:* to call a program, you do not need to specify its path in the filesystem; just type the name of the program and the shell will automatically find its location. Under UNIX, the `PATH` environment variable stores the candidate locations to find programs.

- *No special ending character:* to finish your command, you do not need any special character like a semicolon; just type a newline and the shell will start interpreting your command[3].

- *No types:* to enter a command, you do not need to specify any types such as `char*` as in C. The arguments are always strings.

Once the command you ran finished, the shell gets back in control and displays another *prompt* to indicate that you can type another command.

2.1.2 Redirections

The second important feature of a shell is to allow to *redirect* the output and input of a program, as in `ls / > listing.txt`.

Many programs (e.g., `ls`, `find`, `rm`, `grep`) live in the command-line world and simply use text for their input and output. By using redirections, you can easily save the output of those programs in a file, or use a previously saved file as input for those programs[4], without even changing the code of those programs. In fact, because under UNIX and Plan 9 "everything is a file", including devices, you can use the same program in many different ways. For example, you can print the listing of the root directory by simply typing `ls / > /dev/printer`.

2.1.3 Pipes

Because of the universality of plain text, you can easily combine many command-line programs. For example, you can list all the C files in your home directory by combining `find` and `grep` with the two commands `find /home/pad/ > /tmp/list.txt` and `grep '\.c$' < /tmp/list.txt`. In fact, thanks to another great feature of the shell, the *pipe*, you can just use one command: `find /home/pad | grep '\.c$'`. This is not only shorter, but also more efficient, and it gives results more quickly. You can even use multiple pipes in the same command as in `find | grep '\.c$' | xargs cat | grep foo`.

Pipes allow to combine easily full programs, just like you can combine functions in a functional language. Pipes are one of the greatest innovations introduced by UNIX, and one of the first programming construct allowing a form of *component-oriented programming*. Each program can be treated as a component, which can be combined with other components. You do not program at the granularity of functions but at the higher granularity of programs.

2.1.4 Storing commands in a script

Just like you can type a series of commands separated by newlines in a terminal, you can save those same commands in a file, a *script*, and execute this script with the shell. As the author of the first shell said [Pou00]: "commands should be usable as building blocks for writing more commands, just like subroutine libraries".

Scripts allow to automate easily repetitive tasks. Moreover, because scripts are commands themselves, they can also be combined, leading to more forms of components-oriented programming.

[3] You can also enter commands on multiple lines, which requires to *escape* the newline as explained in Section 5.2.2.

[4] `rc` allows to redirect not just the standard input and output, as explained in Section 14.4.4.

2.1.5 A shell as a scripting language

Once you can write a sequence of commands in a script, you quickly want more, such as the ability to use conditionals or loops. Thus, most shells are also full programming languages, often referred as *scripting languages*, with *variables* and many *control flow* constructs, and even function definitions. A scripting language acts like a glue, allowing to combine many programs together.

Because a scripting language must also support the basic commands you type on the command-line, it shares many characteristics with those basic commands: a scripting language does not use types and is interpreted.

A scripting language operates mainly on strings. Because the string arguments you type on the command-line do not usually require any quotes, the use of variables requires then a special operator. Most shells use `'$'` as a prefix to differentiate variables from regular string arguments, for example, `find $home`.

2.2 `rc` command-line interface

I just described the main features of a shell. I will now focus exclusively on `rc` and give more details about its command-line interface.

It is rare to run `rc` itself on the command-line, like you run `ls`, `cp`, `grep`, etc. After all, if you have a command-line, you already are in a shell. However, as I said in Section 1.4, the shell under UNIX (and Plan 9) is a regular program. You can run a shell under a shell, which can be useful for example if you do not like the default shell when you log-in[5]. To run `rc`, simply type `rc` on the command-line without any arguments, as I did in Section 1.4 Line 8.

You can also pass to `rc` the name of a script as an argument , as well as the arguments of this script, for example, `rc foo.rc arg1 arg2`. However, this is usually not necessary because most scripts use `#!/bin/rc` at the beginning, which is recognized by the kernel (see the KERNEL book [Pad14]).

Here is the full command-line interface of `rc`:

```
% rc -help
Usage: rc [-SsrdiIlxepvV] [-c arg] [-m command] [file [arg...]]
```

The `-c` flag allows to execute commands from a string passed as an argument instead of from the content of a script (see Section 14.1 for the code handling `rc -c`).

The `-m` flag allows to specify the initialization script of `rc`. I will explain fully the complex initialization process of `rc` and the code of `rc -m` in Chapter 12.

Finally, `rc` supports a few options to provide advanced features or to help debug `rc` itself. I will present gradually those options in this book.

2.3 `boot.rc`

In this section, I will present an example of an `rc` script showing a few features of `rc`'s scripting language. The goal here is to illustrate the general features of a shell you have seen in Section 2.1 with the concrete syntax of `rc`'s scripting language. This will also help you understand the grammar of `rc` I will present in Chapter 7. Finally, this example will introduce a few concepts specifics to `rc` that are useful to have in mind while I will explain the code of `rc` in the rest of the document.

`boot.rc`, below, is the first program executed by the kernel when running on an ARM machine (see the KERNEL book [Pad14]). The following sections will explain the main features of `rc` used in this script.

⟨*kernel/init/user/boot/arm/boot.rc* 14⟩≡

```
#!/boot/rc -m /boot/rcmain
```

[5]Under UNIX, you can also use the special program `chsh` to change your login shell.

```
/boot/echo booooooooting...

path=(/bin /boot)

# basic devices
bind -c '#e' /env
...

# storage
bind -a '#S' /dev
fdisk -p /dev/sdM0/data >/dev/sdM0/ctl
dossrv
mount -c /srv/dos /root /dev/sdM0/dos
bind -a -c /root /

bind -a /arm/bin /bin
bind -a /rc/bin /bin
...

# to use 5c, 5a, 5l by default in mk
objtype=arm
...

exec /boot/rc -m /boot/rcmain -i
```

2.3.1 #!

```
%dup:
% mentioned before?

% shebang! handled by kernel
%history: \unix V8 started to handle that, before was handled by sh itself
% but restricted because you could not exec() a shell script, so it was
% working only when the script was run from a shell

% scripts as regular commands! can become filter. components,
% building blocks!
% Before had to prefix command file with shell name (rc foo)

%\n and comment (said later)
```

2.3.2 Initialization script

```
% : {\tt{}rc -m}

% -m, /boot/ because default is /xxx and not accessible at boot time


unix: .profile, .login, etc.
```

12 D.1

```
% execute program qualify full, universal REPL
%  also to echo things need external program! no special support in rc
```

2.3.3 Variables

```
% variables list with () and separated by space
```

2.3.4 $path

```
%  and special path variable

% plan9 support union dirs, so path usually small compared
% to Linux. Also no need /usr/bin/env ugly intermediate for #!
```

8.2.5

2.3.5 Comments

```
% so for #! when pass file as argument,
%  parse file and skip this line because comment
```

2.3.6 Quoting and escaping

```
% quote for # otherwise comment, escaping
% also ^?
```

6.5

2.3.7 The environment

```
% variables exported to process launch from rc, so objtype
%  convenient for mk
```

10

2.3.8 Builtins

```
% cant be command because modify things in current shell process.
% ex: chdir. As you will see soon, when execute commands,
% fork/exec, so chdir in child will have no effect on further commands.
% chdir modify state of shell!

% exec here is to replace current process by another, here same
% rc but with -i for interactive.

% exec builtin et -i for interactive!
% (not needed otherwise because detect Isatty, not
```

```
%  sure why we need it here
```

9

2.3.9 Other features

```
% pipe, redirection seen before.
% control flow construct, shown before too.

% async, globbing (wildcard),
% Present gradually in grammar chapter X
```

2.4 Code organization

Table 2.1 presents short descriptions of the source files of `rc`, together with the main entities (e.g., structures, functions, globals) the file defines, and the corresponding chapter in this document in which the code contained in the file is primarily discussed.

⟨*includes* 18⟩≡ (? 0—1)

```
#include "rc.h"
#include "exec.h"
#include "fns.h"
#include "getflags.h"
#include "io.h"
```

Function	Ch.	File	Entities	LOC
main data structures	3	`rc.h`	`Tree`[??] `Code`[25b] `Word`[??] `Var`[45g]	
execution data structures	3	`exec.h`	`Thread`[28c] `List`[30c]	
AST helpers	3	`tree.c`	`newtree()`[26d] `tree1()`[26f] `tree2()`[27a]	
globals	3	`globals.c`	`codebuf`[27d] `runq`[28d]	
list of strings (words)	3	`words.c`	`newword()`[29b] `count()`[29c] `freewords()`[30a]	
shell variables	3	`var.c`	`setvar()`[31b] `vlook()`[31c] `gvar`[31e] `gvlook()`[31f]	
function prototypes	3	`fns.h`		
entry point	4	`main.c`	`main()`[34b]	
execution utilities	4	`executils.c`	`start()`[35b] `pushlist()`[35e] `pushword()`[36a]	
character input and prompt	5	`input.c`	`getnext()`[38a] `pprompt()`[40b] `nextc()`[40g]	
lexer	6	`lex.c`	`yylex()`[42a]	
parser	7	`syn.y`	`yyparse()`[50b]	
bytecode generation	8	`code.c`	`emitf()`[56d] `compile()`[56g] `outcode()`[57e]	
process status	8	`status.c`	`setstatus()`[58f] `getstatus()`[59a]	
simple command bytecode	8	`simple.c`	`Xsimple()`[61a] `execexec()`[62c] `doredir()`[69a]	
`$path` management	8	`path.c`	`searchpath()`[63b]	
process-related bytecodes	8	`processes.c`	`execforkexec()`[62b] `Xpipe()`[124] `Xasync()`[75b]	
other bytecodes	8	`exec.c`	`Xtrue()`[66d] `Xjump()`[78c] `Xfn()`[82d]	
shell builtins	9	`builtins.c`	`execcd()`[91b] `execexit()`[93a] `execdot()`[93b]	
shell environment	10	`env.c`	`addenv()`[97e] `Vinit()`[98]	
wildcard matching	13	`glob.c`	`glob()`[106e] `match()`[109d]	
signal management	11	`trap.c`	`dotrap()`[101c]	
here documents	14	`here.c`	`readhere()`[116h] `heredoc()`[118b]	
AST dumper	A	`pcmd.c`	`pcmd()`[133e]	
bytecode dumper	A	`pfnc.c`	`pfnc()`[136]	
error management	B	`error.c`	`panic()`[139b]	
memory management	C	`utils.c`	`emalloc()`[140b] `efree()`[140d]	
command-line flags	C	`getflags.c`	`getflags()`[141h] `usage()`[143]	
buffered IO	C	`io.c`	`openfd()`[145f] `rchr()`[146c]	
pretty printer	C	`fmt.c`	`pfmt()`[148b]	
Total				6800

Table 2.1: Chapters and associated `rc` source files.

2.5 Software architecture

2.5.1 The command-line user interface

The shell is only one component of the command-line user interface (CLI). Figure 2.1 shows the components supporting the CLI under Plan 9 (the components for the CLI under UNIX are very similar).

To run commands, the shell needs first a kernel to create processes. The kernel provides services to applications (including the shell) through its *system call interface* (also known as *syscalls*). For example, under Plan 9, `rfork()` creates a new process in which the shell can then `exec()` a program (see KERNEL book [Pad14]).

```
filesystem          +---------------+ +-------------------+
 interface          |               | |                   |  A
|-- bin             |     Shell     | |command (e.g., ls) |  P
|   |-- cp  +-------+-> stdin       | |      stdin        |  P
|   |-- ls  |      +-+---stdout     | |      stdout       |  S
|   |-- rc  |      | +---------------+ +-------------------+
|   `-- ... |      |------------syscall interface------------
|-- boot    |      | +-------------------------------------+
|-- dev     |      | |               Kernel                |
|   |--  cons <----+ |                                     |  K
|   |         < - - +|  device drivers-+                   |  E
|   |--  draw        |  |              |                   |  R
|   |--  mouse      +|- + >console     |                   |  N
|   `--  ...         |  |              |                   |  E
|-- env           + -|- + -display     |                   |  L
|-- sys              |  |  keyboard- - |                   |
`-- ...           |  |  |  mouse - - -|+ - - - - - - +     |
                   |  |              |                   |
                  |  |  +-------------++            |     |
                   +-------------------------------------+
                  v                   |              |        D
            + - - - - - - -                                   E
                            |         v              v        V
            |   Monitor      + - - - - -      + - - - - -    I
                            |   keyboard |        mouse   |   C
            + - - - - - - -  + - - - - -      + - - - - -    E
                        Terminal                              S
                       (console)
```

Figure 2.1: Components of the command-line user interface under Plan 9.

A shell needs also device drivers in the kernel handling the terminal with its keyboard and monitor. To access those devices, the kernel provides a *filesystem interface* (called a *namespace* under Plan 9). An application can `open()`, `read()`, `write()`, or `close()` files in this filesystem. Under UNIX and Plan 9, devices are represented as files. For example, `/dev/cons` (for console) is a file representing the terminal. To read characters from the keyboard, an application can simply read characters from `/dev/cons`. To write characters on the monitor, an application can simply write characters in `/dev/cons`.

```
%\t stdin stdout regular program
%plan9:
%\t under \plan kernel run rc (see \book{Kernel} and Section{boot.rc}).
%unix:
%\t Under \unix run init, then login, then shell in terminal. Even virtual
%\t  terminal in Linux,
%\t under windowing system like rio also virtual terminals connected
%\t stdin and stdout of process (see \book{Windows}).
% Figure X presents archi for plan9.

% shell = stuff around a kernel in a coquillage.
% history: multics.
% A shell is a bastard. See es paper quote in abstract
%  that reference unix programmnig environment book

% separate prog from kernel, can change shell!
%history: originated in Multics (one of the 2 things Thompson liked about
% Multics, the other being the hierarchical filesystem)

% for a long time actually there was no (external) shell? Shell was
% part of kernel? With Unix it changed and shell can be a regular user program.

% for interactive use, and for programming. Both uses,
% and different features for those 2 things.

% regular program
% backspace handled by kerne
% line editing not shell
% \footnote not libreadline raw mode, completion
```

2.5.2 `rc`'s components

Figure 2.2 describes the main data flow of `rc`, whereas Figure 2.3 describes the main control flow of `rc`. At its core, `rc` is both a compiler and a bytecode interpreter. As shown by Figure 2.2, given a series of characters (coming either from what you typed in the terminal or from the content of a script), `rc` first groups those characters in tokens (with `yylex()`[42a]). Then, `rc` parses those tokens (with `yyparse()`[50b]) and builds an abstract syntax tree (AST) of the program (see `Tree`[??]). Finally, `rc` transforms this tree in a series of bytecodes (see `Code`[25b]). This is similar to what a compiler such as `5c` does (see COMPILER book [Pad16a]), except a bytecode here is not an instruction from a concrete machine but from a *virtual machine*.

After this compilation, `rc` goes through the series of bytecodes and interprets them. `rc` keeps track of what is currently executing in a `Thread`[28c] data structure and in a queue `runq`[28d]. This data structure is called a

```
Input characters --> tokens --> AST --> bytecodes --> threads --> processes
```

Figure 2.2: Data flow diagram of `rc`.

```
                                +------------+
                                |    main    |
                                +-+----+-----+
                                  |    |
                     .------------+----v----------------.
                    (      bytecod| interpreter loop      ) (interpreting)
                     `------------+----+----------------'
                                  |    |
                +------------------++---+-------------+--------------------+
                |        +---------+|                 |                    |
     +------v-----+  |   +------v-----+   +-------v----+        +------v-----+
     |  Xrdcmds   |  |   |   Xpipe    |   |  Xsimple   |   ...  |    Xif     | bytecodes
     +-----+------+  |   +--+---------+   +--+-----+---+|       +------------+
           ||        |      |   |            |     |    |
           |+--------+---+  |   | +----------v-+   |   +v-----------+
     +-----+         |   |  |   | |execforkexec|  .|.  |   execcd   | shell builtins
     |            +--v---v--v--+| +------------+   |   +----------+-+
     |            |   start    ||        |         |              |
     v            +------------+|        |         |              |
+---------+                     |        |         |              |
| yyparse | (parsing)   +-------+-+      |         +----+         |
+----+-+--+             |         |      |              |         |
     | +-------+        |         |  +---+----+         |         |
+----v--+  +---v-----+  |         |  |        |         |         |
| yylex |  | compile |  |         |  |        |         |         |
+----+--+  +---------+  |         |  |        |         |         |
     |     (compiling)  |         |  |        |         |         |
+----v--+               |         |  |        |         |         |
| nextc |               v         v  v        v         v         v
+-------+        +--------+  +--------+  +--------+  +--------+  +--------+
                 |  pipe  |  |  fork  |  |  exec  |  |  wait  |  | chdir  |  syscalls
                 +--------+  +--------+  +--------+  +--------+  +--------+
```

Figure 2.3: Control flow diagram of `rc`.

"thread" because `rc` must sometimes manage multiple threads of execution. Indeed, some of the bytecodes can create new processes running concurrently (e.g., `Xpipe()`[124], the bytecode handling pipes).

In fact, `rc` does not start by compiling, but by interpreting. Indeed, `rc` starts first by interpreting some special bytecodes, called the *bootstrap*, that contains a bytecode (`Xrdcmds()`[36c]) that then triggers the compiler. I will now explain briefly the control flow of `rc`, starting from the top of Figure 2.3.

After some basic initializations, `main()`[34b] sets its local `bootstrap`[33a] to contain the initial set of bytecodes to execute. It then calls `start()`[35b] with `bootstrap` as an argument, to start interpreting this series of bytecodes. Internally, `start()` just modifies the global `runq` to point to a newly created `Thread`, and sets the field `Thread.code`[28c] to the content of `bootstrap`. After `start()` returns, `main()` goes in a loop that interprets the

bytecodes in `runq->code`.

The most important bytecode in `bootstrap` is `Xrdcmds()`, which reads a command (hence its name) and starts a new `Thread`. `Xrdcmds()` first calls the parser `yyparse()`, which calls the lexer `yylex()`, which calls the input routine `nextc()`[40g] to get the next character. By default, `nextc()` reads characters from the standard input, and so it waits for the characters you type in a terminal. Once you finished to enter a command with a newline, `yyparse()` will call `compile()`[56g] with the tree it built during parsing as an argument. Internally, `compile()` modifies the global `codebuf`[27d] to store the bytecodes deriving from the tree. Then, after `yyparse()` returned, `Xrdcmds()` calls `start()` to start a new thread with `codebuf` as a parameter. `start()` then modifies again the global `runq`, and when `Xrdcmds()` returns, the main bytecode interpreter loop will process a new series of bytecodes stored in `runq.code`.

Note that before modifying `runq`, `start()` first links the newly created thread with the old thread (through the `Thread.ret`[28e] field). Moreover,`compile()` adds the bytecode `Xreturn()`[73b] at the end of the series of bytecodes deriving from your command. Thus, after `main()` finished interpreting the bytecodes of your command, it will process `Xreturn()`, which will modify `runq` to point to the old thread, the one containing the bootstrap bytecodes. Then, after `Xreturn()` returns, the main bytecode interpreter loop will process again the bytecodes from the bootstrap, which will read another command through `Xrdcmds()`.

In `rc`, the bytecodes are represented by regular C functions starting by convention with an `X` (e.g., `Xrdcmds()`, `Xpipe()`, `Xif()`[76c]). Thus, the bytecode interpreter is mainly a function dispatcher. Internally those functions perform system calls to the kernel to create a pipe (`pipe()`), to fork a new process (`fork()`), to wait for a child process (`wait()`), or to change directory (`chdir()`). An important bytecode is `Xsimple()`[61a], which `rc` uses to run a "simple" command. `Xsimple()` represents the essence of a shell: with the series of system calls `fork()`, `exec()`, and `wait()`, `rc` can run a command in a new process and wait for its termination (or interruption). `Xsimple()` is also responsible for managing the multiple shell builtins. Indeed, if the name of the "simple" command is a builtin (e.g., `cd`), then `Xsimple()` dispatches the appropriate function (e.g., `execcd`) instead of forking a new process.

2.5.3 Trace of a simple command: `ls /`

You can see the bytecodes and the threads created internally by `rc` by running `rc` with the `-r` flag. Here is an example of a trace of the simple command `ls /`:

```
% rc -r
pid 39 cycle 0002D930 1 Xmark ()
...
pid 38 cycle 0001984C 9 Xrdcmds
% ls /
pid 38 cycle 0002CBD0 1 Xmark
pid 38 cycle 0002CBD0 2 Xword ()
pid 38 cycle 0002CBD0 4 Xword (/)
pid 38 cycle 0002CBD0 6 Xsimple (ls /)
bin
boot
...
srv
pid 38 cycle 0002CBD0 7 Xreturn
pid 38 cycle 0001984C 9 Xrdcmds
```

It is not important to fully understand the format of this trace (see Section A.2 for the full explanation and for the code handling `rc -r`), but you can recognize a few of the bytecodes I mentioned before: `Xrdcmds()`[36c], `Xsimple()`[61a], and `Xreturn()`[73b]. I will explain `Xmark()`[58a] and `Xword()`[57f] later in this document.

```
% It is interesting to understand the full trace of a simple command
% through the multiple layers of the software stack.
% Here is again the trace of the command {\tt{}ls /<newline>}
% entered in a terminal:

%\t full trace ls\n
\begin{enumerate}
\item After {\tt{}rc} displayed its prompt,
%\t 1 shell read(), sysread, connected to /dev/cons, consread
%\t blocked (see \book{kernel})

%\t item l

%\t item s space /

%\t item newline
\end{enumerate}
```

2.6 Book structure

You now have enough background to understand the source code of `rc`. The rest of the book is organized as follows. I will start by describing the core data structures of `rc` in Chapter 3. Then, I will use a top-down approach, starting with Chapter 4 with the description of `main()`[34b], the initialization of `rc`, the bytecode interpreter loop, and the function `Xrdcmds()`[36c]. The following chapters will describe the main components of the compiler pipeline: Chapter 5 will present the input routines, Chapter 6 the lexer, Chapter 7 the parser, and finally Chapter 8 the bytecode generator and bytecode interpreter for the main features of `rc`. In Chapter 9, I will present the code of the different shell builtins (e.g., `execcd()`[91b] for `cd`). Chapter 10 contains the code to manage the environment of the processes launched from the shell, and Chapter 11 the code to manage the signals sent to the processes (also known as *notes* under Plan 9). Chapter 12 presents the actual code initializing `rc`. Indeed, Chapter 4 presents only a simplified initialization to not introduce too much complexity early-on. Then, I will present advanced features of `rc` that I did not present before to simplify the explanations, for instance, wildcard matching (e.g., `ls *.c`) in Chapter 13, or command substitutions (e.g., `` `{ls} ``) in Chapter 14. Those advanced features tend to crosscut many components of `rc` with extensions to the lexer, the parser, the bytecode generator, and the bytecode interpreter. Finally, Chapter 15 concludes and gives pointers to other books in the Principia Softwarica series.

Some appendices present the code of non-functional properties: code to help debug `rc` itself in Appendix A and code to manage errors in Appendix B. Appendix C contains the code of utility functions used by `rc` but that are not specific to `rc` (e.g., a library to manage string buffers). Finally, Appendix D presents examples of `rc` scripts.

Chapter 3

Core Data Structures

Show me your code and conceal your data structures, and I shall continue to be mystified. Show me your data structures, and I won't usually need your code; it'll be obvious.

Fred Brooks

In this chapter, I will present the core data structures of `rc`. Most of those data structures are listed in Figure 2.2, which presented the data flow of `rc`. The first three sections will present the data structures of the compiler part of `rc`: the token, the abstract syntax tree, and the bytecode. The bytecode is a data structure used both by the compiler and interpreter. The following sections will present the remaining data structures of the interpreter part of `rc`: the thread, and the variable.

3.1 Token

⟨*token declarations* 25a⟩≡ (50a) 26a ▷

```
%token FOR IN WHILE IF NOT SWITCH FN
%token TWIDDLE BANG  /** ~ ! */
%token REDIR PIPE /** {>, <, <<, >>} | */
%token ANDAND OROR /** && || */
%token COUNT SUB /** $# ( */
%token WORD /** anything else (e.g. foo, --help, 42, /a/b/c, etc) */
```

3.2 Abstract syntax Tree

⟨*struct* Tree 25b⟩≡ (? 0—1)

```
struct Tree {

    // either<enum<Token_kind>, char>
    int type;
    // string of the token or AST dump of the whole subtree for certain nodes
    char *str;

    // array<option<ref_own<Tree>>
    tree *child[3];

    ⟨Tree redirection and pipe specific fields 45d⟩
    ⟨Tree word specific fields 47a⟩

    // Extra
    ⟨Tree extra fields 26c⟩
};
```

⟨*token declarations* 26a⟩+≡ (50a) ◁25a 54d▷

```
/* not used in syntax */
%token SIMPLE
%token ARGLIST WORDS
%token BRACE PAREN
```

⟨*global* `treenodes` 26b⟩≡ (? 0—1)

```
// list<ref_own<Tree>> (next = Tree.next)
tree *treenodes;
```

⟨`Tree` *extra fields* 26c⟩≡ (25b)

```
tree *next;
```

⟨*function* `newtree` 26d⟩≡ (? 0—1)

```
/*
 * create and clear a new tree node, and add it
 * to the node list.
 */
tree*
newtree(void)
{
    tree *t = new(tree);
    t->str = nil;
    t->child[0] = t->child[1] = t->child[2] = nil;

    // add_list(t, treenodes)
    t->next = treenodes;
    treenodes = t;

    return t;
}
```

Uses `nil` and `treenodes` 26b.

⟨*function* `freenodes` 26e⟩≡ (? 0—1)

```
void
freenodes(void)
{
    tree *t, *u;
    for(t = treenodes;t;t = u){
        u = t->next;
        if(t->str)
            efree(t->str);
        efree((char *)t);
    }
    treenodes = nil;
}
```

Uses `efree()`, `nil`, and `treenodes` 26b.

⟨*function* `tree1` 26f⟩≡ (? 0—1)

```
tree*
tree1(int type, tree *c0)
{
    return tree3(type, c0, (tree *)nil, (tree *)nil);
}
```

Uses `nil` and `tree3()` 27b.

⟨*function* tree2 27a⟩≡ (? 0—1)

```
//@Scheck: used by syn.y
tree* tree2(int type, tree *c0, tree *c1)
{
    return tree3(type, c0, c1, (tree *)nil);
}
```

Uses nil and tree3() 27b.

⟨*function* tree3 27b⟩≡ (? 0—1)

```
tree*
tree3(int type, tree *c0, tree *c1, tree *c2)
{
    tree *t;

    ⟨tree3 if some empty sequence 27c⟩
    // else
    t = newtree();
    t->type = type;
    t->child[0] = c0;
    t->child[1] = c1;
    t->child[2] = c2;
    return t;
}
```

Uses newtree() 26d.

⟨tree3 *if some empty sequence* 27c⟩≡ (27b)

```
if(type==';'){
    if(c0==nil)
        return c1;
    if(c1==nil)
        return c0;
}
```

Uses nil.

3.3 ByteCode and codebuf

⟨*global* codebuf 27d⟩≡ (? 0—1)

```
// growing_array<ref_own<Code>>
code *codebuf;    /* compiler output */
```

⟨*struct* Code 27e⟩≡ (? 0—1)

```
/*
 * The first word of any code vector is a reference count.
 * Always create a new reference to a code vector by calling codecopy(.).
 * Always call codefree(.) when deleting a reference.
 */
union Code {
    void (*f)(void); // Xxxx() bytecode
    int i;
    char *s;
};
```

⟨*function* codecopy 27f⟩≡ (? 0—1)

```
code*
codecopy(code *cp)
{
    cp[0].i++;
    return cp;
}
```

⟨*function* codefree 28a⟩≡ (? 0—1)

```
void
codefree(code *cp)
{
    code *p;
    // check ref count
    if(--cp[0].i != 0)
        return;

    for(p = cp+1; p->f; p++){
        ⟨codefree() in loop over code cp, switch bytecode cases 28b⟩
    }
    efree((char *)cp);
}
```

Uses efree().

⟨codefree() *in loop over code* cp, *switch bytecode cases* 28b⟩≡ (28a) 60g ▷

```
if(p->f==Xfalse || p->f==Xtrue
|| p->f==Xread || p->f==Xwrite || p->f==Xrdwr
|| p->f==Xappend || p->f==Xclose
|| p->f==Xasync || p->f==Xbackq || p->f==Xcase
|| p->f==Xfor || p->f==Xjump
|| p->f==Xsubshell)
    p++;
```

Uses Xappend() 71c, Xasync(), Xbackq(), Xcase() 81b, Xclose() 122d, Xfalse() 66e, Xfor() 79a, Xjump() 78c, Xrdwr() 120a, Xread() 71a, Xsubshell(), Xtrue() 66d, and Xwrite() 70a.

3.4 Thread and runqeue

⟨*struct* Thread 28c⟩≡ (? 0—1)

```
struct Thread {
    union Code *code;  /* code for this thread */
    int pc;   /* code[pc] is the next instruction */

    ⟨Thread other fields 28f⟩

    // Extra
    ⟨Thread extra fields 28e⟩
};
```

Uses Code 25b.

⟨*global* runq 28d⟩≡ (? 0—1)

```
// stack<ref_own<Thread>> (next = Thread.ret)
thread *runq;
```

⟨Thread *extra fields* 28e⟩≡ (28c)

```
thread *ret;  /* who continues when this finishes */
```

⟨Thread *other fields* 28f⟩≡ (28c) 28g ▷

```
// list<list<ref_own<word>>> (next = List.next)
struct List *argv;  /* argument stack */
```

Uses List 30c.

⟨Thread *other fields* 28g⟩+≡ (28c) ◁ 28f 37a ▷

```
// list<ref_own<Var>> (next = Var.next)
struct Var *local;  /* list of local variables */
```

Uses Var 45g.

3.5 Words and lists of words

3.5.1 Words

⟨*struct* Word 29a⟩≡ (? 0—1)

```
/*
 * word lists are in correct order,
 * i.e. word0->word1->word2->word3->nil
 */
struct Word {
    char *word;

    // Extra
    word *next;
};
```

⟨*function* newword 29b⟩≡ (? 0—1)

```
word*
newword(char *wd, word *next)
{
    word *p = new(struct Word);
    p->word = strdup(wd);
    p->next = next;
    return p;
}
```

⟨*function* count 29c⟩≡ (? 0—1)

```
int
count(word *w)
{
    int n;
    for(n = 0;w;n++)
        w = w->next;
    return n;
}
```

⟨*function* copynwords 29d⟩≡ (? 0—1)

```
word*
copynwords(word *a, word *tail, int n)
{
    word *v = nil;
    word **end = &v;

    while(n-- > 0){
        *end = newword(a->word, 0);
        end = &(*end)->next;
        a = a->next;
    }
    *end = tail;
    return v;
}
```

Uses newword() 29b and nil.

⟨*function* copywords 29e⟩≡ (? 0—1)

```
/*
 * copy arglist a, adding the copy to the front of tail
 */
word*
copywords(word *a, word *tail)
{
```

```
    word *v = nil;
    word **end;

    for(end=&v;a;a = a->next,end=&(*end)->next)
        *end = newword(a->word, nil);
    *end = tail;
    return v;
}
```

Uses `newword()` 29b and `nil`.

⟨*function* `freewords` 30a⟩≡ (? 0—1)

```
void
freewords(word *w)
{
    word *nw;
    while(w){
        efree(w->word);
        nw = w->next;
        efree((char *)w);
        w = nw;
    }
}
```

Uses `efree()`.

⟨*function* `freelist` 30b⟩≡ (? 0—1)

```
void
freelist(word *w)
{
    word *nw;
    while(w){
        nw = w->next;
        efree(w->word);
        efree((char *)w);
        w = nw;
    }
}
```

Uses `efree()`.

3.5.2 List of sequence of words

⟨*struct* `List` 30c⟩≡ (? 0—1)

```
struct List {
    // list<ref_own<Word>> (next = Word.next)
    word *words;

    // Extra
    list *next;
};
```

3.6 Variables

⟨*struct* `Var` 30d⟩≡ (? 0—1)

```
struct Var {
    // key
    char *name;  /* ascii name */
    // value
```

```
    word *val;  /* value */

    ⟨Var other fields 81e⟩
    // Extra
    ⟨Var extra fields 31a⟩
};
```

⟨Var *extra fields* 31a⟩≡ (30d)

```
var *next;  /* next on hash or local list */
```

⟨*function* setvar 31b⟩≡ (? 0—1)

```
void
setvar(char *name, word *val)
{
    struct Var *v = vlook(name);
    freewords(v->val);
    v->val = val;
    v->changed = true;
}
```

Uses Var 45g, freewords() 30a, true, and vlook() 31c.

⟨*function* vlook 31c⟩≡ (? 0—1)

```
var*
vlook(char *name)
{
    var *v;
    if(runq)
        for(v = runq->local;v;v = v->next)
            if(strcmp(v->name, name)==0)
                return v;

    return gvlook(name);
}
```

Uses gvlook() 31f and runq 28d.

⟨*constant* NVAR 31d⟩≡ (? 0—1)

```
#define NVAR 521
```

⟨*global* gvar 31e⟩≡ (? 0—1)

```
// map<string, ref_own<Var>> (next = Var.next in bucket list)
var *gvar[NVAR];  /* hash for globals */
```

Uses NVAR 29a.

⟨*function* gvlook 31f⟩≡ (? 0—1)

```
var*
gvlook(char *name)
{
    int h = hash(name, NVAR);
    var *v;

    for(v = gvar[h];v;v = v->next)
        if(strcmp(v->name, name)==0)
            return v;
    gvar[h] = newvar(strdup(name), gvar[h]);
    return gvar[h];
}
```

Uses NVAR 29a, gvar 31e, hash() 32a, and newvar() 32b.

⟨*function* hash 32a⟩≡ (? 0—1)

```
unsigned
hash(char *as, int n)
{
    int i = 1;
    unsigned h = 0;
    uchar *s;

    s = (uchar *)as;
    while (*s)
        h += *s++ * i++;
    return h % n;
}
```

⟨*function* newvar 32b⟩≡ (? 0—1)

```
var*
newvar(char *name, var *next)
{
    var *v = new(var);
    v->name = name;
    v->val = nil;

    v->fn = nil;
    v->changed = false;
    v->fnchanged = false;

    v->next = next;
    return v;
}
```

Uses `false` and `nil`.

Chapter 4

main()

4.1 Overview

⟨main() *locals* 33a⟩≡ (33b) ?? ▷
```
code bootstrap[17];
```
Uses true.

⟨*function main (rc/exec.c)* 33b⟩≡ (? 0—1)
```
void main(int argc, char *argv[])
{
    ⟨main() locals 33a⟩

    ⟨main() argc argv processing, modify flags 33f⟩

    ⟨main() initialisation 34d⟩
    ⟨main() initialize boostrap 103a⟩
    ⟨main() initialize runq with bootstrap code 35a⟩
    ⟨main() initialize runq->argv 35d⟩

    ⟨main() interpreter loop 36b⟩
}
```
Uses Xsimple() 61a.

4.2 Command-line arguments processing

⟨*global* flag 33c⟩≡ (? 0—1)
```
// map<char, option<array<string>>>
char **flag[NFLAG];
```
Uses NFLAG 33d.

⟨*constant* NFLAG 33d⟩≡ (? 0—1)
```
#define NFLAG 128
```

⟨*global* flagset 33e⟩≡ (? 0—1)
```
char *flagset[] = {"<flag>"};
```
Uses flagset 33e.

⟨main() *argc argv processing, modify flags* 33f⟩≡ (33b) 34a ▷
```
argc = getflags(argc, argv, "SsrdiIlxepvVc:1m:1[command]", 1);
if(argc==-1)
    usage("[file [arg ...]]");
```

4.2.1 Login mode: `rc -l`

⟨`main()` *argc argv processing, modify flags* 34a⟩+≡ (33b) ◁33f 34b▷

```
if(argv[0][0]=='-')
    flag['l'] = flagset;
```

4.2.2 Interactive mode: `rc -i`

⟨`main()` *argc argv processing, modify flags* 34b⟩+≡ (33b) ◁34a

```
if(flag['I'])
    flag['i'] = nil;
else
    if(flag['i']==nil && argc==1 && Isatty(STDIN))
       flag['i'] = flagset;
```

⟨*function* `Isatty` 34c⟩≡ (? 0—1)

```
bool
Isatty(fdt fd)
{
    char buf[64];

    if(fd2path(fd, buf, sizeof buf) != 0)
        return false;

    /* might be #c/cons during boot - fixed 22 april 2005, remove this later */
    if(strcmp(buf, "#c/cons") == 0)
        return true;

    /* might be /mnt/term/dev/cons */
    return strlen(buf) >= 9 && strcmp(buf+strlen(buf)-9, "/dev/cons") == 0;
}
```

5.2.3

4.3 Initialization

⟨`main()` *initialisation* 34d⟩≡ (33b) 34e▷

```
err = openfd(STDERR);
```

⟨`main()` *initialisation* 34e⟩+≡ (33b) ◁34d 103d▷

```
kinit();    // initialize keywords
Trapinit(); // notify() function setup
Vinit();    // read environment variables and add them in gvar
```

4.4 Bootstrapping bytecodes (simplified)

⟨`main()` *initialize* `boostrap` *(simplified)* 34f⟩≡

```
memset(bootstrap, 0, sizeof bootstrap);

i = 0;
bootstrap[i++].i = 1; // reference count
bootstrap[i++].f = Xrdcmds;
bootstrap[i].i = 0;
```

4.5 Setting runq

⟨main() *initialize* runq *with bootstrap code* 35a⟩≡ (33b) 37b ▷

```
start(bootstrap, 1, (var *)nil);
```

Uses Xword() 57f.

⟨*function* start 35b⟩≡ (? 0—1)

```
void
start(code *c, int pc, var *local)
{
    struct Thread *p = new(struct Thread);

    p->code = codecopy(c);
    p->pc = pc;

    p->argv = nil;
    p->local = local;

    p->cmdfile = nil;
    p->cmdfd = nil;
    p->lineno = 1;
    p->eof = false;
    p->iflag = false;

    ⟨start() set redir 68b⟩

    // add_stack(runq, p)
    p->ret = runq;
    runq = p;
}
```

Uses Thread 28c, codecopy() 27f, false, nil, and runq 28d.

4.6 Setting runq->argv

⟨*global* argv0 35c⟩≡ (? 0—1)

```
/*
 * Start executing the given code at the given pc with the given redirection
 */
char *argv0="rc";
```

Uses argv0 35c.

⟨main() *initialize* runq->argv 35d⟩≡ (33b)

```
/* prime bootstrap argv */
pushlist();
argv0 = strdup(argv[0]);
for(i = argc-1; i!=0; --i)
    pushword(argv[i]);
```

Uses Xdol() 85d and Xword() 57f.

⟨*function* pushlist 35e⟩≡ (? 0—1)

```
void
pushlist(void)
{
    list *p = new(list);

    // add_list(p, runq->argv)
    p->next = runq->argv;
```

```
    p->words = nil;
    runq->argv = p;
}
```

Uses `nil` and `runq` 28d.

⟨*function* `pushword` 36a⟩≡ (? 0—1)

```
void
pushword(char *wd)
{
    if(runq->argv==nil)
        panic("pushword but no argv!", 0);
    runq->argv->words = newword(wd, runq->argv->words);
}
```

Uses `newword()` 29b, `nil`, `panic()` 139b, and `runq` 28d.

4.7 Bytecode interpreter loop

⟨`main()` *interpreter loop* 36b⟩≡ (33b)

```
for(;;){
    ⟨main() debug runq in interpreter loop 135b⟩

    runq->pc++;
    (*runq->code[runq->pc-1].f)();

    ⟨main() handing trap if necessary in interpreter loop 101b⟩
}
```

Uses `STDIN`, `Xexit()` 76a, `nil`, `openfd()` 145f, `pushlist()` 35e, `runq` 28d, and `start()` 35b.

4.8 Reading commands: `Xrdcmds()`

⟨*function* `Xrdcmds` 36c⟩≡ (? 0—1)

```
void
Xrdcmds(void)
{
    struct Thread *p = runq;
    bool error;
    ⟨Xrdcmds() other locals 39g⟩

    ⟨Xrdcmds() flush errors and reset error count 138d⟩
    ⟨Xrdcmds() print status if -s 137a⟩
    ⟨Xrdcmds() set promptstr if interactive mode 39h⟩

    ⟨Xrdcmds() calls Noerror() before yyparse() 102c⟩
    // read one cmd line, compiles it, and modifies codebuf global
    error = yyparse();

    ⟨Xrdcmds() if yyparse() returned an error 37c⟩
    else{
        ⟨Xrdcmds() reset ntrap 102e⟩
        --p->pc; /* re-execute Xrdcmds after codebuf runs */
        // modifies runq, new thread (linked to bootstrap one)
        start(codebuf, 1, runq->local);
    }
    freenodes(); // allocated in yyparse()
}
```

Uses `Thread` 28c, `codebuf` 27d, `freenodes()` 26e, `runq` 28d, and `start()` 35b.

⟨Thread *other fields* 37a⟩+≡ (28c) ◁28g 38d▷

```
struct Io *cmdfd; /* file descriptor for Xrdcmd */
char *cmdfile;  /* file name in Xrdcmd */
bool iflag;  /* interactive? */
```

Uses Io 145e.

⟨main() *initialize* runq *with bootstrap code* 37b⟩+≡ (33b) ◁35a

```
runq->cmdfd = openfd(STDIN); // reading from stdin
runq->cmdfile = "<stdin>";
runq->iflag = flag['i']? true : false;// interactive mode; will print a prompt
```

Uses Xassign() 84c and Xmark() 58a.

⟨Xrdcmds() *if yyparse() returned an error* 37c⟩≡ (36c)

```
if(error){
    if(!p->iflag  ||  p->eof && !Eintr()){
        if(p->cmdfile)
            efree(p->cmdfile);
        closeio(p->cmdfd);

        Xreturn(); /* should this be omitted? */
    }else{
        if(Eintr()){
            pchr(err, '\n');
            p->eof = false;
        }
        --p->pc; /* go back for next command */
    }
}
```

Uses Eintr(), Xreturn() 73b, closeio() 147b, efree(), err 138b, false, and pchr() 146a.

Chapter 5

Input

5.1 Overview

5.2 Reading a character: getnext()

⟨*function* getnext 38a⟩≡ (? 0—1)

```
/*
 * read a character from the input stream
 */
int
getnext(void)
{
    int c;
    ⟨getnext() other locals 39c⟩

    ⟨getnext() peekc handling 39d⟩
    ⟨getnext() return if already at EOF 39a⟩
    ⟨getnext() prompt management before reading the character 40a⟩

    c = rchr(runq->cmdfd);

    ⟨getnext() handle backslash 39b⟩
    ⟨getnext() prompt management after the character is read 40d⟩
    ⟨getnext() if character read is EOF 38c⟩
    ⟨getnext() if not EOF but verbose mode, print character read 137c⟩

    return c;
}
```

Uses rchr() 146c and runq 28d.

5.2.1 End-of-file management

⟨*constant EOF (rc/io.h)* 38b⟩≡ (? 0—1)

```
#define EOF (-1)
```

⟨getnext() *if character read is EOF* 38c⟩≡ (38a)

```
if(c==EOF)
    runq->eof = true;
```

Uses EOF 38b, runq 28d, and true.

⟨Thread *other fields* 38d⟩+≡ (28c) ◁37a 40c▷

```
bool eof;  /* is cmdfd at eof? */
```

⟨getnext() *return if already at EOF* 39a⟩≡ (38a)

```
if(runq->eof)
    return EOF;
```

Uses EOF 38b and runq 28d.

5.2.2 Multiple lines commands and escaped newlines

⟨getnext() *handle backslash* 39b⟩≡ (38a)

```
if(!inquote && c=='\\'){

    c = rchr(runq->cmdfd);

    if(c=='\n' && !incomm){  /* don't continue a comment */
        ⟨getnext() when backslash and newline, set doprompt 40e⟩
        c=' ';
    }
    else{
        peekc = c;
        c='\\';
    }
}
```

Uses incomm 43d, inquote 46d, rchr() 146c, and runq 28d.

⟨getnext() *other locals* 39c⟩≡ (38a)

```
static int peekc = EOF;
```

Uses EOF 38b.

⟨getnext() *peekc handling* 39d⟩≡ (38a)

```
if(peekc!=EOF){
    c = peekc;
    peekc = EOF;
    return c;
}
```

Uses EOF 38b.

5.2.3 Displaying the prompt

⟨*global* doprompt 39e⟩≡ (? 0—1)

```
bool doprompt = true;
```

Uses doprompt 39e and true.

⟨*global* promptstr 39f⟩≡ (? 0—1)

```
char *promptstr;
```

⟨Xrdcmds() *other locals* 39g⟩≡ (36c)

```
word *prompt;
```

⟨Xrdcmds() *set promptstr if interactive mode* 39h⟩≡ (36c)

```
if(runq->iflag){
    prompt = vlook("prompt")->val;
    if(prompt)
        promptstr = prompt->word;
    else
        promptstr="% ";
}
```

Uses promptstr 39f, runq 28d, and vlook() 31c.

⟨getnext() *prompt management before reading the character* 40a⟩≡ (38a)

```
if(doprompt)
    // pprompt() internally set doprompt back to false at the end
    pprompt();
```

Uses doprompt 39e and pprompt() 40b.

⟨*function* pprompt 40b⟩≡ (? 0—1)

```
void
pprompt(void)
{
    var *prompt;

    if(runq->iflag){
        // printing the prompt
        pstr(err, promptstr);
        flush(err);

        // set promptstr for the next pprompt()
        prompt = vlook("prompt");
        if(prompt->val && prompt->val->next)
            promptstr = prompt->val->next->word;
        else
            promptstr="\t";
    }
    runq->lineno++;
    doprompt = false;
}
```

Uses doprompt 39e, err 138b, false, flush() 145h, promptstr 39f, pstr() 149a, runq 28d, and vlook() 31c.

⟨Thread *other fields* 40c⟩+≡ (28c) ◁38d 68d▷

```
int lineno;   /* linenumber */
```

⟨getnext() *prompt management after the character is read* 40d⟩≡ (38a)

```
doprompt = doprompt || c=='\n' || c==EOF;
```

Uses EOF 38b and doprompt 39e.

⟨getnext() *when backslash and newline, set doprompt* 40e⟩≡ (39b)

```
doprompt = true;
```

Uses doprompt 39e and true.

5.3 Looking ahead: nextc() and advance()

⟨*global* future 40f⟩≡ (? 0—1)

```
int future = EOF;
```

Uses EOF 38b and future 40f.

⟨*function* nextc 40g⟩≡ (? 0—1)

```
/*
 * Look ahead in the input stream
 */
int
nextc(void)
{
    if(future==EOF)
        future = getnext();
    return future;
}
```

Uses EOF 38b, future 40f, and getnext() 38a.

⟨*function* advance 41a⟩≡ (? 0—1)

```
/*
 * Consume the lookahead character.
 */
int
advance(void)
{
    int c = nextc();
    ⟨advance() save future in lastc 139a⟩
    future = EOF;
    return c;
}
```

Uses EOF 38b, future 40f, and nextc() 40g.

⟨*function* nextis 41b⟩≡ (? 0—1)

```
bool
nextis(int c)
{
    if(nextc()==c){
        advance();
        return true;
    }
    return false;
}
```

Uses advance() 41a, false, nextc() 40g, and true.

Chapter 6

Lexing

6.1 yylex()

⟨*function* yylex 42a⟩≡ (? 0—1)

```
//@Scheck: called from yyparse()
int yylex(void)
{
    int c;
    ⟨yylex() other locals 44e⟩

    ⟨yylex() hack for SUB 49b⟩
    ⟨yylex() initialisations 45b⟩

    skipwhite();

    switch(c = advance()){
    ⟨yylex() switch c cases 43a⟩
    }
    // else
    ⟨yylex() if c is not a word character 46c⟩
    // else
    ⟨yylex() if c is a word character 47c⟩
}
```

Uses advance() 41a and skipwhite() 43e.

⟨*function* wordchr 42b⟩≡ (? 0—1)

```
int
wordchr(int c)
{
    return !strchr("\n \t#;&|^$=`'{}()<>", c) && c!=EOF;
}
```

Uses EOF 38b.

⟨*function* idchr 42c⟩≡ (? 0—1)

```
int
idchr(int c)
{
    /*
     * Formerly:
     * return 'a'<=c && c<='z' || 'A'<=c && c<='Z' || '0'<=c && c<='9'
     * || c=='_' || c=='*';
     */
    return c > ' ' && !strchr("!\"#$%&'()+,-./:;<=>?@[\\]^`{|}~", c);
}
```

⟨yylex() *switch c cases* 43a⟩≡ (42a) 44a ▷

```
case EOF:
    lastdol = false;
    strcpy(tok, "EOF");
    return EOF;
```

Uses EOF 38b, false, lastdol 44c, and tok 43b.

⟨*global* tok 43b⟩≡ (? 0—1)

```
char tok[NTOK + UTFmax];
```

Uses NTOK<> 43c and UTFmax.

⟨*constant* NTOK 43c⟩≡ (? 0—1)

```
#define NTOK 8192  /* maximum bytes in a word (token) */
```

6.2 Spaces and comments ('#')

⟨*global* incomm 43d⟩≡ (? 0—1)

```
bool incomm;
```

⟨*function* skipwhite 43e⟩≡ (? 0—1)

```
void
skipwhite(void)
{
    int c;
    for(;;){
        c = nextc();
        /* Why did this used to be  if(!inquote && c=='#') ?? */
        if(c=='#'){
            incomm = true;
            for(;;){
                c = nextc();
                if(c=='\n' || c==EOF) {
                    incomm = false;
                    break;
                }
                advance();
            }
        }
        if(c==' ' || c=='\t')
            advance();
        else
            return;
    }
}
```

Uses EOF 38b, advance() 41a, false, incomm 43d, nextc() 40g, and true.

6.3 Newlines

⟨*function* skipnl 43f⟩≡ (? 0—1)

```
void
skipnl(void)
{
    int c;
    for(;;){
        skipwhite();
        c = nextc();
```

```
        if(c!='\n')
            return;
        // consume the newline
        advance();
    }
}
```

Uses advance() 41a, nextc() 40g, and skipwhite() 43e.

6.4 Operators ('&', '&&', '|', '||', '<', '>', '$', ...)

⟨yylex() *switch c cases* 44a⟩+≡ (42a) ◁43a 44b▷

```
case '&':
    lastdol = false;
    if(nextis('&')){
        skipnl();
        strcpy(tok, "&&");
        return ANDAND;
    }
    strcpy(tok, "&");
    return '&';
```

Uses false, lastdol 44c, nextis() 41b, skipnl() 43f, and tok 43b.

⟨yylex() *switch c cases* 44b⟩+≡ (42a) ◁44a 44d▷

```
case '$':
    lastdol = true;
    if(nextis('#')){
        strcpy(tok, "$#");
        return COUNT;
    }
    if(nextis('"')){
        strcpy(tok, "$\"");
        return '"';
    }
    strcpy(tok, "$");
    return '$';
```

Uses lastdol 44c, nextis() 41b, tok 43b, and true.

⟨*global* lastdol 44c⟩≡ (? 0—1)

```
bool lastdol; /* was the last token read '$' or '$#' or '"'? */
```

⟨yylex() *switch c cases* 44d⟩+≡ (42a) ◁44b 46g▷

```
case '|':
    lastdol = false;
    if(nextis('|')){
        skipnl();
        strcpy(tok, "||");
        return OROR;
    }
    // FALLTHROUGH
case '<':
case '>':
    lastdol = false;
    ⟨yylex() in switch when redirection character 45c⟩
```

Uses false, lastdol 44c, nextis() 41b, skipnl() 43f, and tok 43b.

⟨yylex() *other locals* 44e⟩≡ (42a) 45a▷

```
char *w = tok;
```

Uses tok 43b.

⟨yylex() *other locals* 45a⟩+≡ (42a) ◁44e 49a▷

```
struct Tree *t;
```

Uses Tree.

⟨yylex() *initialisations* 45b⟩≡ (42a) 46e▷

```
yylval.tree = nil;
```

Uses nil.

⟨yylex() *in switch when redirection character* 45c⟩≡ (44d)

```
/*
 * funny redirection tokens:
 * redir: arrow | arrow '[' fd ']'
 * arrow: '<' | '<<' | '>' | '>>' | '|'
 * fd: digit | digit '=' | digit '=' digit
 * digit: '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9'
 * some possibilities are nonsensical and get a message.
 */
*w++=c;
t = newtree();
switch(c){
⟨yylex() in switch when redirection character, switch c cases 45h⟩
}
⟨yylex() in switch when redirection character, if bracket after 120b⟩
*w='\0';
yylval.tree = t;
if(t->type==PIPE)
    skipnl();
return t->type;
```

Uses newtree() 26d and skipnl() 43f.

⟨Tree *redirection and pipe specific fields* 45d⟩≡ (25b)

```
//enum<Redirection_kind>
int rtype;

// For a pipe, fd0 is the left fd of the pipe, and fd1 the right fd.
// For a redirection, fd0 is what we redirect (stdout for >, stdin for <)
// and fd1 is what we possibly redirect to (when DUP).
fdt fd0;
fdt fd1; /* details of REDIR PIPE DUP tokens */
```

⟨*constant* WRITE 45e⟩≡ (? 0—1)

```
#define WRITE 2
```

⟨*constant* READ 45f⟩≡ (? 0—1)

```
#define READ 3
```

⟨*constant* APPEND 45g⟩≡ (? 0—1)

```
#define APPEND 1
```

⟨yylex() *in switch when redirection character, switch c cases* 45h⟩≡ (45c) 46a▷

```
case '|':
    t->type = PIPE;
    t->fd0 = 1; // left fd of pipe (stdout of left cmd)
    t->fd1 = 0; // right fd of pipe (stdin of right cmd)
    break;
```

⟨yylex() *in switch when redirection character, switch c cases* 46a⟩+≡ (45c) ◁45h 46b▷

```
case '>':
    t->type = REDIR;
    if(nextis('>')){
        t->rtype = APPEND;
        *w++=c;
    }
    else
        t->rtype = WRITE;
    t->fd0 = 1;
    break;
```

Uses APPEND 45d, WRITE, and nextis() 41b.

⟨yylex() *in switch when redirection character, switch c cases* 46b⟩+≡ (45c) ◁46a

```
case '<':
    t->type = REDIR;
    ⟨yylex() in switch when redirection character, if here document 116b⟩
    ⟨yylex() in switch when redirection character, if read/write redirect 119c⟩
    else
        t->rtype = READ;
    t->fd0 = 0;
    break;
```

Uses READ.

⟨yylex() *if c is not a word character* 46c⟩≡ (42a)

```
if(!wordchr(c)){
    lastdol = false;
    tok[0] = c;
    tok[1]='\0';
    return c;
}
```

Uses false, lastdol 44c, tok 43b, and wordchr() 42b.

6.5 Quoted strings ('...')

⟨*global* inquote 46d⟩≡ (? 0—1)

```
bool inquote;
```

⟨yylex() *initialisations* 46e⟩+≡ (42a) ◁45b

```
inquote = false;
```

Uses false and inquote 46d.

⟨*global* lastword 46f⟩≡ (? 0—1)

```
// used also by syn.y
bool lastword; /* was the last token read a word or compound word terminator? */
```

⟨yylex() *switch c cases* 46g⟩+≡ (42a) ◁44d

```
case '\'':
    inquote = true;
    lastword = true;
    lastdol = false;
    for(;;){
        c = advance();
        if(c==EOF)
            break;
        if(c=='\''){
            if(nextc()!='\'')
```

```
                break;
            advance();
        }
        w = addutf(w, c);
    }
    if(w != nil)
        *w='\0';

    t = token(tok, WORD);
    t->quoted = true;

    yylval.tree = t;
    return t->type;
```

Uses EOF 38b, addutf() 114f, advance() 41a, false, inquote 46d, lastdol 44c, lastword 46f, nextc() 40g, nil, tok 43b, token() 47b, and true.

⟨Tree *word specific fields* 47a⟩≡ (25b)

```
bool quoted;
```

⟨*function* token 47b⟩≡ (? 0—1)

```
tree*
token(char *str, int type)
{
    tree *t = newtree();

    t->type = type;
    t->str = strdup(str);
    return t;
}
```

Uses newtree() 26d.

6.6 Keywords and identifiers (if, for, while, switch, fn, ...)

⟨yylex() *if c is a word character* 47c⟩≡ (42a)

```
for(;;){
    ⟨yylex() when c is a word character, if glob character 105b⟩
    w = addutf(w, c);

    c = nextc();
    if(lastdol ? !idchr(c) : !wordchr(c))
        break;
    advance();
}

lastword = true;
lastdol = false;
if(w!=nil)
    *w='\0';

t = klook(tok);
if(t->type != WORD)
    lastword = false;

t->quoted = false;

yylval.tree = t;
return t->type;
```

Uses addutf() 114f, advance() 41a, false, idchr() 42c, klook() 48a, lastdol 44c, lastword 46f, nextc() 40g, nil, tok 43b, true, and wordchr() 42b.

⟨*function* `klook` 48a⟩≡ (? 0—1)

```
tree*
klook(char *name)
{
    struct Kw *p;
    tree *t = token(name, WORD);

    for(p = kw[hash(name, NKW)];p;p = p->next)
        if(strcmp(p->name, name)==0){
            t->type = p->type;
            break;
        }
    return t;
}
```

Uses `Kw` 48b, `NKW<>` 48d, `hash()` 32a, `kw` 48c, and `token()` 47b.

⟨*struct* `Kw` 48b⟩≡ (? 0—1)

```
struct Kw {
    char *name;
    int type;

    struct Kw *next;
};
```

Uses `Kw` 48b.

⟨*global* `kw` 48c⟩≡ (? 0—1)

```
struct Kw *kw[NKW];
```

Uses `Kw` 48b and `NKW<>` 48d.

⟨*constant* `NKW` 48d⟩≡ (? 0—1)

```
#define NKW 30
```

⟨*function* `kinit` 48e⟩≡ (? 0—1)

```
void
kinit(void)
{
    kenter(FOR, "for");
    kenter(IN, "in");
    kenter(WHILE, "while");
    kenter(IF, "if");
    kenter(NOT, "not");
    kenter(SWITCH, "switch");
    kenter(FN, "fn");

    kenter(TWIDDLE, "~");
    kenter(BANG, "!");
    kenter(SUBSHELL, "@");
}
```

Uses `kenter()` 48f.

⟨*function* `kenter` 48f⟩≡ (? 0—1)

```
void
kenter(int type, char *name)
{
    int h = hash(name, NKW);
    struct Kw *p = new(struct Kw);
    p->type = type;
    p->name = name;
    p->next = kw[h];
```

```
    kw[h] = p;
}
```

Uses Kw 48b, NKW<> 48d, hash() 32a, and kw 48c.

6.7 Array subscript (<arr>(<n>))

⟨yylex() *other locals* 49a⟩+≡ (42a) ◁45a

```
int d = nextc();
```

Uses nextc() 40g.

⟨yylex() *hack for SUB* 49b⟩≡ (42a)

```
/*
 * Embarassing sneakiness:  if the last token read was a quoted or unquoted
 * WORD then we alter the meaning of what follows.  If the next character
 * is '(', we return SUB (a subscript paren) and consume the '('. Otherwise,
 * if the next character is the first character of a simple or compound word,
 * we insert a '^' before it.
 */
if(lastword){
    lastword = false;
    if(d=='('){
        advance();
        strcpy(tok, "( [SUB]");
        return SUB;
    }
    if(wordchr(d) || d=='\'' || d=='`' || d=='$' || d=='"'){
        strcpy(tok, "^");
        return '^';
    }
}
```

Uses advance() 41a, false, lastword 46f, tok 43b, and wordchr() 42b.

Chapter 7

Parsing

7.1 Overview

⟨*rc/syn.y* 50a⟩≡
```
%{
#include "rc.h"
#include "fns.h"
%}

%union {
 struct Tree *tree;
};

⟨token declarations 25a⟩
⟨priority and associativity declarations 50c⟩
⟨type declarations 50d⟩

%%
⟨grammar 50e⟩
```

⟨*function* yyparse 50b⟩≡
```
... generated code from syn.y by yacc ...
```

⟨*priority and associativity declarations* 50c⟩≡ (50a)
```
/* operator priorities -- lowest first */
%left IF WHILE FOR SWITCH ')' NOT
%left ANDAND OROR
%left BANG SUBSHELL
%left PIPE
%left '^'
%right '$' COUNT '"'
%left SUB
```

⟨*type* declarations 50d⟩≡ (50a) ?? ▷
```
%type<tree> line cmd simple word comword
```

⟨*grammar* 50e⟩≡ (50a)
```
rc:
    /*empty*/       { return ERROR_1;}
|   line '\n'       { return !compile($1);}

⟨line rule 51a⟩
⟨cmd rule 51b⟩
⟨simple rule 51c⟩
```

⟨*word rule* 51f⟩
⟨*comword rule* 51e⟩

⟨*other rules* 51d⟩

7.2 Simple commands (<cmd> <arg1>...<argn>)

⟨*line rule* 51a⟩≡ (50e)
```
line:
    cmd
```
⟨*line rule other cases* 52b⟩

⟨*cmd rule* 51b⟩≡ (50e)
```
cmd:
    /*empty*/           {$$=nil;}
|   simple              {$$=simplemung($1);}
```
⟨*cmd rule other cases* 52d⟩

⟨*simple rule* 51c⟩≡ (50e)
```
simple:
    first
|   simple word         {$$=tree2(ARGLIST, $1, $2);}
```
⟨*simple rule other cases* 53e⟩

⟨*other rules* 51d⟩≡ (50e) 51g ▷
```
first:
    comword
|   first '^' word      {$$=tree2('^', $1, $3);}
```

⟨*comword rule* 51e⟩≡ (50e)
```
comword:
    WORD
```
⟨*comword rule other cases* 55b⟩

⟨*word rule* 51f⟩≡ (50e)
```
word:
    comword
|   word '^' word       {$$=tree2('^', $1, $3);}
|   keyword             {lastword=true; $1->type=WORD;}
```

⟨*other rules* 51g⟩+≡ (50e) ◁ 51d 51h ▷
```
keyword: FOR|IN|WHILE|IF|NOT|TWIDDLE|BANG|SUBSHELL|SWITCH|FN
```

⟨*other rules* 51h⟩+≡ (50e) ◁ 51g 52c ▷
```
words:
    /*empty*/       {$$=(struct Tree*)nil;}
|   words word      {$$=tree2(WORDS, $1, $2);}
```

⟨*function* simplemung 51i⟩≡ (? 0—1)
```
/*
 * Add a SIMPLE node at the root of t and percolate all the redirections
 * up to the root.
 */
//@Scheck: used by syn.y
tree* simplemung(tree *t)
{
    tree *u;
    struct Io *s;
```

```
    t = tree1(SIMPLE, t);

    s = openstr();
    pfmt(s, "%t", t);
    t->str = strdup((char *)s->strp);
    closeio(s);

    ⟨simplemung() percolate redirections up to the root 52a⟩
    return t;
}
```

Uses `Io` 145e, `closeio()` 147b, `openstr()` 147c, `pfmt()` 148b, and `tree1()` 26f.

⟨`simplemung()` *percolate redirections up to the root* 52a⟩≡ (51i)

```
for(u = t->child[0]; u->type==ARGLIST; u = u->child[0]){
    if(u->child[1]->type==REDIR || u->child[1]->type==DUP){
        u->child[1]->child[1] = t;
        t = u->child[1];
        u->child[1] = nil;
    }
}
```

Uses `nil`.

7.3 Operators

7.3.1 Sequences (';', '&')

⟨*line rule other cases* 52b⟩≡ (51a)

```
|   cmdsa line      {$$=tree2(';', $1, $2);}
```

⟨*other rules* 52c⟩+≡ (50e) ◁51h 53f▷

```
cmdsa:
    cmd ';'
|   cmd '&'         {$$=tree1('&', $1);}
```

7.3.2 Logical operators ('&&', '||', '!')

⟨*cmd rule other cases* 52d⟩≡ (51b) 52e▷

```
|   cmd ANDAND cmd      {$$=tree2(ANDAND, $1, $3);}
|   cmd OROR cmd        {$$=tree2(OROR, $1, $3);}
```

⟨*cmd rule other cases* 52e⟩+≡ (51b) ◁52d 53b▷

```
|   BANG cmd            {$$=mung1($1, $2);}
```

⟨*function* `mung1` 52f⟩≡ (? 0—1)

```
//@Scheck: used by syn.y
tree* mung1(tree *t, tree *c0)
{
    t->child[0] = c0;
    return t;
}
```

⟨*function* mung3 53a⟩≡ (? 0—1)

```
//@Scheck: used by syn.y
tree* mung3(tree *t, tree *c0, tree *c1, tree *c2)
{
    t->child[0] = c0;
    t->child[1] = c1;
    t->child[2] = c2;
    return t;
}
```

7.3.3 String matching ('~')

⟨*cmd rule other cases* 53b⟩+≡ (51b) ◁52e 53d▷

```
|   TWIDDLE word words  {$$=mung2($1, $2, $3);}
```

⟨*function* mung2 53c⟩≡ (? 0—1)

```
//@Scheck: used by syn.y
tree* mung2(tree *t, tree *c0, tree *c1)
{
    t->child[0] = c0;
    t->child[1] = c1;
    return t;
}
```

7.3.4 Pipe ('|')

⟨*cmd rule other cases* 53d⟩+≡ (51b) ◁53b 53g▷

```
|   cmd PIPE cmd        {$$=mung2($2, $1, $3);}
```

7.3.5 Redirections ('>', '<')

⟨*simple rule other cases* 53e⟩≡ (51c)

```
|   simple redir        {$$=tree2(ARGLIST, $1, $2);}
```

⟨*other rules* 53f⟩+≡ (50e) ◁52c 53h▷

```
redir:
    REDIR word      {$$=mung1($1, $1->rtype==HERE ? heredoc($2) : $2);}
```

⟨*redir rule other cases* 121c⟩

⟨*cmd rule other cases* 53g⟩+≡ (51b) ◁53d 54a▷

```
|   brace epilog        {$$=epimung($1, $2);}
```

⟨*other rules* 53h⟩+≡ (50e) ◁53f 54c▷

```
epilog:
    /*empty*/           {$$=nil;}
|   redir epilog        {$$=mung2($1, $1->child[0], $2);}
```

⟨*function* epimung 53i⟩≡ (? 0—1)

```
//@Scheck: used by syn.y
tree* epimung(tree *comp, tree *epi)
{
    tree *p;
    if(epi==0)
        return comp;
    for(p = epi;p->child[1];p = p->child[1]);
    p->child[1] = comp;
    return epi;
}
```

⟨*cmd rule other cases* 54a⟩+≡ (51b) ◁53g 54b▷

```
  |   redir cmd  %prec BANG
          {$$=mung2($1, $1->child[0], $2);}
```

7.4 Control flow statements (`if`, `if not`, `while`, `switch`, `for`)

⟨*cmd rule other cases* 54b⟩+≡ (51b) ◁54a 54g▷

```
  |   IF paren {skipnl();} cmd  {$$=mung2($1, $2, $4);}
  |   IF NOT   {skipnl();} cmd  {$$=mung1($2, $4);}

  |   WHILE paren {skipnl();} cmd    {$$=mung2($1, $2, $4);}
  |   SWITCH word {skipnl();} brace  {$$=tree2(SWITCH, $2, $4);}

   /*
    * if ``words'' is nil, we need a tree element to distinguish between
    * for(i in ) and for(i), the former being a loop over the empty set
    * and the latter being the implicit argument loop.  so if $5 is nil
    * (the empty set), we represent it as "()".  don't parenthesize non-nil
    * functions, to avoid growing parentheses every time we reread the
    * definition.
    */
  |   FOR '(' word IN words ')' {skipnl();} cmd
      {$$=mung3($1, $3,    $5 ? $5 : tree1(PAREN, $5), $8);}

  |   FOR '(' word ')' {skipnl();} cmd
      {$$=mung3($1, $3, (struct Tree *)0, $6);}
```

⟨*other rules* 54c⟩+≡ (50e) ◁53h 54e▷

```
  paren:  '(' body ')'        {$$=tree1(PCMD, $2);}
```

⟨*token declarations* 54d⟩+≡ (50a) ◁26a 115a▷

```
  %token PCMD
```

⟨*other rules* 54e⟩+≡ (50e) ◁54c 54f▷

```
  brace:  '{' body '}'        {$$=tree1(BRACE, $2);}
```

⟨*other rules* 54f⟩+≡ (50e) ◁54e 55a▷

```
  body:
      cmd
  |   cmdsan body     {$$=tree2(';', $1, $2);}

  cmdsan:
      cmdsa
  |   cmd '\n'
```

7.5 Functions (`fn`)

⟨*cmd rule other cases* 54g⟩+≡ (51b) ◁54b 54h▷

```
  |   FN words brace  {$$=tree2(FN, $2, $3);}
  |   FN words        {$$=tree1(FN, $2);}
```

7.6 Variables (`<x> = ...`)

⟨*cmd rule other cases* 54h⟩+≡ (51b) ◁54g 115b▷

```
  |   assign cmd %prec BANG
        {$$=mung3($1, $1->child[0], $1->child[1], $2);}
```

⟨*other rules* 55a⟩+≡ (50e) ◁54f

```
assign: first '=' word      {$$=tree2('=', $1, $3);}
```

⟨*comword rule other cases* 55b⟩≡ (51e) 55c▷

```
|   '$' word        {$$=tree1('$', $2);}
|   COUNT word      {$$=tree1(COUNT, $2);}
|   '$' word SUB words ')'  {$$=tree2(SUB, $2, $4);}
```

7.7 Lists ((...))

⟨*comword rule other cases* 55c⟩+≡ (51e) ◁55b 122g▷

```
|   '(' words ')'   {$$=tree1(PAREN, $2);}
```

Chapter 8

Bytecode Generation and Interpretation

8.1 Overview

8.1.1 emitxxx()

⟨*global* codep 56a⟩≡ (? 0—1)

```
// idx in codebuf
int codep;
```

⟨*global* ncode 56b⟩≡ (? 0—1)

```
int ncode;
```

⟨*function* emiti 56c⟩≡ (? 0—1)

```
#define emiti(x) ((codep!=ncode || morecode()), codebuf[codep].i = (x), codep++)
```

⟨*function* emitf 56d⟩≡ (? 0—1)

```
#define emitf(x) ((codep!=ncode || morecode()), codebuf[codep].f = (x), codep++)
```

⟨*function* emits 56e⟩≡ (? 0—1)

```
#define emits(x) ((codep!=ncode || morecode()), codebuf[codep].s = (x), codep++)
```

⟨*function* morecode 56f⟩≡ (? 0—1)

```
//@Scheck: used by the macros above (why marked as dead then??? TODO)
int morecode(void)
{
    ncode+=100;
    codebuf = (code *)realloc((char *)codebuf, ncode*sizeof codebuf[0]);
    if(codebuf==nil)
        panic("Can't realloc %d bytes in morecode!", ncode*sizeof(code));
    return OK_0;
}
```

Uses OK_0, codebuf 27d, ncode 56b, nil, and panic() 139b.

8.1.2 compile()

⟨*function* compile 56g⟩≡ (? 0—1)

```
//@Scheck: called from syn.y
error0 compile(tree *t)
{
    ncode = 100;
    codep = 0;
    codebuf = (code *)emalloc(ncode*sizeof(code));

    emiti(0);   /* reference count */
```

```
    outcode(t, flag['e'] ? true : false);

    ⟨compile() check nerror 57a⟩
    ⟨compile() after outcode and error management, read heredoc 116g⟩

    emitf(Xreturn);
    emitf(nil);

    return OK_1;
}
```

Uses OK_1, Xreturn() 73b, codebuf 27d, codep 56a, emalloc(), emitf<> 56d, emiti<> 56c, false, flag 33c, ncode 56b, nil, outcode() 57e, and true.

⟨compile() *check nerror* 57a⟩≡ (56g)

```
if(nerror){
    efree((char *)codebuf);
    return ERROR_0;
}
```

Uses ERROR_0, codebuf 27d, efree(), and nerror 138c.

8.1.3 outcode()

⟨*constant c0 (rc/code.c)* 57b⟩≡ (? 0—1)

```
#define c0 t->child[0]
```

⟨*constant c1 (rc/code.c)* 57c⟩≡ (? 0—1)

```
#define c1 t->child[1]
```

⟨*constant c2 (rc/code.c)* 57d⟩≡ (? 0—1)

```
#define c2 t->child[2]
```

⟨*function* outcode 57e⟩≡ (? 0—1)

```
void
outcode(tree *t, bool eflag)
{
    ⟨outcode() locals 65d⟩

    if(t==nil)
        return;

    ⟨outcode() set iflast before switch 77g⟩
    switch(t->type){
    ⟨outcode() cases 60c⟩
    default:
        pfmt(err, "bad type %d in outcode\n", t->type);
        break;
    }
    ⟨outcode() set iflast after switch 77f⟩
}
```

Uses err 138b, nil, and pfmt() 148b.

8.1.4 argv management

⟨*function* Xword 57f⟩≡ (? 0—1)

```
void
Xword(void)
{
    pushword(runq->code[runq->pc++].s);
}
```

Uses pushword() 36a and runq 28d.

⟨*function* Xmark 58a⟩≡ (? 0—1)

```
void
Xmark(void)
{
    pushlist();
}
```

Uses pushlist() 35e.

⟨*function* poplist 58b⟩≡ (? 0—1)

```
void
poplist(void)
{
    list *p = runq->argv;
    if(p==nil)
        panic("poplist but no argv", 0);
    freelist(p->words);
    runq->argv = p->next;
    efree((char *)p);
}
```

Uses efree(), freelist() 30b, nil, panic() 139b, and runq 28d.

⟨*function* popword 58c⟩≡ (? 0—1)

```
void
popword(void)
{
    word *p;
    ⟨popword() sanity check argv 58d⟩
    p = runq->argv->words;
    ⟨popword() sanity check argv words 58e⟩
    runq->argv->words = p->next;
    efree(p->word);
    efree((char *)p);
}
```

Uses efree() and runq 28d.

⟨popword() *sanity check argv* 58d⟩≡ (58c)

```
if(runq->argv==nil)
    panic("popword but no argv!", 0);
```

Uses nil, panic() 139b, and runq 28d.

⟨popword() *sanity check argv words* 58e⟩≡ (58c)

```
if(p==nil)
    panic("popword but no word!", 0);
```

Uses nil and panic() 139b.

8.1.5 Process status management

⟨*function* setstatus 58f⟩≡ (? 0—1)

```
void
setstatus(char *s)
{
    setvar("status", newword(s, (word *)nil));
}
```

Uses newword() 29b, nil, and setvar() 31b.

⟨*function* getstatus 59a⟩≡ (? 0—1)

```
char*
getstatus(void)
{
    var *status = vlook("status");
    return status->val ? status->val->word : "";
}
```

Uses vlook() 31c.

⟨*function* truestatus 59b⟩≡ (? 0—1)

```
bool
truestatus(void)
{
    char *s;
    for(s = getstatus();*s;s++)
        if(*s!='|' && *s!='0')
            return false;
    return true;
}
```

Uses false, getstatus() 59a, and true.

8.1.6 Subprocesses management

⟨*global* waitpids 59c⟩≡ (? 0—1)

```
// growing_array<pid> (but really a list)
int *waitpids;
```

⟨*global* nwaitpids 59d⟩≡ (? 0—1)

```
int nwaitpids;
```

⟨*function* addwaitpid 59e⟩≡ (? 0—1)

```
void
addwaitpid(int pid)
{
    ⟨addwaitpid() grow waitpids if needed 59f⟩
    waitpids[nwaitpids++] = pid;
}
```

Uses nil, nwaitpids 59e, and waitpids 59d.

⟨addwaitpid() *grow* waitpids *if needed* 59f⟩≡ (59e)

```
waitpids = realloc(waitpids, (nwaitpids+1)*sizeof waitpids[0]);
if(waitpids == nil)
    panic("Can't realloc %d waitpids", nwaitpids+1);
```

⟨*function* delwaitpid 59g⟩≡ (? 0—1)

```
void
delwaitpid(int pid)
{
    int r, w;

    for(r=w=0; r<nwaitpids; r++)
        if(waitpids[r] != pid)
            waitpids[w++] = waitpids[r];
    nwaitpids = w;
}
```

⟨*function* clearwaitpids 60a⟩≡ (? 0—1)

```
void
clearwaitpids(void)
{
    nwaitpids = 0;
}
```

Uses nwaitpids 59e and waitpids 59d.

⟨*function* havewaitpid 60b⟩≡ (? 0—1)

```
bool
havewaitpid(int pid)
{
    int i;

    for(i=0; i<nwaitpids; i++)
        if(waitpids[i] == pid)
            return true;
    return false;
}
```

8.2 Simple commands

8.2.1 Bytecode generation

⟨outcode() *cases* 60c⟩≡ (57e) 60d ▷

```
case SIMPLE:
    emitf(Xmark);
    outcode(c0, eflag); // the arguments and argv0
    emitf(Xsimple);
    ⟨outcode() emit Xeflag after Xsimple 114b⟩
    break;
```

Uses Xmark() 58a, Xsimple() 61a, c0<> 57b 133b, emitf<> 56d, and outcode() 57e.

⟨outcode() *cases* 60d⟩+≡ (57e) ◁ 60c 60e ▷

```
case ARGLIST:
    outcode(c1, eflag);
    outcode(c0, eflag);
    break;
```

Uses c0<> 57b 133b, c1<> 57c 133c, and outcode() 57e.

⟨outcode() *cases* 60e⟩+≡ (57e) ◁ 60d 60f ▷

```
case WORDS:
    outcode(c1, eflag);
    outcode(c0, eflag);
    break;
```

Uses c0<> 57b 133b, c1<> 57c 133c, and outcode() 57e.

⟨outcode() *cases* 60f⟩+≡ (57e) ◁ 60e 65c ▷

```
case WORD:
    emitf(Xword);
    emits(strdup(t->str));
    break;
```

Uses Xword() 57f, emitf<> 56d, and emits<> 56e.

⟨codefree() *in loop over code* cp*, switch bytecode cases* 60g⟩+≡ (28a) ◁ 28b 73a ▷

```
else if(p->f==Xword || p->f==Xdelhere)
        efree((++p)->s);
```

Uses Xdelhere() 117b, Xword() 57f, and efree().

8.2.2 Xsimple()

⟨*function* Xsimple 61a⟩≡ (? 0—1)

```
void
Xsimple(void)
{
    word *a;
    thread *p = runq;
    int pid;
    ⟨Xsimple() other locals 83b⟩

    ⟨Xsimple() initializations, globlist() 105e⟩

    a = runq->argv->words;
    ⟨Xsimple() sanity check a 61b⟩

    ⟨Xsimple() if -x 137b⟩

    ⟨Xsimple() if argv0 is a function 83c⟩
    else{
        ⟨Xsimple() if argv0 is a builtin 90d⟩
        ⟨Xsimple() if exitnext() 65a⟩
        else{
            flush(err);
            Updenv(); /* necessary so changes don't go out again */
            if((pid = execforkexec()) < 0){
                Xerror("try again");
                return;
            }

            /* interrupts don't get us out */
            poplist();
            while(Waitfor(pid, true) < 0)
                ;
        }
    }
}
```

Uses Updenv(), Waitfor() 75b, Xerror() 61c, err 138b, execforkexec(), flush() 145h, poplist() 58b, and runq 28d.

⟨Xsimple() *sanity check a* 61b⟩≡ (61a)

```
if(a==nil){
    Xerror1("empty argument list");
    return;
}
```

Uses Xerror1() 62a and nil.

⟨*function* Xerror 61c⟩≡ (? 0—1)

```
void
Xerror(char *s)
{
    if(strcmp(argv0, "rc")==0 || strcmp(argv0, "/bin/rc")==0)
        pfmt(err, "rc: %s: %r\n", s);
    else
        pfmt(err, "rc (%s): %s: %r\n", argv0, s);
    flush(err);
    setstatus("error");

    while(!runq->iflag)
        Xreturn();
}
```

Uses Xreturn() 73b, argv0 35c, err 138b, flush() 145h, pfmt() 148b, runq 28d, and setstatus() 58f.

⟨*function* Xerror1 62a⟩≡ (? 0—1)

```
void
Xerror1(char *s)
{
    if(strcmp(argv0, "rc")==0 || strcmp(argv0, "/bin/rc")==0)
        pfmt(err, "rc: %s\n", s);
    else
        pfmt(err, "rc (%s): %s\n", argv0, s);
    flush(err);
    setstatus("error");

    while(!runq->iflag)
        Xreturn();
}
```

Uses Xreturn() 73b, argv0 35c, err 138b, flush() 145h, pfmt() 148b, runq 28d, and setstatus() 58f.

8.2.3 Fork

⟨*function* execforkexec 62b⟩≡ (? 0—1)

```
int
execforkexec(void)
{
    int pid;
    int n;
    char buf[ERRMAX];

    // fork()!!
    switch(pid = fork(){
    case -1:
        return -1;
    case 0: // child
        clearwaitpids();
        pushword("exec");
        execexec();

        // should not be reached! unless the command did not exist
        strcpy(buf, "can't exec: ");
        n = strlen(buf);
        errstr(buf+n, ERRMAX-n);
        Exit(buf, __LOC__);
    }
    // parent
    addwaitpid(pid);
    return pid;
}
```

Uses PRD 31a, PWR, READ, Thread 28c, Xerror() 61c, clearwaitpids() 64a, runq 28d, and start() 35b.

8.2.4 Exec

⟨*function* execexec 62c⟩≡ (? 0—1)

```
void
execexec(void)
{
    popword(); /* "exec" */
    ⟨execexec() sanity check arguments 63a⟩
    ⟨execexec() perform the redirections 68a⟩
```

```
    Execute(runq->argv->words, searchpath(runq->argv->words->word));
    // should not be reached! unless command did not exist
    poplist();
}
```

Uses `Execute()` 63d, `poplist()` 58b, `popword()` 58c, `runq` 28d, and `searchpath()` 63b.

⟨`execexec()` *sanity check arguments* 63a⟩≡ (62c)

```
if(runq->argv->words==nil){
    Xerror1("empty argument list");
    return;
}
```

Uses `Xerror1()` 62a, `nil`, and `runq` 28d.

8.2.5 $path management

⟨*function* `searchpath` 63b⟩≡ (? 0—1)

```
word*
searchpath(char *w)
{
    word *path;

    if(strncmp(w, "/", 1)==0
    || strncmp(w, "#", 1)==0
    || strncmp(w, "./", 2)==0
    || strncmp(w, "../", 3)==0
    || (path = vlook("path")->val)==nil)
        path=&nullpath;
    return path;
}
```

Uses `nil`, `nullpath` 63c, and `vlook()` 31c.

⟨*global* `nullpath` 63c⟩≡ (? 0—1)

```
struct Word nullpath = { "", nil};
```

Uses `Word`, `nil`, and `nullpath` 63c.

⟨*function* `Execute` 63d⟩≡ (? 0—1)

```
void
Execute(word *args, word *path)
{
    char **argv = mkargv(args);
    char file[1024];
    char errstr[1024];
    int nc;

    Updenv();
    errstr[0] = '\0';

    for(;path;path = path->next){
        nc = strlen(path->word);
        if(nc < sizeof file - 1){ /* 1 for / */
            strcpy(file, path->word);
            if(file[0]){
                strcat(file, "/");
                nc++;
            }
            if(nc + strlen(argv[1]) < sizeof file){
                strcat(file, argv[1]);
```

```
                // The actual exec() system call!
                exec(file, argv+1);

                // reached if the file does not exist

                rerrstr(errstr, sizeof errstr);
                /*
                 * if file exists and is executable, exec should
                 * have worked, unless it's a directory or an
                 * executable for another architecture.  in
                 * particular, if it failed due to lack of
                 * swap/vm (e.g., arg. list too long) or other
                 * allocation failure, stop searching and print
                 * the reason for failure.
                 */
                if (strstr(errstr, " allocat") != nil ||
                    strstr(errstr, " full") != nil)
                    break;
            }
            else werrstr("command name too long");
        }
    }
    // should not be reached if found an actual binary to exec
    pfmt(err, "%s: %s\n", argv[1], errstr);
    efree((char *)argv);
}
```

Uses `Updenv()`, `count()` 29c, `emalloc()`, `false`, `mkargv()` 63d, `nil`, `nwaitpids` 59e, `true`, and `waitpids` 59d.

⟨*function* `mkargv` 64a⟩≡ (? 0—1)

```
char **
mkargv(word *a)
{
    char **argv = (char **)emalloc((count(a)+2) * sizeof(char *));
    char **argp = argv+1; /* leave one at front for runcoms */

    for(;a;a = a->next)
        *argp++=a->word;
    *argp = nil;
    return argv;
}
```

Uses `nwaitpids` 59e and `waitpids` 59d.

8.2.6 Wait

⟨*function* `Waitfor` 64b⟩≡ (? 0—1)

```
int
Waitfor(int pid, bool _persist)
{
    thread *p;
    Waitmsg *w;
    char errbuf[ERRMAX];

    if(pid >= 0 && !havewaitpid(pid))
        return 0;

    // wait()!! until we found it
    while((w = wait()) != nil){
        delwaitpid(w->pid);
```

```
        if(w->pid==pid){
            setstatus(w->msg);
            free(w);
            return 0;
        }
        ⟨Waitfor() in while loop, if wait returns another pid 74e⟩
    }

    errstr(errbuf, sizeof errbuf);
    if(strcmp(errbuf, "interrupted")==0)
        return -1;
    return 0;
}
```

8.2.7 Fork optimization

⟨Xsimple() *if exitnext()* 65a⟩≡ (61a)

```
if(exitnext()){
    /* fork and wait is redundant */
    pushword("exec");
    execexec();
    Xexit();
}
```

Uses Xexit() 76a, execexec() 62c, exitnext() 65b, and pushword() 36a.

⟨*function* exitnext 65b⟩≡ (? 0—1)

```
/*
 * Search through the following code to see if we're just going to exit.
 */
int
exitnext(void){
    union Code *c = &runq->code[runq->pc];
    while(c->f==Xpopredir)
        c++;
    return c->f==Xexit;
}
```

Uses Code 25b, Xexit() 76a, Xpopredir() 70b, and runq 28d.

8.3 Operators

8.3.1 Basic sequence

⟨outcode() *cases* 65c⟩+≡ (57e) ◁60f 66a▷

```
case ';':
    outcode(c0, eflag);
    outcode(c1, eflag);
    break;
```

Uses c0<> 57b 133b, c1<> 57c 133c, and outcode() 57e.

8.3.2 Logical operators

⟨outcode() *locals* 65d⟩≡ (57e) 72d▷

```
int p;
```

⟨outcode() *cases* 66a⟩+≡ (57e) ◁65c 66c▷

```
case ANDAND:
    outcode(c0, false);
    emitf(Xtrue);
    p = emiti(0);
    outcode(c1, eflag);
    stuffdot(p);
    break;
```

Uses Xtrue() 66d, c0<> 57b 133b, c1<> 57c 133c, emitf<> 56d, emiti<> 56c, false, outcode() 57e, and stuffdot() 66b.

⟨*function* stuffdot 66b⟩≡ (? 0—1)

```
void
stuffdot(int a)
{
    if(a<0 || codep<=a)
        panic("Bad address %d in stuffdot", a);
    codebuf[a].i = codep;
}
```

Uses codebuf 27d, codep 56a, and panic() 139b.

⟨outcode() *cases* 66c⟩+≡ (57e) ◁66a 66f▷

```
case OROR:
    outcode(c0, false);
    emitf(Xfalse);
    p = emiti(0);
    outcode(c1, eflag);
    stuffdot(p);
    break;
```

Uses Xfalse() 66e, c0<> 57b 133b, c1<> 57c 133c, emitf<> 56d, emiti<> 56c, false, outcode() 57e, and stuffdot() 66b.

⟨*function* Xtrue 66d⟩≡ (? 0—1)

```
void
Xtrue(void)
{
    if(truestatus()) runq->pc++;
    else runq->pc = runq->code[runq->pc].i;
}
```

Uses runq 28d and truestatus() 59b.

⟨*function* Xfalse 66e⟩≡ (? 0—1)

```
void
Xfalse(void)
{
    if(truestatus()) runq->pc = runq->code[runq->pc].i;
    else runq->pc++;
}
```

Uses runq 28d and truestatus() 59b.

⟨outcode() *cases* 66f⟩+≡ (57e) ◁66c 67b▷

```
case BANG:
    outcode(c0, eflag);
    emitf(Xbang);
    break;
```

Uses Xbang() 67a, c0<> 57b 133b, emitf<> 56d, and outcode() 57e.

⟨*function* Xbang 67a⟩≡ (? 0—1)

```
void
Xbang(void)
{
    setstatus(truestatus()? "false" : "");
}
```

Uses setstatus() 58f and truestatus() 59b.

8.3.3 String matching

⟨outcode() *cases* 67b⟩+≡ (57e) ◁66f 69d▷

```
case TWIDDLE:
    emitf(Xmark);
    outcode(c1, eflag);
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xmatch);
    if(eflag)
        emitf(Xeflag);
    break;
```

Uses Xeflag() 114e, Xmark() 58a, Xmatch() 67c, c0<> 57b 133b, c1<> 57c 133c, emitf<> 56d, and outcode() 57e.

⟨*function* Xmatch 67c⟩≡ (? 0—1)

```
void
Xmatch(void)
{
    word *p;
    char *subject;

    subject = list2str(runq->argv->words);
    setstatus("no match");
    for(p = runq->argv->next->words;p;p = p->next)
        if(match(subject, p->word, '\0')){
            setstatus("");
            break;
        }
    efree(subject);
    poplist();
    poplist();
}
```

Uses efree(), list2str() 67d, match() 109d, poplist() 58b, runq 28d, and setstatus() 58f.

⟨*function* list2str 67d⟩≡ (? 0—1)

```
char*
list2str(word *words)
{
    char *value, *s, *t;
    int len = 0;
    word *ap;

    for(ap = words;ap;ap = ap->next)
        len += 1+strlen(ap->word);
    value = emalloc(len+1);

    s = value;
    for(ap = words;ap;ap = ap->next){
        for(t = ap->word;*t;)
            *s++=*t++;
```

```
        *s++=' ';
    }
    if(s==value)
        *s='\0';
    else s[-1]='\0';
    return value;
}
```

Uses `emalloc()`.

8.3.4 Redirection

⟨`execexec()` *perform the redirections* 68a⟩≡ (62c)

```
doredir(runq->redir);
```

Uses `doredir()` 69a and `runq` 28d.

⟨`start()` *set redir* 68b⟩≡ (35b)

```
p->redir = p->startredir = runq ? runq->redir : nil;
```

Uses `nil` and `runq` 28d.

Redir

⟨*struct* `Redir` 68c⟩≡ (? 0—1)

```
struct Redir {
    // enum<redirection_kind_bis>
    char type; /* what to do */

    /* what to do it to */
    short from;
    short to;

    // Extra
    ⟨Redir extra fields 68e⟩
};
```

⟨`Thread` *other fields* 68d⟩+≡ (28c) ◁40c 72a▷

```
// list<ref_own<Redir>> (next = Redir.next)
struct Redir *redir; /* redirection stack */
```

Uses `Redir` 68c.

⟨`Redir` *extra fields* 68e⟩≡ (68c)

```
struct Redir *next;  /* what else to do (reverse order) */
```

Uses `Redir` 68c.

⟨*function* `pushredir` 68f⟩≡ (? 0—1)

```
void
pushredir(int type, int from, int to)
{
    redir * rp = new(redir);
    rp->type = type;
    rp->from = from;
    rp->to = to;

    // add_list(runq->redir, rp)
    rp->next = runq->redir;
    runq->redir = rp;
}
```

Uses `runq` 28d.

doredir()

⟨*function* doredir 69a⟩≡ (? 0—1)

```
void
doredir(redir *rp)
{
    if(rp){
        // recurse first, so do them in the reverse order of the list
        doredir(rp->next);

        switch(rp->type){
        ⟨doredir() switch redir type cases 69b⟩
        }
    }
}
```

Uses doredir() 69a.

⟨doredir() *switch redir type cases* 69b⟩≡ (69a) 122e ▷

```
case ROPEN:
    if(rp->from != rp->to){
        dup(rp->from, rp->to);
        close(rp->from);
    }
    break;
```

Uses Dup() 140b and ROPEN 69c.

⟨*constant* ROPEN 69c⟩≡ (? 0—1)

```
/*
 * redir types
 */
#define ROPEN 1   /* dup2(from, to); close(from); */
```

Bytecode generation

⟨outcode() *cases* 69d⟩+≡ (57e) ◁ 67b 72e ▷

```
case REDIR:
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xglob);

    switch(t->rtype){
    ⟨outcode() when REDIR case, switch redirection type cases 69e⟩
    }
    emiti(t->fd0);
    outcode(c1, eflag);
    emitf(Xpopredir);
    break;
```

Uses Xglob() 106c, Xmark() 58a, Xpopredir() 70b, c0<> 57b 133b, c1<> 57c 133c, emitf<> 56d, emiti<> 56c, and outcode() 57e.

Xwrite()

⟨outcode() *when REDIR case, switch redirection type cases* 69e⟩≡ (69d) 70c ▷

```
case WRITE:
    emitf(Xwrite);
    break;
```

Uses WRITE, Xwrite() 70a, and emitf<> 56d.

⟨*function* Xwrite 70a⟩≡ (? 0—1)

```
void
Xwrite(void)
{
    char *file;
    fdt f;
    switch(count(runq->argv->words)){
    default:
        Xerror1("> requires singleton\n");
        return;
    case 0:
        Xerror1("> requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = Creat(file))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses Creat() 140b, ROPEN 69c, Xerror() 61c, Xerror1() 62a, count() 29c, err 138b, pfmt() 148b, poplist() 58b, pushredir() 68f, and runq 28d.

⟨*function* Xpopredir 70b⟩≡ (? 0—1)

```
void
Xpopredir(void)
{
    struct Redir *rp = runq->redir;

    if(rp==nil)
        panic("turfredir null!", 0);

    // pop_list(runq->redir);
    runq->redir = rp->next;

    if(rp->type==ROPEN)
        close(rp->from);

    efree((char *)rp);
}
```

Uses ROPEN 69c, Redir 68c, efree(), nil, panic() 139b, and runq 28d.

Xread()

⟨outcode() *when REDIR case, switch redirection type cases* 70c⟩+≡ (69d) ◁69e 71b▷

```
case READ:
case HERE:
    emitf(Xread);
    break;
```

Uses HERE, READ, Xread() 71a, and emitf<> 56d.

⟨*function* Xread 71a⟩≡ (? 0—1)

```
void
Xread(void)
{
    char *file;
    int f;
    switch(count(runq->argv->words)){
    default:
        Xerror1("< requires singleton\n");
        return;
    case 0:
        Xerror1("< requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, 0))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses ROPEN 69c, Xerror() 61c, Xerror1() 62a, count() 29c, err 138b, pfmt() 148b, poplist() 58b, pushredir() 68f, and runq 28d.

Xappend()

⟨outcode() *when REDIR case, switch redirection type cases* 71b⟩+≡ (69d) ◁70c 119d▷

```
case APPEND:
    emitf(Xappend);
    break;
```

Uses APPEND 45d, Xappend() 71c, and emitf<> 56d.

⟨*function* Xappend 71c⟩≡ (? 0—1)

```
void
Xappend(void)
{
    char *file;
    int f;
    switch(count(runq->argv->words)){
    default:
        Xerror1(">> requires singleton");
        return;
    case 0:
        Xerror1(">> requires file");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, 1))<0 && (f = Creat(file))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
```

```
    seek(f, 0L, SEEK__END);
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses `Creat()` 140b, `ROPEN` 69c, `Seek()` 152, `Xerror()` 61c, `Xerror1()` 62a, `count()` 29c, `err` 138b, `pfmt()` 148b, `poplist()` 58b, `pushredir()` 68f, and `runq` 28d.

Closing redirection opened files

⟨`Thread` *other fields* 72a⟩+≡ (28c) ◁68d 74a▷

```
struct Redir *startredir; /* redir inheritance point */
```

Uses `Redir` 68c.

⟨*function* `turfredir` 72b⟩≡ (? 0—1)

```
void
turfredir(void)
{
    while(runq->redir != runq->startredir)
        Xpopredir();
}
```

Uses `Xpopredir()` 70b and `runq` 28d.

⟨`Xreturn()` *pop the redirections from this thread* 72c⟩≡ (73b)

```
turfredir();
```

Uses `turfredir()` 72b.

8.3.5 Pipe

⟨`outcode()` *locals* 72d⟩+≡ (57e) ◁65d 84a▷

```
int q;
```

⟨`outcode()` *cases* 72e⟩+≡ (57e) ◁69d 75a▷

```
case PIPE:
    emitf(Xpipe);
    emiti(t->fd0); // 1 in the normal case
    emiti(t->fd1); // 0 in the normal case
    p = emiti(0);
    q = emiti(0);

    // for first child
    outcode(c0, eflag);
    emitf(Xexit);

    // for second child
    stuffdot(p);
    outcode(c1, eflag);
    emitf(Xreturn);

    // for parent (rc)
    stuffdot(q);
    emitf(Xpipewait);
    break;
```

Uses `Xexit()` 76a, `Xpipe()` 124, `Xpipewait()` 74d, `Xreturn()` 73b, `c0<>` 57b 133b, `c1<>` 57c 133c, `emitf<>` 56d, `emiti<>` 56c, `outcode()` 57e, and `stuffdot()` 66b.

⟨codefree() *in loop over code* cp, *switch bytecode cases* 73a⟩+≡ (28a) ◁60g 82b▷

```
else if(p->f==Xpipe)
        p+=4;
```

Uses Xpipe() 124.

⟨*function* Xreturn 73b⟩≡ (? 0—1)

```
void
Xreturn(void)
{
    struct Thread *p = runq;

    ⟨Xreturn() pop the redirections from this thread 72c⟩
    // free p
    while(p->argv)
        poplist();
    codefree(p->code);
    // pop(runq)
    runq = p->ret;
    efree((char *)p);

    if(runq==nil)
        Exit(getstatus(), __LOC__);
}
```

Uses Exit() 139c, Thread 28c, codefree() 28a, efree(), getstatus() 59a, nil, poplist() 58b, and runq 28d.

⟨*constant* PRD 73c⟩≡ (? 0—1)

```
/*
 * Which fds are the reading/writing end of a pipe?
 * Unfortunately, this can vary from system to system.
 * 9th edition Unix doesn't care, the following defines
 * work on plan 9.
 */
#define PRD 0
```

⟨*constant* PWR 73d⟩≡ (? 0—1)

```
#define PWR 1
```

⟨*function* Xpipe 73e⟩≡ (? 0—1)

```
void
Xpipe(void)
{
    struct Thread *p = runq;
    int pc = p->pc;
    int forkid;
    fdt lfd = p->code[pc++].i;
    fdt rfd = p->code[pc++].i;
    fdt pfd[2];

    if(pipe(pfd)<0){
        Xerror("can't get pipe");
        return;
    }
    switch(forkid = fork()){
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        // pc+2 so jump the jump addresses
        start(p->code, pc+2, runq->local);
```

```
        runq->ret = nil;
        close(pfd[PRD]);
        pushredir(ROPEN, pfd[PWR], lfd);
        break;
    default: // parent
        addwaitpid(forkid);
        start(p->code, p->code[pc].i, runq->local);
        close(pfd[PWR]);
        pushredir(ROPEN, pfd[PRD], rfd);
        p->pc = p->code[pc+1].i;
        p->pid = forkid;
        break;
    }
}
```

Uses ERRMAX, delwaitpid(), havewaitpid() 63d, nil, runq 28d, and setstatus() 58f.

⟨Thread *other fields* 74a⟩+≡ (28c) ◁72a 74b▷

```
// option<int> (None = -1)
int pid;  /* process for Xpipewait to wait for */
```

⟨Thread *other fields* 74b⟩+≡ (28c) ◁74a 77e▷

```
char status[NSTATUS]; /* status for Xpipewait */
```

Uses NSTATUS 74c.

⟨*constant* NSTATUS 74c⟩≡ (? 0—1)

```
#define NSTATUS ERRMAX   /* length of status (from plan 9) */
```

⟨*function* Xpipewait 74d⟩≡ (? 0—1)

```
void
Xpipewait(void)
{
    char status[NSTATUS+1];
    if(runq->pid==-1)
        setstatus(concstatus(runq->status, getstatus()));
    else{
        strncpy(status, getstatus(), NSTATUS);
        status[NSTATUS]='\0';
        Waitfor(runq->pid, true);
        runq->pid=-1;
        setstatus(concstatus(getstatus(), status));
    }
}
```

Uses NSTATUS 74c, Waitfor() 75b, concstatus() 74f, getstatus() 59a, runq 28d, and setstatus() 58f.

⟨Waitfor() *in while loop, if wait returns another pid* 74e⟩≡ (64b)

```
// else
for(p = runq->ret;p;p = p->ret)
    if(p->pid==w->pid){
        p->pid=-1;
        strcpy(p->status, w->msg);
    }
free(w);
```

⟨*function* concstatus 74f⟩≡ (? 0—1)

```
char*
concstatus(char *s, char *t)
{
    static char v[NSTATUS+1];
    int n = strlen(s);
```

```
    strncpy(v, s, NSTATUS);
    if(n<NSTATUS){
        v[n]='|';
        strncpy(v+n+1, t, NSTATUS-n-1);
    }
    v[NSTATUS]='\0';
    return v;
}
```

Uses NSTATUS 74c.

8.3.6 Asynchronous execution

⟨outcode() *cases* 75a⟩+≡ (57e) ◁72e 76b▷

```
case '&':
    emitf(Xasync);
    p = emiti(0);
    outcode(c0, eflag);
    emitf(Xexit);
    stuffdot(p);
    break;
```

Uses Xasync(), Xexit() 76a, c0<> 57b 133b, emitf<> 56d, emiti<> 56c, outcode() 57e, and stuffdot() 66b.

⟨*function* Xasync 75b⟩≡ (? 0—1)

```
void
Xasync(void)
{
    fdt null = open("/dev/null", 0);
    int pid;
    char npid[10];
    if(null<0){
        Xerror("Can't open /dev/null\n");
        return;
    }
    switch(pid = rfork(RFFDG|RFPROC|RFNOTEG)){
    case -1:
        close(null);
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        pushredir(ROPEN, null, 0);
        // start a new Thread runq->pc+1 so skip pointer to code after &
        start(runq->code, runq->pc+1, runq->local);
        runq->ret = 0;
        break;
    default: // parent
        addwaitpid(pid);
        close(null);
        // jump to code after &
        runq->pc = runq->code[runq->pc].i;
        inttoascii(npid, pid);
        setvar("apid", newword(npid, (word *)nil));
        break;
    }
}
```

Uses efree(), err 138b, nil, and pfmt() 148b.

⟨*function* Xexit 76a⟩≡ (? 0—1)

```
void
Xexit(void)
{
    struct Var *trapreq;
    struct Word *starval;
    static bool beenhere = false;

    if(getpid()==mypid && !beenhere){
        trapreq = vlook("sigexit");
        if(trapreq->fn){
            beenhere = true;
            --runq->pc;
            starval = vlook("*")->val;
            start(trapreq->fn, trapreq->pc, (struct Var *)0);
            runq->local = newvar(strdup("*"), runq->local);
            runq->local->val = copywords(starval, (struct Word *)0);
            runq->local->changed = true;
            runq->redir = runq->startredir = nil;
            return;
        }
    }
    Exit(getstatus(), __LOC__);
}
```

Uses `Exit()` 139c, `Var` 45g, `Word`, `copywords()` 29e, `false`, `getstatus()` 59a, `mypid` 104a, `newvar()` 32b, `nil`, `runq` 28d, `start()` 35b, `true`, and `vlook()` 31c.

8.4 Control flow statements

8.4.1 if

⟨`outcode()` *cases* 76b⟩+≡ (57e) ◁75a 77b▷

```
case IF:
    outcode(c0, false);
    emitf(Xif);
    p = emiti(0);
    outcode(c1, eflag);
    emitf(Xwastrue);
    stuffdot(p);
    break;
```

Uses `Xif()` 76c, `Xwastrue()` 77a, `c0<>` 57b 133b, `c1<>` 57c 133c, `emitf<>` 56d, `emiti<>` 56c, `false`, `outcode()` 57e, and `stuffdot()` 66b.

⟨*function* Xif 76c⟩≡ (? 0—1)

```
void
Xif(void)
{
    ifnot = true;
    if(truestatus())
        runq->pc++;
    else
        runq->pc = runq->code[runq->pc].i;
}
```

Uses `ifnot` 76d, `runq` 28d, `true`, and `truestatus()` 59b.

⟨*global* ifnot 76d⟩≡ (? 0—1)

```
bool ifnot;  /* dynamic if not flag */
```

⟨*function* Xwastrue 77a⟩≡ (? 0—1)

```
void
Xwastrue(void)
{
    ifnot = false;
}
```

Uses false and ifnot 76d.

⟨outcode() *cases* 77b⟩+≡ (57e) ◁76b 78a▷

```
case NOT:
    ⟨outcode() when NOT, sanity check last command was an if 77d⟩
    emitf(Xifnot);
    p = emiti(0);
    outcode(c0, eflag);
    stuffdot(p);
    break;
```

Uses Xifnot() 77c, c0<> 57b 133b, emitf<> 56d, emiti<> 56c, outcode() 57e, and stuffdot() 66b.

⟨*function* Xifnot 77c⟩≡ (? 0—1)

```
void
Xifnot(void)
{
    if(ifnot)
        runq->pc++;
    else
        runq->pc = runq->code[runq->pc].i;
}
```

Uses ifnot 76d and runq 28d.

⟨outcode() *when NOT, sanity check last command was an if* 77d⟩≡ (77b)

```
if(!runq->iflast)
    yyerror("‘if not’ does not follow ‘if(...)’");
```

Uses runq 28d and yyerror() 138e.

⟨Thread *other fields* 77e⟩+≡ (28c) ◁74b

```
bool iflast;  /* static ‘if not’ checking */
```

⟨outcode() *set iflast after switch* 77f⟩≡ (57e)

```
if(t->type!=NOT && t->type!=';')
    runq->iflast = t->type==IF;
else
   if(c0)
       runq->iflast = c0->type==IF;
```

Uses c0<> 57b 133b and runq 28d.

⟨outcode() *set iflast before switch* 77g⟩≡ (57e)

```
if(t->type!=NOT && t->type!=';')
    runq->iflast = false;
```

Uses false and runq 28d.

8.4.2 while

⟨outcode() *cases* 78a⟩+≡ (57e) ◁77b 78d▷

```
case WHILE:
    q = codep;
    outcode(c0, false);
    if(q==codep)
        emitf(Xsettrue); /* empty condition == while(true) */
    emitf(Xtrue);
    p = emiti(0);
    outcode(c1, eflag);
    emitf(Xjump);
    emiti(q);
    stuffdot(p);
    break;
```

Uses Xjump() 78c, Xsettrue() 78b, Xtrue() 66d, c0<> 57b 133b, c1<> 57c 133c, codep 56a, emitf<> 56d, emiti<> 56c, false, outcode() 57e, and stuffdot() 66b.

⟨*function* Xsettrue 78b⟩≡ (? 0—1)

```
void
Xsettrue(void)
{
    setstatus("");
}
```

Uses setstatus() 58f.

⟨*function* Xjump 78c⟩≡ (? 0—1)

```
void
Xjump(void)
{
    runq->pc = runq->code[runq->pc].i;
}
```

Uses runq 28d.

8.4.3 for

⟨outcode() *cases* 78d⟩+≡ (57e) ◁78a 79b▷

```
case FOR:
    emitf(Xmark);
    if(c1){
        outcode(c1, eflag);
        emitf(Xglob);
    }
    else{
        emitf(Xmark);
        emitf(Xword);
        emits(strdup("*"));
        emitf(Xdol);
    }
    emitf(Xmark);  /* dummy value for Xlocal */
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xlocal);
    p = emitf(Xfor);
    q = emiti(0);
    outcode(c2, eflag);
    emitf(Xjump);
    emiti(p);
    stuffdot(q);
```

```
    emitf(Xunlocal);
    break;
```

Uses Xdol() 85d, Xfor() 79a, Xglob() 106c, Xjump() 78c, Xlocal() 85a, Xmark() 58a, Xunlocal() 85b, Xword() 57f, c0<> 57b 133b, c1<> 57c 133c, c2<> 57d 133d, emitf<> 56d, emiti<> 56c, emits<> 56e, outcode() 57e, and stuffdot() 66b.

⟨*function* Xfor 79a⟩≡ (? 0—1)

```
void
Xfor(void)
{
    if(runq->argv->words==0){
        poplist();
        runq->pc = runq->code[runq->pc].i;
    }
    else{
        freelist(runq->local->val);
        runq->local->val = runq->argv->words;
        runq->local->changed = true;
        runq->argv->words = runq->argv->words->next;
        runq->local->val->next = 0;
        runq->pc++;
    }
}
```

Uses freelist() 30b, poplist() 58b, runq 28d, and true.

8.4.4 switch

⟨outcode() *cases* 79b⟩+≡ (57e) ◁78d 81c▷

```
case SWITCH:
    codeswitch(t, eflag);
    break;
```

Uses codeswitch() 79c.

⟨*function* codeswitch 79c⟩≡ (? 0—1)

```
/*
 * switch code looks like this:
 * Xmark
 * (get switch value)
 * Xjump 1f
 * out: Xjump leave
 * 1: Xmark
 * (get case values)
 * Xcase 1f
 * (commands)
 * Xjump out
 * 1: Xmark
 * (get case values)
 * Xcase 1f
 * (commands)
 * Xjump out
 * 1:
 * leave:
 * Xpopm
 */
void
codeswitch(tree *t, bool eflag)
{
    int leave;  /* patch jump address to leave switch */
    int out;  /* jump here to leave switch */
```

```
    int nextcase; /* patch jump address to next case */
    tree *tt;

    // c1 is BRACE { ; ; ; }
    if(c1->child[0]==nil
    || c1->child[0]->type != ';'
    || !iscase(c1->child[0]->child[0])){
        yyerror("case missing in switch");
        return;
    }

    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xjump);

    nextcase = emiti(0);
    out = emitf(Xjump);
    leave = emiti(0);

    stuffdot(nextcase);

    // from now on c0, c1, ... refer to this new t
    t = c1->child[0];
    while(t->type==';'){
        tt = c1;
        emitf(Xmark);
        for(t = c0->child[0];t->type==ARGLIST;t = c0)
            outcode(c1, eflag);
        emitf(Xcase);
        nextcase = emiti(0);
        t = tt;
        for(;;){
            if(t->type==';'){
                if(iscase(c0))
                    break;
                outcode(c0, eflag);
                t = c1;
            }
            else{
                if(!iscase(t))
                    outcode(t, eflag);
                break;
            }
        }
        emitf(Xjump);
        emiti(out);
        stuffdot(nextcase);
    }
    stuffdot(leave);
    emitf(Xpopm);
}
```

Uses `Xcase()` 81b, `Xjump()` 78c, `Xmark()` 58a, `Xpopm()` 80, `c0<>` 57b 133b, `c1<>` 57c 133c, `emitf<>` 56d, `emiti<>` 56c, `iscase()` 81a, `nil`, `outcode()` 57e, `stuffdot()` 66b, and `yyerror()` 138e.

⟨*function* `Xpopm` 80⟩≡ (? 0—1)

```
void
Xpopm(void)
{
    poplist();
}
```

Uses poplist() 58b.

⟨*function* iscase 81a⟩≡ (? 0—1)

```
bool
iscase(tree *t)
{
    if(t->type!=SIMPLE)
        return false;
    do { t = c0; } while(t->type==ARGLIST);
    return t->type==WORD && !t->quoted && strcmp(t->str, "case")==0;
}
```

Uses c0<> 57b 133b and false.

⟨*function* Xcase 81b⟩≡ (? 0—1)

```
void
Xcase(void)
{
    word *p;
    char *s;
    bool ok = false;

    s = list2str(runq->argv->next->words);
    for(p = runq->argv->words;p;p = p->next){
        if(match(s, p->word, '\0')){
            ok = true;
            break;
        }
    }
    efree(s);
    if(ok)
        runq->pc++;
    else
        runq->pc = runq->code[runq->pc].i;
    poplist();
}
```

Uses efree(), false, list2str() 67d, match() 109d, poplist() 58b, runq 28d, and true.

8.4.5 Blocks: '{...}'

⟨outcode() *cases* 81c⟩+≡ (57e) ◁79b 81d▷

```
case PAREN:
    outcode(c0, eflag);
    break;
```

Uses c0<> 57b 133b and outcode() 57e.

⟨outcode() *cases* 81d⟩+≡ (57e) ◁81c 82a▷

```
case PCMD:
case BRACE:
    outcode(c0, eflag);
    break;
```

Uses c0<> 57b 133b and outcode() 57e.

8.5 Functions

⟨Var *other fields* 81e⟩≡ (30d) 97d▷

```
code *fn;  /* pointer to function's code vector */
int pc;  /* pc of start of function */
bool fnchanged;
```

8.5.1 Function definitions (fn <foo> ...)

⟨outcode() *cases* 82a⟩+≡ (57e) ◁81d 84b▷

```
case FN:
    emitf(Xmark);
    outcode(c0, eflag);
    if(c1){
        emitf(Xfn);
        p = emiti(0);
        emits(fnstr(c1));
        outcode(c1, eflag); // body of the function
        emitf(Xunlocal); /* get rid of $* */ //$
        emitf(Xreturn);
        stuffdot(p);
    }
    else
        emitf(Xdelfn);
    break;
```

Uses Xdelfn() 83a, Xfn() 82d, Xmark() 58a, Xreturn() 73b, Xunlocal() 85b, c0<> 57b 133b, c1<> 57c 133c, emitf<> 56d, emiti<> 56c, emits<> 56e, fnstr() 82c, outcode() 57e, and stuffdot() 66b.

⟨codefree() *in loop over code* cp*, switch bytecode cases* 82b⟩+≡ (28a) ◁73a 122a▷

```
else if(p->f==Xfn){
        efree(p[2].s);
        p+=2;
    }
```

Uses Xfn() 82d and efree().

⟨*function* fnstr 82c⟩≡ (? 0—1)

```
char*
fnstr(tree *t)
{
    void *v;
    extern char nl;
    char svnl = nl;
    io *f = openstr();

    nl = ';';
    pfmt(f, "%t", t);
    nl = svnl;
    v = f->strp;
    f->strp = nil;
    closeio(f);
    return v;
}
```

Uses closeio() 147b, nil, nl 133a, openstr() 147c, and pfmt() 148b.

⟨*function* Xfn 82d⟩≡ (? 0—1)

```
void
Xfn(void)
{
    var *v;
    word *a;
    int end;

    end = runq->code[runq->pc].i;
    globlist();
    for(a = runq->argv->words;a;a = a->next){
        v = gvlook(a->word);
        if(v->fn)
```

```
            codefree(v->fn);
        v->fn = codecopy(runq->code);
        v->pc = runq->pc+2;
        v->fnchanged = true;
    }
    runq->pc = end;
    poplist();
}
```

Uses codecopy() 27f, codefree() 28a, globlist(), gvlook() 31f, poplist() 58b, runq 28d, and true.

⟨*function* Xdelfn 83a⟩≡ (? 0—1)

```
void
Xdelfn(void)
{
    var *v;
    word *a;

    for(a = runq->argv->words;a;a = a->next){
        v = gvlook(a->word);
        if(v->fn)
            codefree(v->fn);
        v->fn = nil;
        v->fnchanged = true;
    }
    poplist();
}
```

Uses codefree() 28a, gvlook() 31f, nil, poplist() 58b, runq 28d, and true.

8.5.2 Function uses (<foo>(...))

⟨Xsimple() *other locals* 83b⟩≡ (61a) 90c ▷

```
var *v;
```

⟨Xsimple() *if argv0 is a function* 83c⟩≡ (61a)

```
v = gvlook(a->word);
if(v->fn)
    execfunc(v);
```

Uses execfunc() 83d and gvlook() 31f.

⟨*function* execfunc 83d⟩≡ (? 0—1)

```
void
execfunc(var *func)
{
    word *starval;

    popword();
    starval = runq->argv->words;
    runq->argv->words = nil;
    poplist();
    start(func->fn, func->pc, runq->local);
    runq->local = newvar(strdup("*"), runq->local);
    runq->local->val = starval;
    runq->local->changed = true;
}
```

Uses newvar() 32b, nil, poplist() 58b, popword() 58c, runq 28d, start() 35b, and true.

8.6 Variables

8.6.1 Variable definitions (<x>=...)

⟨outcode() *locals* 84a⟩+≡ (57e) ◁72d

```
tree *tt;
```

⟨outcode() *cases* 84b⟩+≡ (57e) ◁82a 85c▷

```
case '=':
    tt = t;
    for(;t && t->type=='='; t = c2);

    if(t){     /* var=value cmd */
        for(t = tt;t->type=='=';t = c2){
            emitf(Xmark);
            outcode(c1, eflag);
            emitf(Xmark);
            outcode(c0, eflag);
            emitf(Xlocal);  /* push var for cmd */
        }
        outcode(t, eflag);  /* gen. code for cmd */
        for(t = tt; t->type == '='; t = c2)
            emitf(Xunlocal); /* pop var */
    }
    else{     /* var=value */
        for(t = tt;t;t = c2){
            emitf(Xmark);
            outcode(c1, eflag);
            emitf(Xmark);
            outcode(c0, eflag);
            emitf(Xassign); /* set var permanently */
        }
    }
    t = tt; /* so tests below will work */
    break;
```

Uses Xassign() 84c, Xlocal() 85a, Xmark() 58a, Xunlocal() 85b, c0<> 57b 133b, c1<> 57c 133c, c2<> 57d 133d, emitf<> 56d, and outcode() 57e.

⟨*function* Xassign 84c⟩≡ (? 0—1)

```
void
Xassign(void)
{
    var *v;
    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    deglob(runq->argv->words->word); // remove the special \001 mark
    v = vlook(runq->argv->words->word);
    poplist();

    globlist();
    freewords(v->val);
    v->val = runq->argv->words;
    v->changed = true;
    runq->argv->words = nil;
    poplist();
}
```

Uses Xerror1() 62a, count() 29c, deglob() 107c, freewords() 30a, globlist(), nil, poplist() 58b, runq 28d, true, and vlook() 31c.

⟨*function* Xlocal 85a⟩≡ (? 0—1)

```
void
Xlocal(void)
{
    if(count(runq->argv->words)!=1){
        Xerror1("variable name must be singleton\n");
        return;
    }
    deglob(runq->argv->words->word);
    runq->local = newvar(strdup(runq->argv->words->word), runq->local);
    poplist();
    globlist();
    runq->local->val = runq->argv->words;
    runq->local->changed = true;
    // change ownership
    runq->argv->words = nil;
    poplist();
}
```

Uses Xerror1() 62a, count() 29c, deglob() 107c, globlist(), newvar() 32b, nil, poplist() 58b, runq 28d, and true.

⟨*function* Xunlocal 85b⟩≡ (? 0—1)

```
void
Xunlocal(void)
{
    var *v = runq->local, *hid;
    if(v==0)
        panic("Xunlocal: no locals!", 0);
    runq->local = v->next;
    hid = vlook(v->name);
    hid->changed = true;
    efree(v->name);
    freewords(v->val);
    efree((char *)v);
}
```

Uses efree(), freewords() 30a, panic() 139b, runq 28d, true, and vlook() 31c.

8.6.2 Variable uses ($<x>)

⟨outcode() *cases* 85c⟩+≡ (57e) ◁84b 86a▷

```
case '$': //$
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xdol);
    break;
```

Uses Xdol() 85d, Xmark() 58a, c0<> 57b 133b, emitf<> 56d, and outcode() 57e.

⟨*function* Xdol 85d⟩≡ (? 0—1)

```
void
Xdol(void)
{
    word *a, *star;
    char *s, *t;
    int n;

    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
```

```
    s = runq->argv->words->word;
    deglob(s);

    n = 0;
    for(t = s;'0'<=*t && *t<='9';t++)
        n = n*10+*t-'0';

    a = runq->argv->next->words;
    if(n==0 || *t)
        a = copywords(vlook(s)->val, a);
    else{
        star = vlook("*")->val;
        if(star && 1 <= n && n <= count(star)){
            while(--n)
                star = star->next;
            a = newword(star->word, a);
        }
    }
    poplist();
    runq->argv->words = a;
}
```

Uses `Xerror1()` 62a, `copywords()` 29e, `count()` 29c, `deglob()` 107c, `newword()` 29b, `poplist()` 58b, `runq` 28d, and `vlook()` 31c.

⟨`outcode()` *cases* 86a⟩+≡ (57e) ◁85c 87a▷

```
case COUNT:
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xcount);
    break;
```

Uses `Xcount()` 86b, `Xmark()` 58a, `c0<>` 57b 133b, `emitf<>` 56d, and `outcode()` 57e.

⟨*function* `Xcount` 86b⟩≡ (? 0—1)

```
void
Xcount(void)
{
    word *a;
    char *s, *t;
    int n;
    char num[12];

    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->words->word;
    deglob(s);

    // n = int_of_string(s)
    n = 0;
    for(t = s;'0'<=*t && *t<='9';t++)
        n = n*10 + *t - '0';
    if(n==0 || *t){
        a = vlook(s)->val;
        inttoascii(num, count(a));
    }
    else{
        a = vlook("*")->val;
        inttoascii(num, (a && 1<=n && n<=count(a)) ? 1 : 0);
    }
    poplist();
```

```
        pushword(num);
    }
```

Uses Xerror1() 62a, count() 29c, deglob() 107c, inttoascii(), poplist() 58b, pushword() 36a, runq 28d, and vlook() 31c.

⟨outcode() *cases* 87a⟩+≡ (57e) ◁86a 88a▷

```
    case SUB:
        emitf(Xmark);
        outcode(c0, eflag);
        emitf(Xmark);
        outcode(c1, eflag);
        emitf(Xsub);
        break;
```

Uses Xmark() 58a, Xsub() 87b, c0<> 57b 133b, c1<> 57c 133c, emitf<> 56d, and outcode() 57e.

⟨*function* Xsub 87b⟩≡ (? 0—1)

```
    void
    Xsub(void)
    {
        word *a, *v;
        char *s;
        if(count(runq->argv->next->words)!=1){
            Xerror1("variable name not singleton!");
            return;
        }
        s = runq->argv->next->words->word;
        deglob(s);
        a = runq->argv->next->next->words;
        v = vlook(s)->val;
        a = subwords(v, count(v), runq->argv->words, a);
        poplist();
        poplist();
        runq->argv->words = a;
    }
```

Uses Xerror1() 62a, count() 29c, deglob() 107c, poplist() 58b, runq 28d, subwords() 87c, and vlook() 31c.

⟨*function* subwords 87c⟩≡ (? 0—1)

```
    word*
    subwords(word *val, int len, word *sub, word *a)
    {
        int n, m;
        char *s;
        if(!sub)
            return a;
        a = subwords(val, len, sub->next, a);
        s = sub->word;
        deglob(s);
        m = 0;
        n = 0;
        while('0'<=*s && *s<='9')
            n = n*10+ *s++ -'0';
        if(*s == '-'){
            if(*++s == 0)
                m = len - n;
            else{
                while('0'<=*s && *s<='9')
                    m = m*10+ *s++ -'0';
                m -= n;
            }
        }
```

```
    if(n<1 || n>len || m<0)
        return a;
    if(n+m>len)
        m = len-n;
    while(--n > 0)
        val = val->next;
    return copynwords(val, a, m+1);
}
```

Uses copynwords() 29d, deglob() 107c, and subwords() 87c.

⟨outcode() *cases* 88a⟩+≡ (57e) ◁87a 115c▷

```
case '^':
    emitf(Xmark);
    outcode(c1, eflag);
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xconc);
    break;
```

Uses Xconc() 88b, Xmark() 58a, c0<> 57b 133b, c1<> 57c 133c, emitf<> 56d, and outcode() 57e.

⟨*function* Xconc 88b⟩≡ (? 0—1)

```
void
Xconc(void)
{
    word *lp = runq->argv->words;
    word *rp = runq->argv->next->words;
    word *vp = runq->argv->next->next->words;
    int lc = count(lp), rc = count(rp);
    if(lc!=0 || rc!=0){
        if(lc==0 || rc==0){
            Xerror1("null list in concatenation");
            return;
        }
        if(lc!=1 && rc!=1 && lc!=rc){
            Xerror1("mismatched list lengths in concatenation");
            return;
        }
        vp = conclist(lp, rp, vp);
    }
    poplist();
    poplist();
    runq->argv->words = vp;
}
```

Uses Xerror1() 62a, conclist() 88c, count() 29c, poplist() 58b, and runq 28d.

⟨*function* conclist 88c⟩≡ (? 0—1)

```
word*
conclist(word *lp, word *rp, word *tail)
{
    char *buf;
    word *v;
    if(lp->next || rp->next)
        tail = conclist(lp->next==0? lp: lp->next,
            rp->next==0? rp: rp->next, tail);
    buf = emalloc(strlen(lp->word)+strlen((char *)rp->word)+1);
    strcpy(buf, lp->word);
    strcat(buf, rp->word);
    v = newword(buf, tail);
    efree(buf);
```

```
    return v;
}
```

Uses `conclist()` 88c, `efree()`, `emalloc()`, and `newword()` 29b.

8.6.3 Special variables

Chapter 9

Builtins

9.1 Overview

⟨*struct* Builtin 90a⟩≡ (? 0—1)

```
struct Builtin {
    char *name;
    void (*fnc)(void);
};
```

⟨*global* builtins 90b⟩≡ (? 0—1)

```
builtin builtins[] = {
    "cd"     ,      execcd,
    "exit"   ,      execexit,
    "."      ,      execdot,
    "eval"   ,      execeval,

    "whatis" ,      execwhatis,

    "exec"   ,      execexec,   /* but with popword first */
    "rfork"  ,      execnewpgrp,
    "wait"   ,      execwait,

    "shift"  ,      execshift,
    "finit"  ,      execfinit,
    "flag"   ,      execflag,
    //TODO: unix-specific "umask", execumask,
    0
};
```

Uses execdot() 93b, execeval() 95b, execexec() 62c, execfinit() 130b, execflag() 129a, execnewpgrp() 128, execshift() 129b, execwait() 96b, and execwhatis() 126.

⟨Xsimple() *other locals* 90c⟩+≡ (61a) ◁83b

```
struct Builtin *bp;
```

Uses Builtin 90a.

⟨Xsimple() *if argv0 is a builtin* 90d⟩≡ (61a)

```
⟨Xsimple() if argv0 is the builtin keyword 91a⟩
for(bp = builtins;bp->name;bp++)
    if(strcmp(a->word, bp->name)==0){
        (*bp->fnc)();
        return;
    }
```

Uses builtins.

⟨Xsimple() *if argv0 is the builtin keyword* 91a⟩≡ (90d)

```
if(strcmp(a->word, "builtin")==0){
    if(count(a)==1){
        pfmt(err, "builtin: empty argument list\n");
        setstatus("empty arg list");
        poplist();
        return;
    }
    a = a->next;
    popword();
}
```

Uses count() 29c, err 138b, pfmt() 148b, poplist() 58b, popword() 58c, and setstatus() 58f.

9.2 $ cd

⟨*function* execcd 91b⟩≡ (? 0—1)

```
void
execcd(void)
{
    word *a = runq->argv->words;
    word *cdpath;
    char *dir;

    setstatus("can't cd");
    cdpath = vlook("cdpath")->val;

    switch(count(a)){
    case 1:
        a = vlook("home")->val;
        if(count(a)>=1){
            if(dochdir(a->word)>=0)
                setstatus("");
            else
                pfmt(err, "Can't cd %s: %r\n", a->word);
        }
        else
            pfmt(err, "Can't cd -- $home empty\n"); //$
        break;
    case 2:
        if(a->next->word[0]=='/' || cdpath==nil)
            cdpath = &nullpath;
        for(; cdpath; cdpath = cdpath->next){
            if(cdpath->word[0] != '\0')
                dir = appfile(cdpath->word, a->next->word);
            else
                dir = strdup(a->next->word);

            if(dochdir(dir) >= 0){
                if(cdpath->word[0] != '\0' &&
                    strcmp(cdpath->word, ".") != 0)
                    pfmt(err, "%s\n", dir);
                free(dir);
                setstatus("");
                break;
            }
            free(dir);
        }
        if(cdpath==nil)
```

```
            pfmt(err, "Can't cd %s: %r\n", a->next->word);
        break;
    default:
        pfmt(err, "Usage: cd [directory]\n");
        break;
    }
    poplist();
}
```

Uses appfile<>() 92a, count() 29c, dochdir() 92b, err 138b, nil, nullpath 63c, pfmt() 148b, poplist() 58b, runq 28d, setstatus() 58f, and vlook() 31c.

⟨*function* appfile 92a⟩≡ (? 0—1)

```
static char *
appfile(char *dir, char *comp)
{
    int dirlen, complen;
    char *s, *p;

    dirlen = strlen(dir);
    complen = strlen(comp);
    s = emalloc(dirlen + 1 + complen + 1);
    memmove(s, dir, dirlen);
    p = s + dirlen;
    *p++ = '/';
    memmove(p, comp, complen);
    p[complen] = '\0';
    return s;
}
```

Uses emalloc().

⟨*function* dochdir 92b⟩≡ (? 0—1)

```
errorneg1
dochdir(char *word)
{
    ⟨dochdir() locals 92c⟩

    // the actual syscall
    if(chdir(word)<0)
        return ERROR_NEG1;

    ⟨dochdir() adjust /dev/wdir if run under rio 92d⟩
    return OK_1;
}
```

Uses ERROR_NEG1 and OK_1.

⟨dochdir() *locals* 92c⟩≡ (92b)

```
/* report to /dev/wdir if it exists and we're interactive */
static fdt wdirfd = -2;
```

⟨dochdir() *adjust* /dev/wdir *if run under* rio 92d⟩≡ (92b)

```
if(flag['i']!=nil){
    if(wdirfd==-2)  /* try only once */
        wdirfd = open("/dev/wdir", OWRITE); // TODO: |OCEXEC but plan9 specific?
    if(wdirfd>=0) {
        //fcntl(wdirfd, F_SETFD, FD_CLOEXEC);
        write(wdirfd, word, strlen(word));
    }
}
```

Uses OCEXEC, OWRITE, flag 33c, and nil.

WINDOWS book [Pad16b]

9.3 $ exit

⟨*function* execexit 93a⟩≡ (? 0—1)

```
void
execexit(void)
{
    switch(count(runq->argv->words)){
    default:
        pfmt(err, "Usage: exit [status]\nExiting anyway\n");
        // FALLTHROUGH
    case 2:
        setstatus(runq->argv->words->next->word);
        // FALLTHROUGH
    case 1:
        Xexit();
    }
}
```

Uses Xexit() 76a, count() 29c, err 138b, pfmt() 148b, runq 28d, and setstatus() 58f.

9.4 $.

⟨*function* execdot 93b⟩≡ (? 0—1)

```
void
execdot(void)
{
    thread *p = runq;
    bool iflag = false;
    fdt fd;
    list *av;
    char *zero; // new stdin
    char *file;
    word *path;
    ⟨execdot() other locals 94c⟩

    ⟨execdot() if first 95a⟩
    ⟨execdot() if not first execution 114d⟩

    popword(); // "."
    if(p->argv->words && strcmp(p->argv->words->word, "-i")==0){
        iflag = true;
        popword();
    }

    /* get input file */
    if(p->argv->words==nil){
        Xerror1("Usage: . [-i] file [arg ...]");
        return;
    }
    zero = strdup(p->argv->words->word);
    popword();

    fd = -1;
    for(path = searchpath(zero); path; path = path->next){
        if(path->word[0] != '\0')
```

```
            file = appfile(path->word, zero);
        else
            file = strdup(zero);

        fd = open(file, 0);
        free(file);
        if(fd >= 0)
            break;
    }
    if(fd<0){
        pfmt(err, "%s: ", zero);
        setstatus("can't open"); // what for? it is reseted by Xerror anyway
        Xerror(".: can't open");
        return;
    }

    /* set up for a new command loop */
    start(dotcmds, 1, (struct Var *)nil);

    pushredir(RCLOSE, fd, 0);
    runq->cmdfile = zero;
    runq->cmdfd = openfd(fd);
    runq->iflag = iflag;

    runq->iflast = false;

    /* push $* value */
    pushlist();
    runq->argv->words = p->argv->words;

    /* free caller's copy of $* */
    av = p->argv;
    p->argv = av->next;
    efree((char *)av);

    /* push $0 value */
    pushlist();
    pushword(zero);

    ndot++;
}
```

Uses RCLOSE 122c, Var 45g, Xerror() 61c, Xerror1() 62a, appfile<>() 92a, dotcmds 94b, efree(), err 138b, false, ndot 94a, nil, openfd() 145f, pfmt() 148b, popword() 58c, pushlist() 35e, pushredir() 68f, pushword() 36a, runq 28d, searchpath() 63b, setstatus() 58f, start() 35b, and true.

⟨*global* ndot 94a⟩≡ (? 0—1)

```
/*
 * How many dot commands have we executed?
 * Used to ensure that -v flag doesn't print rcmain.
 */
int ndot;
```

⟨*global* dotcmds 94b⟩≡ (? 0—1)

```
union Code dotcmds[14];
```

Uses Code 25b.

⟨execdot() *other locals* 94c⟩≡ (93b)

```
static bool first = true;
```

Uses true.

⟨execdot() *if first* 95a⟩≡ (93b)

```
if(first){
    dotcmds[0].i = 1;
    dotcmds[1].f = Xmark;
      dotcmds[2].f = Xword;
      dotcmds[3].s = "0";
    dotcmds[4].f = Xlocal; // will pop_list twice

    dotcmds[5].f = Xmark;
      dotcmds[6].f = Xword;
      dotcmds[7].s="*";
    dotcmds[8].f = Xlocal; // will pop_list twice

    dotcmds[9].f = Xrdcmds; // =~ a REPL

    dotcmds[10].f = Xunlocal;
    dotcmds[11].f = Xunlocal;
    dotcmds[12].f = Xreturn;

    first = false;
}
```

Uses `Xlocal()` 85a, `Xmark()` 58a, `Xrdcmds()` 36c, `Xreturn()` 73b, `Xunlocal()` 85b, `Xword()` 57f, `dotcmds` 94b, and `false`.

9.5 $ eval

⟨*function* execeval 95b⟩≡ (? 0—1)

```
void
execeval(void)
{
    char *cmdline, *s, *t;
    int len = 0;
    word *ap;

    if(count(runq->argv->words)<=1){
        Xerror1("Usage: eval cmd ...");
        return;
    }
    eflagok = true;
    for(ap = runq->argv->words->next;ap;ap = ap->next)
        len+=1+strlen(ap->word);

    cmdline = emalloc(len);
    s = cmdline;
    for(ap = runq->argv->words->next;ap;ap = ap->next){
        for(t = ap->word;*t;) *s++=*t++;
        *s++=' ';
    }
    s[-1]='\n';
    poplist();

    execcmds(opencore(cmdline, len));
    efree(cmdline);
}
```

Uses `Xerror1()` 62a, `count()` 29c, `eflagok` 114c, `efree()`, `emalloc()`, `execcmds()` 96a, `opencore()` 147d, `poplist()` 58b, `runq` 28d, and `true`.

⟨*global* rdcmds 95c⟩≡ (? 0—1)

```
union Code rdcmds[4];
```

Uses Code 25b.

⟨*function* execcmds 96a⟩≡ (? 0—1)

```
void
execcmds(io *f)
{
    static bool first = true;
    if(first){
        rdcmds[0].i = 1;
        rdcmds[1].f = Xrdcmds;
        rdcmds[2].f = Xreturn;
        first = false;
    }

    start(rdcmds, 1, runq->local);
    runq->cmdfd = f;
    runq->iflast = false;
}
```

Uses Xrdcmds() 36c, Xreturn() 73b, false, rdcmds 95c, runq 28d, start() 35b, and true.

9.6 $ wait

⟨*function* execwait 96b⟩≡ (? 0—1)

```
void
execwait(void)
{
    switch(count(runq->argv->words)){
    default:
        Xerror1("Usage: wait [pid]");
        return;
    case 2:
        Waitfor(atoi(runq->argv->words->next->word), false);
        break;
    case 1:
        Waitfor(-1, false);
        break;
    }
    poplist();
}
```

Uses Waitfor() 75b, Xerror1() 62a, count() 29c, poplist() 58b, and runq 28d.

Chapter 10

Environment

10.1 Updenv()

⟨*function* Updenv 97a⟩≡ (? 0—1)

```
void
Updenv(void)
{
    var *v, **h;

    for(h = gvar;h!=&gvar[NVAR];h++)
        for(v=*h;v;v = v->next)
            addenv(v);
    if(runq)
        updenvlocal(runq->local);
}
```

⟨*function* updenvlocal 97b⟩≡ (? 0—1)

```
void
updenvlocal(var *v)
{
    if(v){
        updenvlocal(v->next);
        addenv(v);
    }
}
```

⟨*enum* MiscPlan9 97c⟩≡ (? 0—1)

```
enum {
    Maxenvname = 256, /* undocumented limit */
};
```

⟨Var *other fields* 97d⟩+≡ (30d) ◁81e

```
bool changed;
```

⟨*function* addenv 97e⟩≡ (? 0—1)

```
void
addenv(var *v)
{
    char envname[Maxenvname];
    word *w;
    int f;
    io *fd;

    if(v->changed){
        v->changed = false;
```

```
        snprint(envname, sizeof envname, "/env/%s", v->name);
        if((f = Creat(envname))<0)
            pfmt(err, "rc: can't open %s: %r\n", envname);
        else{
            for(w = v->val;w;w = w->next)
                write(f, w->word, strlen(w->word)+1L);
            close(f);
        }
    }
    if(v->fnchanged){
        v->fnchanged = false;
        snprint(envname, sizeof envname, "/env/fn#%s", v->name);
        if((f = Creat(envname))<0)
            pfmt(err, "rc: can't open %s: %r\n", envname);
        else{
            if(v->fn){
                fd = openfd(f);
                pfmt(fd, "fn %q %s\n", v->name, v->fn[v->pc-1].s);
                closeio(fd);
            }
            close(f);
        }
    }
}
```

10.2 Vinit()

⟨*function* Vinit 98⟩≡ (? 0—1)

```
void
Vinit(void)
{
    int dir, f, len, i, n, nent;
    char *buf, *s;
    char envname[Maxenvname];
    word *val;
    Dir *ent;

    dir = open("/env", OREAD);
    if(dir<0){
        pfmt(err, "rc: can't open /env: %r\n");
        return;
    }
    ent = nil;
    for(;;){
        nent = dirread(dir, &ent);
        if(nent <= 0)
            break;
        for(i = 0; i<nent; i++){
            len = ent[i].length;
            if(len && strncmp(ent[i].name, "fn#", 3)!=0){
                snprint(envname, sizeof envname, "/env/%s", ent[i].name);
                if((f = open(envname, 0))>=0){
                    buf = emalloc(len+1);
                    n = readn(f, buf, len);
                    if (n <= 0)
                        buf[0] = '\0';
                    else
                        buf[n] = '\0';
```

```
                    val = 0;
                    /* Charitably add a 0 at the end if need be */
                    if(buf[len-1])
                        buf[len++]='\0';
                    s = buf+len-1;
                    for(;;){
                        while(s!=buf && s[-1]!='\0') --s;
                        val = newword(s, val);
                        if(s==buf)
                            break;
                        --s;
                    }
                    setvar(ent[i].name, val);
                    vlook(ent[i].name)->changed = false;
                    close(f);
                    efree(buf);
                }
            }
        }
        free(ent);
    }
    close(dir);
}
```

⟨*global* envdir 99⟩≡ (? 0—1)

```
fdt envdir;
```

Chapter 11

Signals

⟨*constant* NSIG 100a⟩≡ (? 0—1)
```
#define NSIG 32
```

⟨*constant* SIGINT 100b⟩≡ (? 0—1)
```
#define SIGINT 2
```

⟨*constant* SIGQUIT 100c⟩≡ (? 0—1)
```
#define SIGQUIT 3
```

⟨*global* syssigname 100d⟩≡ (? 0—1)
```
char *syssigname[] = {
    "exit",  /* can't happen */
    "hangup",
    "interrupt",
    "quit",  /* can't happen */
    "alarm",
    "kill",
    "sys: fp: ",
    "term",
    0
};
```

⟨*global* signame 100e⟩≡ (? 0—1)
```
char *signame[] = {
    "sigexit",
    "sighup",
    "sigint",
    "sigquit",
    "sigalrm",
    "sigkill",
    "sigfpe",
    "sigterm",
    0
};
```

⟨*global* ntrap 100f⟩≡ (? 0—1)
```
int ntrap;    /* number of outstanding traps */
```

⟨*global* trap 100g⟩≡ (? 0—1)
```
int trap[NSIG];    /* number of outstanding traps per type */
```

⟨*function* Trapinit 100h⟩≡ (? 0—1)
```
void
Trapinit(void)
{
    notify(notifyf);
}
```

⟨*function* notifyf 101a⟩≡ (? 0—1)

```
void
notifyf(void*, char *s)
{
    int i;
    for(i = 0;syssigname[i];i++)
     if(strncmp(s, syssigname[i], strlen(syssigname[i]))==0){
        if(strncmp(s, "sys: ", 5)!=0)
            interrupted = true;
        goto Out;
    }
    pfmt(err, "rc: note: %s\n", s);
    noted(NDFLT);
    return;
Out:
    if(strcmp(s, "interrupt")!=0 || trap[i]==0){
        trap[i]++;
        ntrap++;
    }
    if(ntrap>=32){ /* rc is probably in a trap loop */
        pfmt(err, "rc: Too many traps (trap %s), aborting\n", s);
        abort();
    }
    noted(NCONT);
}
```

⟨main() *handing trap if necessary in interpreter loop* 101b⟩≡ (36b)

```
if(ntrap)
    dotrap();
```

⟨*function* dotrap 101c⟩≡ (? 0—1)

```
void
dotrap(void)
{
    int i;
    struct Var *trapreq;
    struct Word *starval;

    starval = vlook("*")->val;
    while(ntrap)
     for(i = 0;i!=NSIG;i++)
      while(trap[i]){
        --trap[i];
        --ntrap;
        if(getpid()!=mypid)
            Exit(getstatus(), __LOC__);
        trapreq = vlook(signame[i]);
        if(trapreq->fn){
            start(trapreq->fn, trapreq->pc, (struct Var *)nil);
            runq->local = newvar(strdup("*"), runq->local);
            runq->local->val = copywords(starval, (struct Word *)nil);
            runq->local->changed = true;
            runq->redir = runq->startredir = nil;
        }
        else if(i==SIGINT || i==SIGQUIT){
            /*
             * run the stack down until we uncover the
             * command reading loop.  Xreturn will exit
             * if there is none (i.e. if this is not
             * an interactive rc.)
```

```
            */
            while(!runq->iflag)
                Xreturn();
        }
        else
            Exit(getstatus(), __LOC__);
    }
}
```

Uses NSIG<>, false, interrupted 101c, signame 101c, and syssigname 101c.

⟨*global* interrupted 102a⟩≡ (? 0—1)

```
bool interrupted = false;
```

⟨*function* Eintr 102b⟩≡ (? 0—1)

```
bool
Eintr(void)
{
    return interrupted;
}
```

⟨Xrdcmds() *calls Noerror() before yyparse()* 102c⟩≡ (36c)

```
Noerror();
```

Uses Noerror().

⟨*function* Noerror 102d⟩≡ (? 0—1)

```
void
Noerror(void)
{
    interrupted = false;
}
```

⟨Xrdcmds() *reset ntrap* 102e⟩≡ (36c)

```
ntrap = 0; /* avoid double-interrupts during blocked writes */
```

Uses ntrap.

Chapter 12

Initialization

12.1 Actual bootstrapping code

⟨main() *initialize* boostrap 103a⟩≡ (33b)

```
memset(bootstrap, 0, sizeof bootstrap);

i = 0;
bootstrap[i++].i=1;
// runq->argv is populated with the arguments to rc
// we just need to add '*=(argv)'
bootstrap[i++].f = Xmark;
  bootstrap[i++].f = Xword;
  bootstrap[i++].s="*";
bootstrap[i++].f = Xassign; // will pop_list() x2

bootstrap[i++].f = Xmark;
  bootstrap[i++].f = Xmark;
    bootstrap[i++].f = Xword;
    bootstrap[i++].s="*";
  bootstrap[i++].f = Xdol; // will pop_list()
  bootstrap[i++].f = Xword;
  bootstrap[i++].s = rcmain;
  bootstrap[i++].f = Xword;
  bootstrap[i++].s=".";
bootstrap[i++].f = Xsimple; // will pop_list()

bootstrap[i++].f = Xexit;

bootstrap[i].i = 0;
```

Uses Rcmain, STDERR, Trapinit(), Vinit(), err 138b, flag 33c, inttoascii(), kinit() 48e, mypid 104a, newword() 29b, nil, openfd() 145f, and setvar() 31b.

12.2 Intitialization script and rc -m /path/to/rcmain

⟨*global* Rcmain 103b⟩≡ (? 0—1)

```
char *Rcmain = "/rc/lib/rcmain";
```

⟨main() *locals* 103c⟩+≡ (33b) ◁?? 104b▷

```
char *rcmain;
```

⟨main() *initialisation* 103d⟩+≡ (33b) ◁34e 104c▷

```
rcmain = flag['m'] ? flag['m'][0] : Rcmain;
```

12.3 Actual environment

⟨*global* `mypid` 104a⟩≡ (? 0—1)

```
int mypid;
```

⟨`main()` *locals* 104b⟩+≡ (33b) ◁103c

```
char num[12];
```

⟨`main()` *initialisation* 104c⟩+≡ (33b) ◁103d 104d▷

```
mypid = getpid();
inttoascii(num, mypid);
setvar("pid", newword(num, (word *)nil));
```

Uses `flag` 33c and `flagset` 33e.

⟨`main()` *initialisation* 104d⟩+≡ (33b) ◁104c 114a▷

```
setvar("rcname", newword(argv[0], (word *)nil));
```

Uses `flag` 33c.

Chapter 13

Globbing

13.1 Lexing globbing characters

⟨*constant* GLOB 105a⟩≡ (? 0—1)

```
/*
 * Glob character escape in strings:
 * In a string, GLOB must be followed by *?[ or GLOB.
 * GLOB* matches any string
 * GLOB? matches any single character
 * GLOB[...] matches anything in the brackets
 * GLOBGLOB matches GLOB
 */
#define GLOB '\001'
```

⟨yylex() *when c is a word character, if glob character* 105b⟩≡ (47c)

```
if(c=='*' || c=='[' || c=='?' || c==GLOB)
    w = addtok(w, GLOB);
```

Uses GLOB 30d and addtok() 105c.

⟨*function* addtok 105c⟩≡ (? 0—1)

```
char*
addtok(char *p, int val)
{
    if(p==nil)
        return nil;
    if(p >= &tok[NTOK]){
        *p = '\0';
        yyerror("token buffer too short");
        return nil;
    }
    *p++=val;
    return p;
}
```

Uses NTOK<> 43c, nil, tok 43b, and yyerror() 138e.

13.2 Expanding globbing characters

⟨*global* globv 105d⟩≡ (? 0—1)

```
struct Word *globv;
```

⟨Xsimple() *initializations,* globlist() 105e⟩≡ (61a)

```
globlist();
```

Uses globlist().

⟨*function* `globlist` 106a⟩≡ (? 0—1)

```
void
globlist(void)
{
    word *a;
    globv = nil;

    globlist1(runq->argv->words);
    poplist();
    pushlist();

    if(globv){
        for(a = globv;a->next;a = a->next)
            ;
        a->next = runq->argv->words;
        runq->argv->words = globv;
    }
}
```

Uses `false`, `glob()`, and `globlist1()` 106a.

⟨*function* `globlist1` 106b⟩≡ (? 0—1)

```
void
globlist1(word *gl)
{
    if(gl){
        globlist1(gl->next);
        glob(gl->word);
    }
}
```

Uses `false`, `nextutf()` 109c, and `unicode()` 109c.

⟨*function* `Xglob` 106c⟩≡ (? 0—1)

```
void
Xglob(void)
{
    globlist();
}
```

Uses `globlist()`.

13.3 glob()

⟨*global* `globname` 106d⟩≡ (? 0—1)

```
char *globname;
```

⟨*function* `glob` 106e⟩≡ (? 0—1)

```
/*
 * Push all file names matched by p on the current thread's stack.
 * If there are no matches, the list consists of p.
 */
void
glob(void *ap)
{
    uchar *p = ap;
    word *svglobv = globv;
    int globlen = Globsize(ap);

    if(!globlen){
        deglob(p);
```

```
        globv = newword((char *)p, globv);
        return;
    }
    globname = emalloc(globlen);
    globname[0]='\0';
    globdir(p, (uchar *)globname);
    efree(globname);
    if(svglobv==globv){
        deglob(p);
        globv = newword((char *)p, globv);
    }
    else
        globsort(globv, svglobv);
}
```

Uses `Closedir()`, `Opendir()` 151c, `Readdir()`, `globdir()` 108a, `globname`, `globv` 107b, `matchfn()` 109d, and `newword()` 29b.

⟨*function* `deglob` 107a⟩≡ (? 0—1)

```
/*
 * delete all the GLOB marks from s, in place
 */
void
deglob(void *as)
{
    char *s = as;
    char *t = s;
    do{
        if(*t==GLOB)
            t++;
        *s++=*t;
    }while(*t++);
}
```

Uses `GLOB` 30d and `NDIR<>`.

⟨*function* `globcmp` 107b⟩≡ (? 0—1)

```
int
globcmp(const void *s, const void *t)
{
    return strcmp(*(char**)s, *(char**)t);
}
```

Uses `Word`.

⟨*function* `globsort` 107c⟩≡ (? 0—1)

```
void
globsort(word *left, word *right)
{
    char **list;
    word *a;
    int n = 0;
    for(a = left;a!=right;a = a->next) n++;
    list = (char **)emalloc(n*sizeof(char *));
    for(a = left,n = 0;a!=right;a = a->next,n++) list[n] = a->word;
    qsort((void *)list, n, sizeof(void *), globcmp);
    for(a = left,n = 0;a!=right;a = a->next,n++) a->word = list[n];
    efree((char *)list);
}
```

Uses `GLOB` 30d.

⟨*function* globdir 108a⟩≡ (? 0—1)

```
/*
 * Push names prefixed by globname and suffixed by a match of p onto the astack.
 * namep points to the end of the prefix in globname.
 */
void
globdir(uchar *p, uchar *namep)
{
    uchar *t, *newp;
    int f;
    /* scan the pattern looking for a component with a metacharacter in it */
    if(*p=='\0'){
        globv = newword(globname, globv);
        return;
    }
    t = namep;
    newp = p;
    while(*newp){
        if(*newp==GLOB)
            break;
        *t=*newp++;
        if(*t++=='/'){
            namep = t;
            p = newp;
        }
    }
    /* If we ran out of pattern, append the name if accessible */
    if(*newp=='\0'){
        *t='\0';
        if(access(globname, 0)==0)
            globv = newword(globname, globv);
        return;
    }
    /* read the directory and recur for any entry that matches */
    *namep='\0';
    if((f = Opendir(globname[0]?globname:"."))<0) return;
    while(*newp!='/' && *newp!='\0') newp++;
    while(Readdir(f, namep, *newp=='/')){
        if(matchfn(namep, p)){
            for(t = namep;*t;t++);
            globdir(newp, t);
        }
    }
    Closedir(f);
}
```

Uses GLOB 30d, efree(), emalloc(), globcmp() 108a, globname, globv 107b, and newword() 29b.

⟨*function* equtf 108b⟩≡ (? 0—1)

```
/*
 * Do p and q point at equal utf codes
 */
bool
equtf(uchar *p, uchar *q)
{
    Rune pr, qr;
    if(*p!=*q)
        return false;

    chartorune(&pr, (char*)p);
    chartorune(&qr, (char*)q);
```

```
    return pr == qr;
}
```

Uses Globsize(), deglob() 107c, efree(), emalloc(), globdir() 108a, globname, globv 107b, and newword() 29b.

⟨*function* nextutf 109a⟩≡ (? 0—1)

```
/*
 * Return a pointer to the next utf code in the string,
 * not jumping past nuls in broken utf codes!
 */
uchar*
nextutf(uchar *p)
{
    Rune dummy;
    return p + chartorune(&dummy, (char*)p);
}
```

Uses globsort() 108a and globv 107b.

⟨*function* unicode 109b⟩≡ (? 0—1)

```
/*
 * Convert the utf code at *p to a unicode value
 */
int
unicode(uchar *p)
{
    Rune r;

    chartorune(&r, (char*)p);
    return r;
}
```

Uses false.

13.4 match()

⟨*function* matchfn 109c⟩≡ (? 0—1)

```
/*
 * Does the string s match the pattern p
 * . and .. are only matched by patterns starting with .
 * * matches any sequence of characters
 * ? matches any single character
 * [...] matches the enclosed list of characters
 */
bool
matchfn(void *as, void *ap)
{
    uchar *s = as, *p = ap;

    if(s[0]=='.' && (s[1]=='\0' || s[1]=='.' && s[2]=='\0') && p[0]!='.')
        return false;
    return match(s, p, '/');
}
```

⟨*function* match 109d⟩≡ (? 0—1)

```
bool
match(void *as, void *ap, int stop)
{
    int compl, hit, lo, hi, t, c;
    uchar *s = as, *p = ap;
```

```
    for(; *p!=stop && *p!='\0'; s = nextutf(s), p = nextutf(p)){
        if(*p!=GLOB){
            if(!equtf(p, s)) return false;
        }
        else switch(*++p){
        case GLOB:
            if(*s!=GLOB)
                return false;
            break;
        case '*':
            for(;;){
                if(match(s, nextutf(p), stop)) return 1;
                if(!*s)
                    break;
                s = nextutf(s);
            }
            return false;
        case '?':
            if(*s=='\0')
                return false;
            break;
        case '[':
            if(*s=='\0')
                return false;
            c = unicode(s);
            p++;
            compl=*p=='~';
            if(compl)
                p++;
            hit = 0;
            while(*p!=']'){
                if(*p=='\0')
                    return false;  /* syntax error */
                lo = unicode(p);
                p = nextutf(p);
                if(*p!='-')
                    hi = lo;
                else{
                    p++;
                    if(*p=='\0')
                        return false; /* syntax error */
                    hi = unicode(p);
                    p = nextutf(p);
                    if(hi<lo){ t = lo; lo = hi; hi = t; }
                }
                if(lo<=c && c<=hi)
                    hit = 1;
            }
            if(compl)
                hit=!hit;
            if(!hit)
                return false;
            break;
        }
    }
    return *s=='\0';
}
```

Uses GLOB 30d, equtf() 109a, false, match() 109d, nextutf() 109c, and unicode() 109c.

⟨*constant* NDIR 110⟩≡ (? 0—1)

```
#define NDIR 256  /* shoud be a better way */
```

⟨*function* Globsize 111a⟩≡ (? 0—1)

```
int
Globsize(char *p)
{
    int isglob = 0, globlen = NDIR+1;
    for(;*p;p++){
        if(*p==GLOB){
            p++;
            if(*p!=GLOB)
                isglob++;
            globlen+=*p=='*'?NDIR:1;
        }
        else
            globlen++;
    }
    return isglob?globlen:0;
}
```

⟨*constant* NFD 111b⟩≡ (? 0—1)

```
#define NFD 50
```

⟨*struct* DirEntryWrapper 111c⟩≡ (? 0—1)

```
struct DirEntryWrapper {
    Dir *dbuf;
    int i;
    int n;
};
```

⟨*global* dir 111d⟩≡ (? 0—1)

```
struct DirEntryWrapper dir[NFD];
```

⟨*function* Opendir 111e⟩≡ (? 0—1)

```
int
Opendir(char *name)
{
    Dir *db;
    int f;
    f = open(name, 0);
    if(f==-1)
        return f;
    db = dirfstat(f);
    if(db!=nil && (db->mode&DMDIR)){
        if(f<NFD){
            dir[f].i = 0;
            dir[f].n = 0;
        }
        free(db);
        return f;
    }
    free(db);
    close(f);
    return -1;
}
```

⟨*function* trimdirs 112a⟩≡ (? 0—1)

```
static int
trimdirs(Dir *d, int nd)
{
    int r, w;

    for(r=w=0; r<nd; r++)
        if(d[r].mode&DMDIR)
            d[w++] = d[r];
    return w;
}
```

⟨*function* Readdir 112b⟩≡ (? 0—1)

```
/*
 * onlydirs is advisory -- it means you only
 * need to return the directories.  it's okay to
 * return files too (e.g., on unix where you can't
 * tell during the readdir), but that just makes
 * the globber work harder.
 */
int
Readdir(int f, void *p, int onlydirs)
{
    int n;

    if(f<0 || f>=NFD)
        return 0;
Again:
    if(dir[f].i==dir[f].n){ /* read */
        free(dir[f].dbuf);
        dir[f].dbuf = 0;
        n = dirread(f, &dir[f].dbuf);
        if(n>0){
            if(onlydirs){
                n = trimdirs(dir[f].dbuf, n);
                if(n == 0)
                    goto Again;
            }
            dir[f].n = n;
        }else
            dir[f].n = 0;
        dir[f].i = 0;
    }
    if(dir[f].i == dir[f].n)
        return 0;
    strcpy(p, dir[f].dbuf[dir[f].i].name);
    dir[f].i++;
    return 1;
}
```

⟨*function* Closedir 112c⟩≡ (? 0—1)

```
void
Closedir(int f)
{
    if(f>=0 && f<NFD){
        free(dir[f].dbuf);
        dir[f].i = 0;
        dir[f].n = 0;
        dir[f].dbuf = 0;
    }
```

```
    close(f);
}
```

Chapter 14

Advanced Topics TODO

14.1 Reading commands from a string: `rc -c`

⟨`main()` *initialisation* 114a⟩+≡ (33b) ◁104d

```
setvar("cflag", flag['c']? newword(flag['c'][0], (word *)nil) : (word *)nil);
```

14.2 Failing fast: `rc -e`

⟨`outcode()` *emit Xeflag after Xsimple* 114b⟩≡ (60c)

```
if(eflag)
    emitf(Xeflag);
```

Uses `Xeflag()` 114e and `emitf<>` 56d.

⟨*global* `eflagok` 114c⟩≡ (? 0—1)

```
bool eflagok; /* kludge flag so that -e doesn't exit in startup */
```

⟨`execdot()` *if not first execution* 114d⟩≡ (93b)

```
else
    eflagok = true;
```

Uses `eflagok` 114c and `true`.

⟨*function* `Xeflag` 114e⟩≡ (? 0—1)

```
void
Xeflag(void)
{
    if(eflagok && !truestatus())
        Xexit();
}
```

Uses `Xexit()` 76a, `eflagok` 114c, and `truestatus()` 59b.

14.3 Unicode

⟨*function* `addutf` 114f⟩≡ (? 0—1)

```
char*
addutf(char *p, int c)
{
    uchar b, m;
    int i;

    p = addtok(p, c); /* 1-byte UTF runes are special */
    if(c < Runeself)
```

```
        return p;

    m = 0xc0;
    b = 0x80;
    for(i=1; i < UTFmax; i++){
        if((c&m) == b)
            break;
        p = addtok(p, advance());
        b = m;
        m = (m >> 1)|0x80;
    }
    return p;
}
```

Uses `Runeself`, `UTFmax`, `addtok()` 105c, and `advance()` 41a.

14.4 Advanced constructs

14.4.1 Subshell: `@ <cmd>`

⟨*token declarations* 115a⟩+≡ (50a) ◁54d 121b▷

```
%token SUBSHELL /** @ */
```

⟨*cmd rule other cases* 115b⟩+≡ (51b) ◁54h

```
|   SUBSHELL cmd        {$$=mung1($1, $2);}
```

⟨`outcode()` *cases* 115c⟩+≡ (57e) ◁88a 121e▷

```
case SUBSHELL:
    emitf(Xsubshell);
    p = emiti(0);
    outcode(c0, eflag);
    emitf(Xexit);
    stuffdot(p);
    if(eflag)
        emitf(Xeflag);
    break;
```

Uses `Xeflag()` 114e, `Xexit()` 76a, `Xsubshell()`, `c0<>` 57b 133b, `emitf<>` 56d, `emiti<>` 56c, `outcode()` 57e, and `stuffdot()` 66b.

⟨*function* `Xsubshell` 115d⟩≡ (? 0—1)

```
void
Xsubshell(void)
{
    int pid;
    switch(pid = fork()){
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        start(runq->code, runq->pc+1, runq->local);
        runq->ret = nil;
        break;
    default: // parent
        addwaitpid(pid);
        Waitfor(pid, true);
        runq->pc = runq->code[runq->pc].i;
        break;
    }
}
```

Uses `Waitfor()` 75b, `closeio()` 147b, `newword()` 29b, and `runq` 28d.

14.4.2 Here documents: << <HERE>

⟨*constant* HERE 116a⟩≡ (? 0—1)
```
#define HERE 4
```

⟨yylex() *in switch when redirection character, if here document* 116b⟩≡ (46b)
```
if(nextis(c)){
    t->rtype = HERE;
    *w++=c;
}
```
Uses HERE and nextis() 41b.

⟨*struct* Here 116c⟩≡ (? 0—1)
```
struct Here {
    tree *tag;
    char *name;
    struct Here *next;
};
```

⟨*global* here 116d⟩≡ (? 0—1)
```
struct Here *here;
```
Uses Here.

⟨*global* ehere 116e⟩≡ (? 0—1)
```
struct Here **ehere;
```
Uses Here.

⟨*constant* NLINE 116f⟩≡ (? 0—1)
```
/*
 * bug: lines longer than NLINE get split -- this can cause spurious
 * missubstitution, or a misrecognized EOF marker.
 */
#define NLINE 4096
```

⟨compile() *after outcode and error management, read heredoc* 116g⟩≡ (56g)
```
readhere();
```
Uses readhere() 116h.

⟨*function* readhere 116h⟩≡ (? 0—1)
```
void
readhere(void)
{
    int c, subst;
    char *s, *tag;
    char line[NLINE+1];
    io *f;
    struct Here *h, *nexth;

    for(h = here; h; h = nexth){
        subst = !h->tag->quoted;
        tag = h->tag->str;
        c = Creat(h->name);
        if(c < 0)
            yyerror("can't create here document");
        f = openfd(c);
        s = line;
        pprompt();
        while((c = rchr(runq->cmdfd)) != EOF){
            if(c == '\n' || s == &line[NLINE]){
                *s = '\0';
```

```
                if(tag && strcmp(line, tag) == 0)
                    break;
                if(subst)
                    psubst(f, (uchar *)line);
                else
                    pstr(f, line);
                s = line;
                if(c == '\n'){
                    pprompt();
                    pchr(f, c);
                }else
                    *s++ = c;
            }else
                *s++ = c;
        }
        flush(f);
        closeio(f);
        cleanhere(h->name);
        nexth = h->next;
        efree((char *)h);
    }
    here = nil;
    doprompt = true;
}
```

Uses `Creat()` 140b, `EOF` 38b, `Here`, `NLINE<>` 116f, `cleanhere()` 117a, `closeio()` 147b, `doprompt` 39e, `efree()`, `flush()` 145h, `here` 116d, `nil`, `openfd()` 145f, `pchr()` 146a, `pprompt()` 40b, `pstr()` 149a, `psubst()` 118c, `rchr()` 146c, `runq` 28d, `true`, and `yyerror()` 138e.

⟨*function* `cleanhere` 117a⟩≡ (? 0—1)

```
void
cleanhere(char *f)
{
    emitf(Xdelhere);
    emits(strdup(f));
}
```

Uses `Xdelhere()` 117b, `emitf<>` 56d, and `emits<>` 56e.

⟨*function* `Xdelhere` 117b⟩≡ (? 0—1)

```
void
Xdelhere(void)
{
    remove(runq->code[runq->pc++].s);
}
```

Uses `Unlink()` and `runq` 28d.

⟨*function* `Unlink` 117c⟩≡ (? 0—1)

⟨*global* `ser` 117d⟩≡ (? 0—1)

```
int ser = 0;
```

Uses `ser` 117d.

⟨*global tmp (rc/here.c)* 117e⟩≡ (? 0—1)

```
char tmp[] = "/tmp/here0000.0000";
```

Uses `tmp` 117e.

⟨*global* `hex` 117f⟩≡ (? 0—1)

```
char hex[] = "0123456789abcdef";
```

Uses `hex` 117f.

⟨*function* hexnum 118a⟩≡ (? 0—1)

```
void
hexnum(char *p, int n)
{
    *p++ = hex[(n>>12)&0xF];
    *p++ = hex[(n>>8)&0xF];
    *p++ = hex[(n>>4)&0xF];
    *p = hex[n&0xF];
}
```

Uses hex 117f.

⟨*function* heredoc 118b⟩≡ (? 0—1)

```
//@Scheck: used by syn.y
tree* heredoc(tree *tag)
{
    struct Here *h = new(struct Here);

    if(tag->type != WORD)
        yyerror("Bad here tag");
    h->next = 0;
    if(here)
        *ehere = h;
    else
        here = h;
    ehere = &h->next;
    h->tag = tag;
    hexnum(&tmp[9], getpid());
    hexnum(&tmp[14], ser++);
    h->name = strdup(tmp);
    return token(tmp, WORD);
}
```

Uses Here, ehere 116e, here 116d, hexnum() 118a, ser 117d, tmp 117e, token() 47b, and yyerror() 138e.

⟨*function* psubst 118c⟩≡ (? 0—1)

```
void
psubst(io *f, uchar *s)
{
    int savec, n;
    uchar *t, *u;
    Rune r;
    word *star;

    while(*s){
        if(*s != '$'){  /* copy plain text rune */
            if(*s < Runeself)
                pchr(f, *s++);
            else{
                n = chartorune(&r, (char *)s);
                while(n-- > 0)
                    pchr(f, *s++);
            }
        }else{   /* $something -- perform substitution */
            t = ++s;
            if(*t == '$')
                pchr(f, *t++);
            else{
                while(*t && idchr(*t))
                    t++;
                savec = *t;
                *t = '\0';
```

```
                n = 0;
                for(u = s; *u && '0' <= *u && *u <= '9'; u++)
                    n = n*10 + *u - '0';
                if(n && *u == '\0'){
                    star = vlook("*")->val;
                    if(star && 1 <= n && n <= count(star)){
                        while(--n)
                            star = star->next;
                        pstr(f, star->word);
                    }
                }else
                    pstrs(f, vlook((char *)s)->val);
                *t = savec;
                if(savec == '^')
                    t++;
            }
            s = t;
        }
    }
}
```

Uses `Runeself`, `count()` 29c, `idchar()` 42c, `pchr()` 146a, `pstr()` 149a, `pstrs()` 119a, and `vlook()` 31c.

⟨*function* `pstrs` 119a⟩≡ (? 0—1)

```
void
pstrs(io *f, word *a)
{
    if(a){
        while(a->next && a->next->word){
            pstr(f, a->word);
            pchr(f, ' ');
            a = a->next;
        }
        pstr(f, a->word);
    }
}
```

Uses `pchr()` 146a and `pstr()` 149a.

14.4.3 Read-write redirections: <> <file>

⟨*constant* `RDWR` 119b⟩≡ (? 0—1)

```
#define RDWR 7
```

⟨`yylex()` *in switch when redirection character, if read/write redirect* 119c⟩≡ (46b)

```
else if (nextis('>')){
    t->rtype = RDWR;
    *w++=c;
}
```

Uses `RDWR` and `nextis()` 41b.

⟨`outcode()` *when REDIR case, switch redirection type cases* 119d⟩+≡ (69d) ◁71b

```
case RDWR:
    emitf(Xrdwr);
    break;
```

Uses `RDWR`, `Xrdwr()` 120a, and `emitf<>` 56d.

⟨*function* Xrdwr 120a⟩≡ (? 0—1)

```
void
Xrdwr(void)
{
    char *file;
    int f;

    switch(count(runq->argv->words)){
    default:
        Xerror1("<> requires singleton\n");
        return;
    case 0:
        Xerror1("<> requires file\n");
        return;
    case 1:
        break;
    }
    file = runq->argv->words->word;
    if((f = open(file, ORDWR))<0){
        pfmt(err, "%s: ", file);
        Xerror("can't open");
        return;
    }
    pushredir(ROPEN, f, runq->code[runq->pc].i);
    runq->pc++;
    poplist();
}
```

Uses ORDWR, ROPEN 69c, Xerror() 61c, Xerror1() 62a, count() 29c, err 138b, pfmt() 148b, poplist() 58b, pushredir() 68f, and runq 28d.

14.4.4 General redirections: >[2] <file>

⟨yylex() *in switch when redirection character, if bracket after* 120b⟩≡ (45c)

```
if(nextis('[')){
    *w++='[';
    c = advance();
    *w++=c;
    if(c<'0' || '9'<c){
    RedirErr:
        *w = 0;
        yyerror(t->type==PIPE?"pipe syntax"
                :"redirection syntax");
        return EOF;
    }
    t->fd0 = 0;
    do{
        t->fd0 = t->fd0*10 + c-'0';
        *w++=c;
        c = advance();
    }while('0'<=c && c<='9');

    if(c=='='){
        *w++='=';
        if(t->type==REDIR)
            // change the token type
            t->type = DUP;
        c = advance();
        if('0'<=c && c<='9'){
            t->rtype = DUPFD;
```

```
            t->fd1 = t->fd0;
            t->fd0 = 0;
            do{
                t->fd0 = t->fd0*10+c-'0';
                *w++=c;
                c = advance();
            }while('0'<=c && c<='9');
        }
        else{
            if(t->type==PIPE)
                goto RedirErr;
            t->rtype = CLOSE;
        }
    }
    if(c!=']'
    || t->type==DUP && (t->rtype==HERE || t->rtype==APPEND))
        goto RedirErr;
    *w++=']';
}
```

Uses APPEND 45d, CLOSE, DUPFD 25b, EOF 38b, HERE, advance() 41a, nextis() 41b, and yyerror() 138e.

⟨*constant* CLOSE 121a⟩≡ (? 0—1)

```
#define CLOSE 6
```

14.4.5 Advanced dup: >[<fd0>=<fd1>], <>[<fd0>=<fd1>] , <[<fd0>=<fd1>]

⟨*token declarations* 121b⟩+≡ (50a) ◁115a 122h▷

```
%token DUP
```

⟨*redir rule other cases* 121c⟩≡ (53f)

```
|   DUP
```

⟨*constant* DUPFD 121d⟩≡ (? 0—1)

```
#define DUPFD 5
```

⟨outcode() *cases* 121e⟩+≡ (57e) ◁115c 122i▷

```
case DUP:
    if(t->rtype==DUPFD){
        emitf(Xdup);
        emiti(t->fd0);
        emiti(t->fd1);
    }
    else{
        emitf(Xclose);
        emiti(t->fd0);
    }
    outcode(c1, eflag);
    emitf(Xpopredir);
    break;
```

Uses DUPFD 25b, Xclose() 122d, Xdup() 121f, Xpopredir() 70b, c1<> 57c 133c, emitf<> 56d, emiti<> 56c, and outcode() 57e.

⟨*function* Xdup 121f⟩≡ (? 0—1)

```
void
Xdup(void)
{
    pushredir(RDUP, runq->code[runq->pc].i, runq->code[runq->pc+1].i);
    runq->pc+=2;
}
```

Uses RDUP 122b, pushredir() 68f, and runq 28d.

⟨codefree() *in loop over code* cp, *switch bytecode cases* 122a⟩+≡ (28a) ◁82b

```
else if(p->f==Xdup || p->f==Xpipefd)
        p+=2;
```

Uses Xdup() 121f and Xpipefd() 62b.

⟨*constant* RDUP 122b⟩≡ (? 0—1)

```
#define RDUP 2   /* dup2(from, to); */
```

⟨*constant* RCLOSE 122c⟩≡ (? 0—1)

```
#define RCLOSE 3   /* close(from); */
```

⟨*function* Xclose 122d⟩≡ (? 0—1)

```
void
Xclose(void)
{
    pushredir(RCLOSE, runq->code[runq->pc].i, 0);
    runq->pc++;
}
```

Uses RCLOSE 122c, pushredir() 68f, and runq 28d.

⟨doredir() *switch redir type cases* 122e⟩+≡ (69a) ◁69b 122f▷

```
case RDUP:
    dup(rp->from, rp->to);
    break;
```

Uses Dup() 140b and RDUP 122b.

⟨doredir() *switch redir type cases* 122f⟩+≡ (69a) ◁122e

```
case RCLOSE:
    close(rp->from);
    break;
```

Uses RCLOSE 122c.

14.4.6 Advanced pipes: |[<fd>] , |[<fd0>=<fd1>]

14.4.7 Command output as a file: '<{<cmd>}'

⟨*comword rule other cases* 122g⟩+≡ (51e) ◁55c 123b▷

```
|   REDIR brace     {$$=mung1($1, $2); $$->type=PIPEFD;}
```

⟨*token declarations* 122h⟩+≡ (50a) ◁121b

```
%token PIPEFD
```

⟨outcode() *cases* 122i⟩+≡ (57e) ◁121e 123c▷

```
case PIPEFD:
    emitf(Xpipefd);
    emiti(t->rtype);
    p = emiti(0);
    outcode(c0, eflag);
    emitf(Xexit);
    stuffdot(p);
    break;
```

Uses Xexit() 76a, Xpipefd() 62b, c0<> 57b 133b, emitf<> 56d, emiti<> 56c, outcode() 57e, and stuffdot() 66b.

⟨*global* Fdprefix 122j⟩≡ (? 0—1)

```
char *Fdprefix = "/fd/";
```

⟨*function* Xpipefd 123a⟩≡ (? 0—1)

```
void
Xpipefd(void)
{
    struct Thread *p = runq;
    int pc = p->pc, pid;
    char name[40];
    int pfd[2];
    int sidefd, mainfd;
    if(pipe(pfd)<0){
        Xerror("can't get pipe");
        return;
    }
    if(p->code[pc].i==READ){
        sidefd = pfd[PWR];
        mainfd = pfd[PRD];
    }
    else{
        sidefd = pfd[PRD];
        mainfd = pfd[PWR];
    }
    switch(pid = fork()){
    case -1:
        Xerror("try again");
        break;
    case 0: // child
        clearwaitpids();
        start(p->code, pc+2, runq->local);
        close(mainfd);
        pushredir(ROPEN, sidefd, p->code[pc].i==READ?1:0);
        runq->ret = 0;
        break;
    default: // parent
        addwaitpid(pid);
        close(sidefd);
        pushredir(ROPEN, mainfd, mainfd); /* isn't this a noop? */
        strcpy(name, Fdprefix);
        inttoascii(name+strlen(name), mainfd);
        pushword(name);
        p->pc = p->code[pc+1].i;
        break;
    }
}
```

Uses EOF 38b, Io 145e, PRD 31a, PWR, ROPEN 69c, UTFmax, Xerror() 61c, addwaitpid() 59f, clearwaitpids() 64a, nil, openfd() 145f, pushredir() 68f, runq 28d, rutf() 147a, start() 35b, and vlook() 31c.

14.4.8 Command substitution: ‘{<cmd>}

⟨*comword rule other cases* 123b⟩+≡ (51e) ◁122g 125a▷

```
|   ''` brace        {$$=tree1(''`, $2);}
```

⟨outcode() *cases* 123c⟩+≡ (57e) ◁122i 125b▷

```
case ''`:
    emitf(Xbackq);
    p = emiti(0);
    outcode(c0, false);
    emitf(Xexit);
    stuffdot(p);
    break;
```

Uses Xbackq(), Xexit() 76a, c0<> 57b 133b, emitf<> 56d, emiti<> 56c, false, outcode() 57e, and stuffdot() 66b.

⟨*function* Xbackq 124⟩≡ (? 0—1)

```
/*
 * Who should wait for the exit from the fork?
 */
void
Xbackq(void)
{
    int n, pid;
    int pfd[2];
    char *stop;
    char utf[UTFmax+1];
    struct Io *f;
    var *ifs = vlook("ifs");
    word *v, *nextv;
    Rune r;
    //String *word;

    stop = ifs->val? ifs->val->word: "";
    if(pipe(pfd)<0){
        Xerror("can't make pipe");
        return;
    }
    switch(pid = fork()){
    case -1:
        Xerror("try again");
        close(pfd[PRD]);
        close(pfd[PWR]);
        return;
    case 0: // child
        clearwaitpids();
        close(pfd[PRD]);
        start(runq->code, runq->pc+1, runq->local);
        pushredir(ROPEN, pfd[PWR], 1);
        return;
    default: // parent
        Exit("TODO", __LOC__);
        //pad: commented because use the string.h s_xxx library
        // that I didn't want to port to goken
        //addwaitpid(pid);
        //close(pfd[PWR]);
        //f = openfd(pfd[PRD]);
        //word = s_new();
        //v = nil;
        ///* rutf requires at least UTFmax+1 bytes in utf */
        //while((n = rutf(f, utf, &r)) != EOF){
        //    utf[n] = '\0';
        //    if(utfutf(stop, utf) == nil)
        //        s_nappend(word, utf, n);
        //    else
        //        /*
        //         * utf/r is an ifs rune (e.g., \t, \n), thus
        //         * ends the current word, if any.
        //         */
        //        if(s_len(word) > 0){
        //            v = newword(s_to_c(word), v);
        //            s_reset(word);
        //        }
        //}
```

```
        //if(s_len(word) > 0)
        //    v = newword(s_to_c(word), v);
        //s_free(word);
        //closeio(f);
        //Waitfor(pid, false);
        ///* v points to reversed arglist -- reverse it onto argv */
        //while(v){
        //    nextv = v->next;
        //    v->next = runq->argv->words;
        //    runq->argv->words = v;
        //    v = nextv;
        //}
        //runq->pc = runq->code[runq->pc].i;
        //return;
    }
}
```

Uses PRD 31a, PWR, RFFDG, RFNOTEG, RFPROC, ROPEN 69c, Thread 28c, Xerror() 61c, addwaitpid() 59f, clearwaitpids() 64a, inttoascii(), newword() 29b, nil, pushredir() 68f, runq 28d, setvar() 31b, and start() 35b.

14.4.9 Stringification of variables: $"<foo>

⟨*comword rule other cases* 125a⟩+≡ (51e) ◁123b

```
| '"' word        {$$=tree1('"', $2);}
```

⟨outcode() *cases* 125b⟩+≡ (57e) ◁123c

```
case '"':
    emitf(Xmark);
    outcode(c0, eflag);
    emitf(Xqdol);
    break;
```

Uses Xmark() 58a, Xqdol() 125c, c0<> 57b 133b, emitf<> 56d, and outcode() 57e.

⟨*function* Xqdol 125c⟩≡ (? 0—1)

```
void
Xqdol(void)
{
    word *a, *p;
    char *s;
    int n;
    if(count(runq->argv->words)!=1){
        Xerror1("variable name not singleton!");
        return;
    }
    s = runq->argv->words->word;
    deglob(s);
    a = vlook(s)->val;
    poplist();
    n = count(a);
    if(n==0){
        pushword("");
        return;
    }
    for(p = a;p;p = p->next) n+=strlen(p->word);
    s = emalloc(n);
    if(a){
        strcpy(s, a->word);
        for(p = a->next;p;p = p->next){
            strcat(s, " ");
```

```
            strcat(s, p->word);
        }
    }
    else
        s[0]='\0';
    pushword(s);
    efree(s);
}
```

Uses Xerror1() 62a, count() 29c, deglob() 107c, efree(), emalloc(), poplist() 58b, pushword() 36a, runq 28d, and vlook() 31c.

14.5 Advanced builtins

14.5.1 $ exec

14.5.2 $ whatis

⟨*function* execwhatis 126⟩≡ (? 0—1)

```
void
execwhatis(void){   /* mildly wrong -- should fork before writing */
    word *a, *b, *path;
    var *v;
    struct Builtin *bp;
    char *file;
    struct Io out[1];
    int found, sep;
    a = runq->argv->words->next;
    if(a==0){
        Xerror1("Usage: whatis name ...");
        return;
    }
    setstatus("");
    memset(out, 0, sizeof out);
    out->fd = mapfd(1);
    out->bufp = out->buf;
    out->ebuf = &out->buf[NBUF];
    out->strp = nil;
    for(;a;a = a->next){
        v = vlook(a->word);
        if(v->val){
            pfmt(out, "%s=", a->word);
            if(v->val->next==0)
                pfmt(out, "%q\n", v->val->word);
            else{
                sep='(';
                for(b = v->val;b && b->word;b = b->next){
                    pfmt(out, "%c%q", sep, b->word);
                    sep=' ';
                }
                pfmt(out, ")\n");
            }
            found = 1;
        }
        else
            found = 0;
        v = gvlook(a->word);
        if(v->fn)
            pfmt(out, "fn %q %s\n", v->name, v->fn[v->pc-1].s);
```

```
        else{
            for(bp = builtins;bp->name;bp++)
                if(strcmp(a->word, bp->name)==0){
                    pfmt(out, "builtin %s\n", a->word);
                    break;
                }
            if(!bp->name){
                for(path = searchpath(a->word); path;
                    path = path->next){
                    if(path->word[0] != '\0')
                        file = appfile(path->word, a->word);
                    else
                        file = strdup(a->word);
                    if(Executable(file)){
                        pfmt(out, "%s\n", file);
                        free(file);
                        break;
                    }
                    free(file);
                }
                if(!path && !found){
                    pfmt(err, "%s: not found\n", a->word);
                    setstatus("not found");
                }
            }
        }
    }
    poplist();
    flush(err);
}
```

Uses Builtin 90a, Executable() 127b, Io 145e, NBUF 145d, Xerror1() 62a, appfile<>() 92a, builtins, err 138b, flush() 145h, gvlook() 31f, mapfd() 127a, nil, pfmt() 148b, poplist() 58b, runq 28d, searchpath() 63b, setstatus() 58f, and vlook() 31c.

⟨*function* mapfd 127a⟩≡ (? 0—1)

```
int
mapfd(int fd)
{
    redir *rp;
    for(rp = runq->redir;rp;rp = rp->next){
        switch(rp->type){
        case RCLOSE:
            if(rp->from==fd)
                fd=-1;
            break;
        case RDUP:
        case ROPEN:
            if(rp->to==fd)
                fd = rp->from;
            break;
        }
    }
    return fd;
}
```

Uses RCLOSE 122c, RDUP 122b, ROPEN 69c, and runq 28d.

⟨*function* Executable 127b⟩≡ (? 0—1)

```
bool
Executable(char *file)
{
```

```
    Dir *statbuf;
    bool ret;

    statbuf = dirstat(file);
    if(statbuf == nil)
        return false;
    ret = ((statbuf->mode&0111)!=0 && (statbuf->mode&DMDIR)==0);
    free(statbuf);
    return ret;
}
```

Uses DMDIR, false, and nil.

14.5.3 $ rfork

⟨*function* execnewpgrp 128⟩≡ (? 0—1)

```
void
execnewpgrp(void)
{
    int arg;
    char *s;
    switch(count(runq->argv->words)){
    case 1:
        arg = RFENVG|RFNAMEG|RFNOTEG;
        break;
    case 2:
        arg = 0;
        for(s = runq->argv->words->next->word;*s;s++) switch(*s){
        default:
            goto Usage;
        case 'n':
            arg|=RFNAMEG;  break;
        case 'N':
            arg|=RFCNAMEG;
            break;
        //pad: commented for goken
        //case 'm':
        //    arg|=RFNOMNT;  break;
        case 'e':
            arg|=RFENVG;   break;
        case 'E':
            arg|=RFCENVG;  break;
        case 's':
            arg|=RFNOTEG;  break;
        case 'f':
            arg|=RFFDG;    break;
        case 'F':
            arg|=RFCFDG;   break;
        }
        break;
    default:
    Usage:
        pfmt(err, "Usage: %s [fnesFNEm]\n", runq->argv->words->word);
        setstatus("rfork usage");
        poplist();
        return;
    }
    if(rfork(arg)==-1){
        pfmt(err, "rc: %s failed\n", runq->argv->words->word);
        setstatus("rfork failed");
```

```
    }
    else
        setstatus("");
    poplist();
}
```

Uses RFCENVG, RFCFDG, RFCNAMEG, RFENVG, RFFDG, RFNAMEG, RFNOMNT, RFNOTEG, count() 29c, err 138b, pfmt() 148b, poplist() 58b, runq 28d, and setstatus() 58f.

14.5.4 $ flag

⟨*function* execflag 129a⟩≡ (? 0—1)

```
void
execflag(void)
{
    char *letter, *val;

    switch(count(runq->argv->words)){
    case 2:
        setstatus(flag[(uchar)runq->argv->words->next->word[0]]?"":"flag not set");
        break;
    case 3:
        letter = runq->argv->words->next->word;
        val = runq->argv->words->next->next->word;
        if(strlen(letter)==1){
            if(strcmp(val, "+")==0){
                flag[(uchar)letter[0]] = flagset;
                break;
            }
            if(strcmp(val, "-")==0){
                flag[(uchar)letter[0]] = 0;
                break;
            }
        }
    default:
        Xerror1("Usage: flag [letter] [+-]");
        return;
    }
    poplist();
}
```

Uses Xerror1() 62a, count() 29c, flag 33c, flagset 33e, poplist() 58b, runq 28d, and setstatus() 58f.

14.5.5 $ shift

⟨*function* execshift 129b⟩≡ (? 0—1)

```
void
execshift(void)
{
    int n;
    word *a;
    var *star;
    switch(count(runq->argv->words)){
    default:
        pfmt(err, "Usage: shift [n]\n");
        setstatus("shift usage");
        poplist();
        return;
    case 2:
```

```
        n = atoi(runq->argv->words->next->word);
        break;
    case 1:
        n = 1;
        break;
    }
    star = vlook("*");
    for(;n && star->val;--n){
        a = star->val->next;
        efree(star->val->word);
        efree((char *)star->val);
        star->val = a;
        star->changed = true;
    }
    setstatus("");
    poplist();
}
```

Uses `count()` 29c, `efree()`, `err` 138b, `pfmt()` 148b, `poplist()` 58b, `runq` 28d, `setstatus()` 58f, `true`, and `vlook()` 31c.

14.5.6 $ finit

⟨*global* `rdfns` 130a⟩≡ (? 0—1)

```
union Code rdfns[4];
```

⟨*function* `execfinit` 130b⟩≡ (? 0—1)

```
void
execfinit(void)
{
    //TODO: commented for goken for now because Linux does not have a /env
    // probably need to move to Plan9.c
    //static bool first = true;
    //if(first){
    //    rdfns[0].i = 1;
    //    rdfns[1].f = Xrdfn;
    //    rdfns[2].f = Xjump;
    //    rdfns[3].i = 1;
    //    first = false;
    //}
    //Xpopm(); // pop_list()
    //envdir = open("/env", 0);
    //if(envdir<0){
    //    pfmt(err, "rc: can't open /env: %r\n");
    //    return;
    //}
    //start(rdfns, 1, runq->local);
}
```

Uses `Xjump()` 78c, `Xpopm()` 80, `Xrdfn()`, `envdir` 99, `err` 138b, `false`, `pfmt()` 148b, `rdfns`, `runq` 28d, `start()` 35b, and `true`.

⟨*function* `Xrdfn` 130c⟩≡ (? 0—1)

```
void
Xrdfn(void)
{
    int f, len;
    Dir *e;
    char envname[Maxenvname];
    static Dir *ent, *allocent;
    static int nent;
```

```
    for(;;){
        if(nent == 0){
            free(allocent);
            nent = dirread(envdir, &allocent);
            ent = allocent;
        }
        if(nent <= 0)
            break;
        while(nent){
            e = ent++;
            nent--;
            len = e->length;
            if(len && strncmp(e->name, "fn#", 3)==0){
                snprint(envname, sizeof envname, "/env/%s", e->name);
                if((f = open(envname, 0))>=0){
                    execcmds(openfd(f));
                    return;
                }
            }
        }
    }
    close(envdir);
    Xreturn();
}
```

Chapter 15

Conclusion

Appendix A

Debugging

A.1 AST dumper

⟨*global* nl 133a⟩≡ (? 0—1)

```
char nl='\n';  /* change to semicolon for bourne-proofing */
```

Uses nl 133a.

⟨*constant* c0 133b⟩≡ (? 0—1)

```
#define c0 t->child[0]
```

⟨*constant* c1 133c⟩≡ (? 0—1)

```
#define c1 t->child[1]
```

⟨*constant* c2 133d⟩≡ (? 0—1)

```
#define c2 t->child[2]
```

⟨*function* pcmd 133e⟩≡ (? 0—1)

```
void
pcmd(io *f, tree *t)
{
    if(t==nil)
        return;
    assert(f != nil);

    switch(t->type){
    case '$': pfmt(f, "$%t", c0); break;
    case '"': pfmt(f, "$\"%t", c0); break;
    case '&': pfmt(f, "%t&", c0); break;
    case '^': pfmt(f, "%t^%t", c0, c1); break;
    case '`': pfmt(f, "`%t", c0); break;
    case ANDAND:pfmt(f, "%t && %t", c0, c1); break;
    case OROR: pfmt(f, "%t || %t", c0, c1); break;
    case BANG: pfmt(f, "! %t", c0); break;
    case BRACE: pfmt(f, "{%t}", c0); break;
    case COUNT: pfmt(f, "$#%t", c0); break;
    case FN: pfmt(f, "fn %t %t", c0, c1); break;
    case IF: pfmt(f, "if%t%t", c0, c1); break;
    case NOT: pfmt(f, "if not %t", c0); break;
    case PCMD:
    case PAREN: pfmt(f, "(%t)", c0); break;
    case SUB: pfmt(f, "$%t(%t)", c0, c1); break;
    case SIMPLE: pfmt(f, "%t", c0); break;
    case SUBSHELL: pfmt(f, "@ %t", c0); break;
    case SWITCH: pfmt(f, "switch %t %t", c0, c1); break;
    case TWIDDLE: pfmt(f, "~ %t %t", c0, c1); break;
```

```
case WHILE:  pfmt(f, "while %t%t", c0, c1); break;

case ARGLIST:
    if(c0==nil)
        pfmt(f, "%t", c1);
    else if(c1==nil)
        pfmt(f, "%t", c0);
    else
        pfmt(f, "%t %t", c0, c1);
    break;
case ';':
    if(c0){
        if(c1)
            pfmt(f, "%t%c%t", c0, nl, c1);
        else pfmt(f, "%t", c0);
    }
    else pfmt(f, "%t", c1);
    break;
case WORDS:
    if(c0)
        pfmt(f, "%t ", c0);
    pfmt(f, "%t", c1);
    break;
case FOR:
    pfmt(f, "for(%t", c0);
    if(c1)
        pfmt(f, " in %t", c1);
    pfmt(f, ")%t", c2);
    break;
case WORD:
    if(t->quoted)
        pfmt(f, "%Q", t->str);
    else pdeglob(f, t->str);
    break;
case DUP:
    if(t->rtype==DUPFD)
        pfmt(f, ">[%d=%d]", t->fd1, t->fd0); /* yes, fd1, then fd0; read lex.c */
    else
        pfmt(f, ">[%d=]", t->fd0);
    pfmt(f, "%t", c1);
    break;
case PIPEFD:
case REDIR:
    switch(t->rtype){
    case HERE:
        pchr(f, '<');
    case READ:
    case RDWR:
        pchr(f, '<');
        if(t->rtype==RDWR)
            pchr(f, '>');
        if(t->fd0!=0)
            pfmt(f, "[%d]", t->fd0);
        break;
    case APPEND:
        pchr(f, '>');
    case WRITE:
        pchr(f, '>');
        if(t->fd0!=1)
            pfmt(f, "[%d]", t->fd0);
```

```
            break;
        }
        pfmt(f, "%t", c0);
        if(c1)
            pfmt(f, " %t", c1);
        break;
    case '=':
        pfmt(f, "%t=%t", c0, c1);
        if(c2)
            pfmt(f, " %t", c2);
        break;
    case PIPE:
        pfmt(f, "%t|", c0);
        if(t->fd1==0){
            if(t->fd0!=1)
                pfmt(f, "[%d]", t->fd0);
        }
        else pfmt(f, "[%d=%d]", t->fd0, t->fd1);
        pfmt(f, "%t", c1);
        break;

    default:
        pfmt(f, "bad cmd %d %p %p %p", t->type, c0, c1, c2);
        break;
    }
}
```

Uses APPEND 45d, DUPFD 25b, HERE, RDWR, READ, WRITE, assert, c0<> 57b 133b, c1<> 57c 133c, c2<> 57d 133d, nil, nl 133a, pchr() 146a, pdeglob() 135a, and pfmt() 148b.

⟨*function* pdeglob 135a⟩≡ (? 0—1)

```
void
pdeglob(io *f, char *s)
{
    while(*s){
        if(*s==GLOB)
            s++;
        pchr(f, *s++);
    }
}
```

Uses GLOB 30d and pchr() 146a.

A.2 Bytecode generator trace: rc -r

⟨main() *debug runq in interpreter loop* 135b⟩≡ (36b)

```
if(flag['r'])
    pfnc(err, runq);
```

⟨*global* fname 135c⟩≡ (? 0—1)

```
struct{
    void (*f)(void);
    char *name;
} fname[] = {
    Xappend, "Xappend",
    Xassign, "Xassign",
    Xasync, "Xasync",
    Xbackq, "Xbackq",
    Xbang, "Xbang",
    Xcase, "Xcase",
```

```
    Xclose, "Xclose",
    Xconc, "Xconc",
    Xcount, "Xcount",
    Xdelfn, "Xdelfn",
    Xdelhere, "Xdelhere",
    Xdol, "Xdol",
    Xdup, "Xdup",
    Xeflag, "Xeflag",
    (void (*)(void))Xerror, "Xerror",
    Xexit, "Xexit",
    Xfalse, "Xfalse",
    Xfn, "Xfn",
    Xfor, "Xfor",
    Xglob, "Xglob",
    Xif, "Xif",
    Xifnot, "Xifnot",
    Xjump, "Xjump",
    Xlocal, "Xlocal",
    Xmark, "Xmark",
    Xmatch, "Xmatch",
    Xpipe, "Xpipe",
    Xpipefd, "Xpipefd",
    Xpipewait, "Xpipewait",
    Xpopm, "Xpopm",
    Xpopredir, "Xpopredir",
    Xqdol, "Xqdol",
    Xrdcmds, "Xrdcmds",
    Xrdfn, "Xrdfn",
    Xrdwr, "Xrdwr",
    Xread, "Xread",
    Xreturn, "Xreturn",
    Xsimple, "Xsimple",
    Xsub, "Xsub",
    Xsubshell, "Xsubshell",
    Xtrue, "Xtrue",
    Xunlocal, "Xunlocal",
    Xwastrue, "Xwastrue",
    Xword, "Xword",
    Xwrite, "Xwrite",
    0
};
```

Uses Xappend() 71c, Xassign() 84c, Xasync(), Xbackq(), Xbang() 67a, Xcase() 81b, Xclose() 122d, Xconc() 88b, Xcount() 86b, Xdelfn() 83a, Xdelhere() 117b, Xdol() 85d, Xdup() 121f, Xeflag() 114e, Xerror() 61c, Xexit() 76a, Xfalse() 66e, Xfn() 82d, Xfor() 79a, Xglob() 106c, Xif() 76c, Xifnot() 77c, Xjump() 78c, Xlocal() 85a, Xmark() 58a, Xmatch() 67c, Xpipe() 124, Xpipefd() 62b, Xpipewait() 74d, Xpopm() 80, Xpopredir() 70b, Xqdol() 125c, Xrdcmds() 36c, Xrdfn(), Xrdwr() 120a, Xread() 71a, Xreturn() 73b, Xsimple() 61a, Xsub() 87b, Xsubshell(), Xtrue() 66d, Xunlocal() 85b, Xwastrue() 77a, Xword() 57f, Xwrite() 70a, __anon_struct_7 135c, and fname 135c.

⟨*function* pfnc 136⟩≡ (? 0—1)

```
void
pfnc(io *fd, thread *t)
{
    int i;
    void (*fn)(void) = t->code[t->pc].f;
    list *a;

    pfmt(fd, "pid %d cycle %p %d ", getpid(), t->code, t->pc);
    for (i = 0; fname[i].f; i++)
        if (fname[i].f == fn) {
            pstr(fd, fname[i].name);
```

```
            break;
        }
    if (!fname[i].f)
        pfmt(fd, "%p", fn);
    for (a = t->argv; a; a = a->next)
        pfmt(fd, " (%v)", a->words);
    pchr(fd, '\n');
    flush(fd);
}
```

Uses `flush()` 145h, `fname` 135c, `pchr()` 146a, `pfmt()` 148b, and `pstr()` 149a.

A.3 Printing subprocesses status: `rc -s`

⟨`Xrdcmds()` *print status if -s* 137a⟩≡ (36c)

```
if(flag['s'] && !truestatus())
    pfmt(err, "status=%v\n", vlook("status")->val);
```

Uses `err` 138b, `flag` 33c, `pfmt()` 148b, `truestatus()` 59b, and `vlook()` 31c.

A.4 Printing commands: `rc -x`

⟨`Xsimple()` *if -x* 137b⟩≡ (61a)

```
if(flag['x'])
    pfmt(err, "%v\n", p->argv->words); /* wrong, should do redirs */
```

Uses `err` 138b, `flag` 33c, and `pfmt()` 148b.

A.5 Printing characters: `rc -v`

⟨`getnext()` *if not EOF but verbose mode, print character read* 137c⟩≡ (38a)

```
else
    if(flag['V'] || ndot>=2 && flag['v'])
        pchr(err, c);
```

Uses `err` 138b, `flag` 33c, `ndot` 94a, and `pchr()` 146a.

A.6 Printing all characters: `rc -V`

Appendix B

Error Management

⟨*constant* ERRMAX 138a⟩≡ (? 0—1)

```
#define ERRMAX 128
```

⟨*global* err 138b⟩≡ (? 0—1)

```
io *err;
```

⟨*global* nerror 138c⟩≡ (? 0—1)

```
int nerror;  /* number of errors encountered during compilation */
```

⟨Xrdcmds() *flush errors and reset error count* 138d⟩≡ (36c)

```
flush(err);
nerror = 0;
```

Uses err 138b, flush() 145h, and nerror 138c.

⟨*function* yyerror 138e⟩≡ (? 0—1)

```
void
yyerror(char *m)
{
    pfmt(err, "rc: ");
    if(runq->cmdfile && !runq->iflag)
        pfmt(err, "%s:%d: ", runq->cmdfile, runq->lineno);
    else if(runq->cmdfile)
        pfmt(err, "%s: ", runq->cmdfile);
    else if(!runq->iflag)
        pfmt(err, "line %d: ", runq->lineno);

    if(tok[0] && tok[0]!='\n')
        pfmt(err, "token %q: ", tok);
    pfmt(err, "%s\n", m);
    flush(err);
    lastword = false;
    lastdol = false;

    while(lastc!='\n' && lastc!=EOF)
        advance();

    nerror++;
    setvar("status", newword(m, (word *)nil));
}
```

Uses EOF 38b, advance() 41a, err 138b, false, flush() 145h, lastc 138f, lastdol 44c, lastword 46f, nerror 138c, newword() 29b, nil, pfmt() 148b, runq 28d, setvar() 31b, and tok 43b.

⟨*global* lastc 138f⟩≡ (? 0—1)

```
int lastc;
```

⟨advance() *save future in lastc* 139a⟩≡ (41a)

```
lastc = future;
```

Uses future 40f and lastc 138f.

⟨*function* panic 139b⟩≡ (? 0—1)

```
void
panic(char *s, int n)
{
    pfmt(err, "rc: ");
    pfmt(err, s, n);
    pchr(err, '\n');
    flush(err);
    Abort();
}
```

Uses Abort(), err 138b, flush() 145h, pchr() 146a, and pfmt() 148b.

⟨*function* Abort 139c⟩≡ (? 0—1)

```
void
Abort(void)
{
    pfmt(err, "aborting\n");
    flush(err);
    Exit("aborting", __LOC__);
}
```

Uses Updenv(), getstatus() 59a, setstatus() 58f, and truestatus() 59b.

⟨*function* Exit 139d⟩≡ (? 0—1)

```
void
Exit(char *stat, char* _LOC)
{
    USED(_LOC);
    Updenv();
    setstatus(stat);
    exits(truestatus() ? "" : getstatus());
}
```

Appendix C

Utilities

⟨*function* _efgfmt 140a⟩≡ (? 0—1)

```
/* avoid loading any floating-point library code */
//@Scheck: weird, probably linker trick
int _efgfmt(Fmt *)
{
    return -1;
}
```

C.1 Memory management

⟨*function* emalloc 140b⟩≡ (? 0—1)

```
void *
emalloc(long n)
{
    void *p = Malloc(n);

    if(p==nil)
        panic("Can't malloc %d bytes", n);
    return p;
}
```

⟨*function* Malloc 140c⟩≡ (? 0—1)

```
void*
Malloc(ulong n)
{
    return mallocz(n, 1);
}
```

⟨*function* efree 140d⟩≡ (? 0—1)

```
void
efree(void *p)
{
    if(p)
        free(p);
    else pfmt(err, "free 0\n");
}
```

C.2 Command-line arguments

⟨*global* cmdname 140e⟩≡ (? 0—1)

```
char *cmdname;
```

⟨*global* flagarg 141a⟩≡ (? 0—1)

```
static char *flagarg="";
```

Uses flagarg<> 141a.

⟨*global* reason 141b⟩≡ (? 0—1)

```
static int reason;
```

⟨*constant* RESET 141c⟩≡ (? 0—1)

```
#define RESET 1
```

⟨*constant* FEWARGS 141d⟩≡ (? 0—1)

```
#define FEWARGS 2
```

⟨*constant* FLAGSYN 141e⟩≡ (? 0—1)

```
#define FLAGSYN 3
```

⟨*constant* BADFLAG 141f⟩≡ (? 0—1)

```
#define BADFLAG 4
```

⟨*global* badflag 141g⟩≡ (? 0—1)

```
static int badflag;
```

⟨*function* getflags 141h⟩≡ (? 0—1)

```
int
getflags(int argc, char *argv[], char *flags, int stop)
{
    char *s;
    int i, j, c, count;
    flagarg = flags;
    if(cmdname==0)
        cmdname = argv[0];

    i = 1;
    while(i!=argc){
        if(argv[i][0] != '-' || argv[i][1] == '\0'){
            if(stop)  /* always true in rc */
                return argc;
            i++;
            continue;
        }
        s = argv[i]+1;
        while(*s){
            c=*s++;
            count = scanflag(c, flags);
            if(count==-1)
                return -1;
            if(flag[c]){ reason = RESET; badflag = c; return -1; }
            if(count==0){
                flag[c] = flagset;
                if(*s=='\0'){
                    for(j = i+1;j<=argc;j++)
                        argv[j-1] = argv[j];
                    --argc;
                }
            }
            else{
                if(*s=='\0'){
                    for(j = i+1;j<=argc;j++)
                        argv[j-1] = argv[j];
                    --argc;
```

```
                    s = argv[i];
                }
                if(argc-i<count){
                    reason = FEWARGS;
                    badflag = c;
                    return -1;
                }
                reverse(argv+i, argv+argc);
                reverse(argv+i, argv+argc-count);
                reverse(argv+argc-count+1, argv+argc);
                argc-=count;
                flag[c] = argv+argc+1;
                flag[c][0] = s;
                s="";
            }
        }
    }
    return argc;
}
```

Uses FEWARGS<> 141d, RESET<> 141c, badflag<> 141g, cmdname 140e, flag 33c, flagarg<> 141a, flagset 33e, reason<> 141b, reverse<>() 142a, and scanflag<>() 142b.

⟨*function* reverse 142a⟩≡ (? 0—1)

```
static void
reverse(char **p, char **q)
{
    char *t;
    for(;p<q;p++,--q){ t=*p; *p=*q; *q = t; }
}
```

⟨*function* scanflag 142b⟩≡ (? 0—1)

```
static int
scanflag(int c, char *f)
{
    int fc, count;
    if(0<=c && c<NFLAG)
        while(*f){
            if(*f==' '){
                f++;
                continue;
            }
            fc=*f++;
            if(*f==':'){
                f++;
                if(*f<'0' || '9'<*f){ reason = FLAGSYN; return -1; }
                count = 0;
                while('0'<=*f && *f<='9') count = count*10+*f++-'0';
            }
            else
                count = 0;
            if(*f=='['){
                do{
                    f++;
                    if(*f=='\0'){ reason = FLAGSYN; return -1; }
                }while(*f!=']');
                f++;
            }
            if(c==fc)
                return count;
        }
```

```
    reason = BADFLAG;
    badflag = c;
    return -1;
}
```

Uses BADFLAG<> 141f, FLAGSYN<> 141e, NFLAG 33d, badflag<> 141g, and reason<> 141b.

⟨*function* usage 143⟩≡ (? 0—1)

```
void
usage(char *tail)
{
    char *s, *t, c;
    int count, nflag = 0;
    switch(reason){
    case RESET:
        errs("Flag -");
        errc(badflag);
        errs(": set twice\n");
        break;
    case FEWARGS:
        errs("Flag -");
        errc(badflag);
        errs(": too few arguments\n");
        break;
    case FLAGSYN:
        errs("Bad argument to getflags!\n");
        break;
    case BADFLAG:
        errs("Illegal flag -");
        errc(badflag);
        errc('\n');
        break;
    }
    errs("Usage: ");
    errs(cmdname);
    for(s = flagarg;*s;){
        c=*s;
        if(*s++==' ')
            continue;
        if(*s==':'){
            s++;
            count = 0;
            while('0'<=*s && *s<='9') count = count*10+*s++-'0';
        }
        else count = 0;
        if(count==0){
            if(nflag==0)
                errs(" [-");
            nflag++;
            errc(c);
        }
        if(*s=='['){
            s++;
            while(*s!=']' && *s!='\0') s++;
            if(*s==']')
                s++;
        }
    }
    if(nflag)
        errs("]");
    for(s = flagarg;*s;){
```

```
        c=*s;
        if(*s++==' ')
            continue;
        if(*s==':'){
            s++;
            count = 0;
            while('0'<=*s && *s<='9') count = count*10+*s++-'0';
        }
        else count = 0;
        if(count!=0){
            errs(" [-");
            errc(c);
            if(*s=='['){
                s++;
                t = s;
                while(*s!=']' && *s!='\0') s++;
                errs(" ");
                errn(t, s-t);
                if(*s==']')
                    s++;
            }
            else
                while(count--) errs(" arg");
            errs("]");
        }
        else if(*s=='['){
            s++;
            while(*s!=']' && *s!='\0') s++;
            if(*s==']')
                s++;
        }
    }
    if(tail){
        errs(" ");
        errs(tail);
    }
    errs("\n");
    Exit("bad flags", "getflags.c");
}
```

Uses BADFLAG<> 141f, Exit() 139c, FEWARGS<> 141d, FLAGSYN<> 141e, RESET<> 141c, badflag<> 141g, cmdname 140e, errc<>() 145c, errn<>() 144a, errs<>() 144b, flagarg<> 141a, and reason<> 141b.

⟨*function* errn 144a⟩≡ (? 0—1)

```
static void
errn(char *s, int count)
{
    while(count){ errc(*s++); --count; }
}
```

Uses errc<>() 145c.

⟨*function* errs 144b⟩≡ (? 0—1)

```
static void
errs(char *s)
{
    while(*s) errc(*s++);
}
```

Uses errc<>() 145c.

⟨*constant NBUF (rc/getflags.c)* 144c⟩≡ (? 0—1)

```
#define NBUF 80
```

⟨*global* buf 145a⟩≡ (? 0—1)

```
static char buf[NBUF];
```

Uses NBUF<> 144c.

⟨*global* bufp 145b⟩≡ (? 0—1)

```
static char *bufp = buf;
```

Uses buf<> 145a and bufp<> 145b.

⟨*function* errc 145c⟩≡ (? 0—1)

```
static void
errc(int c)
{
    *bufp++=c;
    if(bufp==&buf[NBUF] || c=='\n'){
        write(2, buf, bufp-buf);
        bufp = buf;
    }
}
```

Uses NBUF<> 144c, Write(), buf<> 145a, and bufp<> 145b.

C.3 Buffered IO

⟨*constant* NBUF 145d⟩≡ (? 0—1)

```
#define NBUF 512
```

⟨*struct* Io 145e⟩≡ (? 0—1)

```
struct Io {
    fdt fd;
    byte *bufp, *ebuf, *strp;
    byte buf[NBUF];
};
```

Uses NBUF 145d.

⟨*function* openfd 145f⟩≡ (? 0—1)

```
io*
openfd(fdt fd)
{
    io *f = new(struct Io);
    f->fd = fd;
    f->bufp = f->ebuf = f->buf;
    f->strp = nil;
    return f;
}
```

Uses nil.

⟨*enum* MiscConstants 145g⟩≡ (? 0—1)

```
enum { Stralloc = 100, };
```

⟨*function* flush 145h⟩≡ (? 0—1)

```
void
flush(io *f)
{
    int n;

    if(f->strp){
        n = f->ebuf - f->strp;
        f->strp = realloc(f->strp, n+Stralloc+1);
```

```
        if(f->strp==0)
            panic("Can't realloc %d bytes in flush!", n+Stralloc+1);
        f->bufp = f->strp + n;
        f->ebuf = f->bufp + Stralloc;
        memset(f->bufp, '\0', Stralloc+1);
    }
    else{
        n = f->bufp-f->buf;
        if(n && write(f->fd, f->buf, n) != n){
            write(2, "Write error\n", 12);
            if(ntrap)
                dotrap();
        }
        f->bufp = f->buf;
        f->ebuf = f->buf+NBUF;
    }
}
```

Uses NBUF 145d, Stralloc<> 145g, Write(), dotrap(), ntrap, and panic() 139b.

⟨*function* pchr 146a⟩≡ (? 0—1)

```
void
pchr(io *b, int c)
{
    if(b->bufp==b->ebuf)
        fullbuf(b, c);
    else *b->bufp++=c;
}
```

Uses fullbuf() 146b.

⟨*function* fullbuf 146b⟩≡ (? 0—1)

```
int
fullbuf(io *f, int c)
{
    flush(f);
    return *f->bufp++=c;
}
```

Uses flush() 145h.

⟨*function* rchr 146c⟩≡ (? 0—1)

```
int
rchr(io *b)
{
    if(b->bufp==b->ebuf)
        return emptybuf(b);
    return *b->bufp++;
}
```

Uses emptybuf() 146d.

⟨*function* emptybuf 146d⟩≡ (? 0—1)

```
int
emptybuf(io *f)
{
    int n;
    if(f->fd==-1 || (n = read(f->fd, f->buf, NBUF))<=0) return EOF;
    f->bufp = f->buf;
    f->ebuf = f->buf + n;
    return *f->bufp++;
}
```

Uses EOF 38b, NBUF 145d, and Read().

⟨*function* rutf 147a⟩≡ (? 0—1)

```
int
rutf(io *b, char *buf, Rune *r)
{
    int n, i, c;

    c = rchr(b);
    if(c == EOF)
        return EOF;
    *buf = c;
    if(c < Runesync){
        *r = c;
        return 1;
    }
    for(i = 1; (c = rchr(b)) != EOF; ){
        buf[i++] = c;
        buf[i] = 0;
        if(fullrune(buf, i)){
            n = chartorune(r, buf);
            b->bufp -= i - n; /* push back unconsumed bytes */
            assert(b->fd == -1 || b->bufp > b->buf);
            return n;
        }
    }
    /* at eof */
    b->bufp -= i - 1;   /* consume 1 byte */
    *r = Runeerror;
    return runetochar(buf, r);
}
```

Uses EOF 38b, Runeerror, Runesync, assert, and rchr() 146c.

⟨*function* closeio 147b⟩≡ (? 0—1)

```
void
closeio(io *io)
{
    if(io->fd>=0)
        close(io->fd);
    if(io->strp)
        efree(io->strp);
    efree(io);
}
```

Uses efree().

⟨*function* openstr 147c⟩≡ (? 0—1)

```
io*
openstr(void)
{
    io *f = new(struct Io);

    f->fd = -1;
    f->bufp = f->strp = emalloc(Stralloc+1);
    f->ebuf = f->bufp + Stralloc;
    memset(f->bufp, '\0', Stralloc+1);
    return f;
}
```

Uses Stralloc<> 145g and emalloc().

⟨*function* opencore 147d⟩≡ (? 0—1)

```
/*
```

```
 * Open a corebuffer to read.  EOF occurs after reading len
 * characters from buf.
 */

io*
opencore(char *s, int len)
{
    io *f = new(struct Io);
    uchar *buf = emalloc(len);

    f->fd = -1 /*open("/dev/null", 0)*/;
    f->bufp = f->strp = buf;
    f->ebuf = buf+len;
    Memcpy(buf, s, len);
    return f;
}
```

Uses `Memcpy()` and `emalloc()`.

C.4 Format

⟨*global* `pfmtnest` 148a⟩≡ (? 0—1)

```
int pfmtnest = 0;
```

Uses `pfmtnest` 148a.

⟨*function* `pfmt` 148b⟩≡ (? 0—1)

```
void
pfmt(io *f, char *fmt, ...)
{
    va_list ap;
    char err[ERRMAX];

    va_start(ap, fmt);
    pfmtnest++;
    for(;*fmt;fmt++) {
        if(*fmt!='%') {
            pchr(f, *fmt);
            continue;
        }
        if(*++fmt == '\0')  /* "blah%"? */
            break;
        switch(*fmt){
        case 'c':
            pchr(f, va_arg(ap, int));
            break;
        case 'd':
            pdec(f, va_arg(ap, int));
            break;
        case 'o':
            poct(f, va_arg(ap, unsigned int));
            break;
        case 'p':
            pptr(f, va_arg(ap, void*));
            break;
        case 'Q':
            pquo(f, va_arg(ap, char *));
            break;
        case 'q':
            pwrd(f, va_arg(ap, char *));
```

```
            break;
        case 's':
            pstr(f, va_arg(ap, char *));
            break;

        case 'r':
            errstr(err, sizeof err); pstr(f, err);
            break;

        // rc specific, TODO LP split here
        case 't':
            pcmd(f, va_arg(ap, struct Tree *));
            break;
        case 'v':
            pval(f, va_arg(ap, struct Word *));
            break;

        default:
            pchr(f, *fmt);
            break;
        }
    }
    va_end(ap);
    if(--pfmtnest==0)
        flush(f);
}
```

Uses ERRMAX, Tree, Word, flush() 145h, pchr() 146a, pcmd() 133e, pdec() 149b, pfmtnest 148a, poct() 150a, pptr() 150d, pquo() 150b, pstr() 149a, pval() 150e, and pwrd() 150c.

⟨*function* pstr 149a⟩≡ (? 0—1)

```
void
pstr(io *f, char *s)
{
    if(s==0)
        s="(null)";
    while(*s) pchr(f, *s++);
}
```

Uses pchr() 146a.

⟨*function* pdec 149b⟩≡ (? 0—1)

```
void
pdec(io *f, int n)
{
    if(n<0){
        n=-n;
        if(n>=0){
            pchr(f, '-');
            pdec(f, n);
            return;
        }
        /* n is two's complement minimum integer */
        n = 1-n;
        pchr(f, '-');
        pdec(f, n/10);
        pchr(f, n%10+'1');
        return;
    }
    if(n>9)
        pdec(f, n/10);
    pchr(f, n%10+'0');
```

```
}
```

Uses pchr() 146a and pdec() 149b.

⟨*function* poct 150a⟩≡ (? 0—1)

```
void
poct(io *f, unsigned int n)
{
    if(n>7)
        poct(f, n>>3);
    pchr(f, (n&7)+'0');
}
```

Uses pchr() 146a and poct() 150a.

⟨*function* pquo 150b⟩≡ (? 0—1)

```
void
pquo(io *f, char *s)
{
    pchr(f, '\'');
    for(;*s;s++)
        if(*s=='\'')
            pfmt(f, "'''");
        else pchr(f, *s);
    pchr(f, '\'');
}
```

Uses pchr() 146a and pfmt() 148b.

⟨*function* pwrd 150c⟩≡ (? 0—1)

```
void
pwrd(io *f, char *s)
{
    char *t;
    for(t = s;*t;t++) if(*t >= 0 && needsrcquote(*t)) break;
    if(t==s || *t)
        pquo(f, s);
    else pstr(f, s);
}
```

Uses pquo() 150b and pstr() 149a.

⟨*function* pptr 150d⟩≡ (? 0—1)

```
void
pptr(io *f, void *v)
{
    int n;
    uintptr p;

    p = (uintptr)v;
    if(sizeof(uintptr) == sizeof(uvlong) && p>>32)
        for(n = 60;n>=32;n-=4) pchr(f, "0123456789ABCDEF"[(p>>n)&0xF]);

    for(n = 28;n>=0;n-=4) pchr(f, "0123456789ABCDEF"[(p>>n)&0xF]);
}
```

Uses pchr() 146a.

⟨*function* pval 150e⟩≡ (? 0—1)

```
void
pval(io *f, word *a)
{
    if(a){
        while(a->next && a->next->word){
```

```
            pwrd(f, (char *)a->word);
            pchr(f, ' ');
            a = a->next;
        }
        pwrd(f, (char *)a->word);
    }
}
```

Uses `pchr()` 146a and `pwrd()` 150c.

C.5 String conversions

⟨*global* `bp` 151a⟩≡ (? 0—1)

```
char *bp;
```

⟨*function* `inttoascii` 151b⟩≡ (? 0—1)

```
void
inttoascii(char *s, long n)
{
    bp = s;
    iacvt(n);
    *bp='\0';
}
```

Uses `NFD<>` 140d and `dir` 151c.

⟨*function* `iacvt` 151c⟩≡ (? 0—1)

```
static void
iacvt(int n)
{
    if(n<0){
        *bp++='-';
        n=-n; /* doesn't work for n==-inf */
    }
    if(n/10)
        iacvt(n/10);
    *bp++=n%10+'0';
}
```

Uses `DirEntryWrapper` and `NFD<>` 140d.

C.6 Misc

⟨*function* `Write` 151d⟩≡ (? 0—1)

⟨*function* `Read` 151e⟩≡ (? 0—1)

⟨*function* `Seek` 151f⟩≡ (? 0—1)

⟨*function* `Creat` 151g⟩≡ (? 0—1)

```
int
Creat(char *file)
{
    return create(file, 1, 0666L);
}
```

⟨*function* `Dup` 151h⟩≡ (? 0—1)

⟨*function* Memcpy 152⟩≡ (? 0—1)

```
void
Memcpy(void *a, void *b, long n)
{
    memmove(a, b, n);
}
```

Appendix D

Examples of `rc` scripts TODO

D.1 `/rc/lib/rcmain`

D.2 `/home/pad/lib/profile`

Glossary

```
LOC = Lines Of Code
CLI = Command-Line Interface
GUI = Graphical User Interface
IDE = Integrated Development Environment
```

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

`Abort()`: 139b
`addtok()`: 105b, 105c, 114f
`addutf()`: 46g, 47c, 114f
`addwaitpid()`: 59f, 123a, 124
`advance()`: 41a, 41b, 42a, 43e, 43f, 46g, 47c, 49b, 114f, 120b, 138e
`APPEND`: 45d, 46a, 71b, 120b, 133e
`appfile<>()`: 91b, 92a, 93b, 126
`argv0`: 35c, 35c, 61c, 62a
`assert`: 133e, 147a
`BADFLAG<>`: 141f, 142b, 143
`badflag<>`: 141g, 141h, 142b, 143
`buf<>`: 145a, 145b, 145c
`bufp<>`: 145b, 145b, 145c
`Builtin`: 90a, 90c, 126
`Builtin.fnc`: 90a
`Builtin.name`: 90a
`builtin` (typedef):
`builtins`: 90d, 126
`c0<>`: 57b, 60c, 60d, 60e, 65c, 66a, 66c, 66f, 67b, 69d, 72e, 75a, 76b, 77b, 77f, 78a, 78d, 79c, 81a, 81c, 81d, 82a, 84b, 85c, 86a, 87a, 88a, 115c, 122i, 123c, 125b, 133b, 133e
`c1<>`: 57c, 60d, 60e, 65c, 66a, 66c, 67b, 69d, 72e, 76b, 78a, 78d, 79c, 82a, 84b, 87a, 88a, 121e, 133c, 133e
`c2<>`: 57d, 78d, 84b, 133d, 133e
`cleanhere()`: 116h, 117a
`clearwaitpids()`: 62b, 64a, 123a, 124
`CLOSE`: 120b
`Closedir()`: 106e
`closeio()`: 37c, 51i, 82c, 115d, 116h, 147b
`cmdname`: 140e, 141h, 143
`Code`: 25b, 28c, 65b, 94b, 95c
`Code.f`: 25b
`Code.s`: 45d
`code` (typedef):
`codebuf`: 27d, 36c, 56f, 56g, 57a, 66b
`codecopy()`: 27f, 35b, 82d
`codefree()`: 28a, 73b, 82d, 83a
`codep`: 56a, 56g, 66b, 78a
`codeswitch()`: 79b, 79c
`compile()`: 56g

conclist(): 88b, 88c, 88c
concstatus(): 74d, 74f
copynwords(): 29d, 87c
copywords(): 29e, 76a, 85d
count(): 29c, 63d, 70a, 71a, 71c, 84c, 85a, 85d, 86b, 87b, 88b, 91a, 91b, 93a, 95b, 96b, 118c, 120a, 125c, 128, 129a, 129b
Creat(): 70a, 71c, 116h, 140b
deglob(): 84c, 85a, 85d, 86b, 87b, 87c, 107c, 108b, 125c
delwaitpid(): 73e
dir: 151b, 151c
DirEntryWrapper: 151c
DirEntryWrapper.dbuf:
DirEntryWrapper.n: 151a
DMDIR: 127b
dochdir(): 91b, 92b
doprompt: 39e, 39e, 40a, 40b, 40d, 40e, 116h
doredir(): 68a, 69a, 69a
dotcmds: 93b, 94b, 95a
dotrap(): 145h
Dup(): 69b, 122e, 140b
DUPFD: 25b, 120b, 121e, 133e
eflagok: 95b, 114c, 114d, 114e
efree(): 26e, 28a, 30a, 30b, 37c, 57a, 58b, 58c, 60g, 67c, 70b, 73b, 75b, 81b, 82b, 85b, 88c, 93b, 95b, 108a, 108b, 116h, 125c, 129b, 147b
ehere: 116e, 118b
Eintr(): 37c
emalloc(): 56g, 63d, 67d, 88c, 92a, 95b, 108a, 108b, 125c, 147c, 147d
emitf<>: 56d, 56g, 60c, 60f, 66a, 66c, 66f, 67b, 69d, 69e, 70c, 71b, 72e, 75a, 76b, 77b, 78a, 78d, 79c, 82a, 84b, 85c, 86a, 87a, 88a, 114b, 115c, 117a, 119d, 121e, 122i, 123c, 125b
emiti<>: 56c, 56g, 66a, 66c, 69d, 72e, 75a, 76b, 77b, 78a, 78d, 79c, 82a, 115c, 121e, 122i, 123c
emits<>: 56e, 60f, 78d, 82a, 117a
emptybuf(): 146c, 146d
envdir: 99, 130b
EOF: 38b, 38c, 39a, 39c, 39d, 40d, 40f, 40g, 41a, 42b, 43a, 43e, 46g, 116h, 120b, 123a, 138e, 146d, 147a
epimung(): 53i
equtf(): 109a, 109d
err: 37c, 40b, 57e, 61a, 61c, 62a, 70a, 71a, 71c, 75b, 91a, 91b, 93a, 93b, 103a, 120a, 126, 128, 129b, 130b, 137a, 137b, 137c, 138b, 138d, 138e, 139b
errc<>(): 143, 144a, 144b, 145c
ERRMAX: 73e, 148b
errn<>(): 143, 144a
ERROR_0: 57a
ERROR_NEG1: 92b
errs<>(): 143, 144b
execcd(): 91b
execcmds(): 95b, 96a
execdot(): 90b, 93b
execeval(): 90b, 95b
execexec(): 62c, 65a, 90b

execexit(): 93a
execfinit(): 90b, 130b
execflag(): 90b, 129a
execforkexec(): 61a
execfunc(): 83c, 83d
execnewpgrp(): 90b, 128
execshift(): 90b, 129b
Executable(): 126, 127b
Execute(): 62c, 63d
execwait(): 90b, 96b
execwhatis(): 90b, 126
Exit(): 73b, 76a, 139c, 143
exitnext(): 65a, 65b
false: 32b, 35b, 37c, 40b, 41b, 43a, 43e, 44a, 44d, 46c, 46e, 46g, 47c, 49b, 56g, 59b, 63d, 66a, 66c, 76a, 76b, 77a, 77g, 78a, 81a, 81b, 93b, 95a, 96a, 101c, 106a, 106b, 109b, 109d, 123c, 127b, 130b, 138e
Fdprefix:
FEWARGS<>: 141d, 141h, 143
flag: 33c, 56g, 92d, 103a, 104c, 104d, 129a, 137a, 137b, 137c, 141h
flagarg<>: 141a, 141a, 141h, 143
flagset: 33e, 33e, 104c, 129a, 141h
FLAGSYN<>: 141e, 142b, 143
flush(): 40b, 61a, 61c, 62a, 116h, 126, 136, 138d, 138e, 139b, 145h, 146b, 148b
fname: 135c, 135c, 136
fnstr(): 82a, 82c
freelist(): 30b, 58b, 79a
freenodes(): 26e, 36c
freewords(): 30a, 31b, 84c, 85b
fullbuf(): 146a, 146b
future: 40f, 40f, 40g, 41a, 139a
getflags(): 141h
getnext(): 38a, 40g
getstatus(): 59a, 59b, 73b, 74d, 76a, 139c
GLOB: 30d, 105b, 107a, 107c, 108a, 109d, 135a
glob(): 106a
globcmp(): 108a, 108a
globdir(): 106e, 108a, 108b
globlist(): 82d, 84c, 85a, 105e, 106c
globlist1(): 106a, 106a
globname: 106e, 108a, 108b
Globsize(): 108b
globsort(): 108a, 109a
globv: 106e, 107b, 108a, 108b, 109a
gvar: 31e, 31f
gvlook(): 31c, 31f, 82d, 83a, 83c, 126
hash(): 31f, 32a, 48a, 48f
havewaitpid(): 63d, 73e
HERE: 70c, 116b, 120b, 133e
Here: 116d, 116e, 116h, 118b
here: 116d, 116h, 118b

Here.name:
Here.next:
Here.tag:
heredoc(): 118b
hex: 117f, 117f, 118a
hexnum(): 118a, 118b
idchr(): 42c, 47c, 118c
ifnot: 76c, 76d, 77a, 77c
incomm: 39b, 43d, 43e
inquote: 39b, 46d, 46e, 46g
interrupted: 101c, 101c
inttoascii(): 86b, 103a, 124
Io: 37a, 51i, 123a, 126, 145e
Io.buf: 145e
Io.bufp: 145e
Io.ebuf: 145e
Io.fd: 145e
Io.strp: 145e
io (typedef):
Isatty():
iscase(): 79c, 81a
kenter(): 48e, 48f
kinit(): 48e, 103a
klook(): 47c, 48a
Kw: 48a, 48b, 48b, 48c, 48f
kw: 48a, 48c, 48f
Kw.name: 48b
Kw.next: 48b
Kw.type: 48b
lastc: 138e, 138f, 139a
lastdol: 43a, 44a, 44b, 44c, 44d, 46c, 46g, 47c, 138e
lastword: 46f, 46g, 47c, 49b, 138e
List: 28f, 30c
List.next: 30c
List.words: 30c
list2str(): 67c, 67d, 81b
list (typedef):
main<>(): 34b
mapfd(): 126, 127a
match(): 67c, 81b, 109d, 109d
matchfn(): 106e, 109d
Maxenvname:
Memcpy(): 147d
mkargv(): 63d, 63d
morecode(): 56f
mung1(): 52f
mung2(): 53c
mung3(): 53a
mypid: 76a, 103a, 104a

NBUF: 126, 145d, 145e, 145h, 146d
NBUF<>: 144c, 145a, 145c
ncode: 56b, 56f, 56g
NDIR<>: 107a
ndot: 93b, 94a, 137c
nerror: 57a, 138c, 138d, 138e
newtree(): 26d, 27b, 45c, 47b
newvar(): 31f, 32b, 76a, 83d, 85a
newword(): 29b, 29d, 29e, 36a, 58f, 85d, 88c, 103a, 106e, 108a, 108b, 115d, 124, 138e
nextc(): 40g, 41a, 41b, 43e, 43f, 46g, 47c, 49a
nextis(): 41b, 44a, 44b, 44d, 46a, 116b, 119c, 120b
nextutf(): 106b, 109c, 109d
NFD<>: 140d, 151b, 151c
NFLAG: 33c, 33d, 142b
nil: 26d, 26e, 26f, 27a, 27c, 29d, 29e, 32b, 35b, 35e, 36a, 36b, 45b, 46g, 47c, 52a, 56f, 56g, 57e, 58b, 58d, 58e, 58f, 59e, 61b, 63a, 63b, 63c, 63d, 68b, 70b, 73b, 73e, 75b, 76a, 79c, 82c, 83a, 83d, 84c, 85a, 91b, 92d, 93b, 103a, 105c, 116h, 123a, 124, 126, 127b, 133e, 138e, 145f
NKW<>: 48a, 48c, 48d, 48f
nl: 82c, 133a, 133a, 133e
NLINE<>: 116f, 116h
Noerror(): 102c
NSIG<>: 101c
NSTATUS: 74b, 74c, 74d, 74f
NTOK<>: 43b, 43c, 105c
ntrap: 102e, 145h
nullpath: 63b, 63c, 63c, 91b
NVAR: 29a, 31e, 31f
nwaitpids: 59e, 59e, 60a, 63d, 64a
OCEXEC: 92d
OK_0: 56f
OK_1: 56g, 92b
opencore(): 95b, 147d
Opendir(): 106e, 151c
openfd(): 36b, 93b, 103a, 116h, 123a, 145f
openstr(): 51i, 82c, 147c
ORDWR: 120a
outcode(): 56g, 57e, 60c, 60d, 60e, 65c, 66a, 66c, 66f, 67b, 69d, 72e, 75a, 76b, 77b, 78a, 78d, 79c, 81c, 81d, 82a, 84b, 85c, 86a, 87a, 88a, 115c, 121e, 122i, 123c, 125b
OWRITE: 92d
panic(): 36a, 56f, 58b, 58d, 58e, 66b, 70b, 85b, 139b, 145h
pchr(): 37c, 116h, 118c, 119a, 133e, 135a, 136, 137c, 139b, 146a, 148b, 149a, 149b, 150a, 150b, 150d, 150e
pcmd(): 133e, 148b
pdec(): 148b, 149b, 149b
pdeglob(): 133e, 135a
pfmt(): 51i, 57e, 61c, 62a, 70a, 71a, 71c, 75b, 82c, 91a, 91b, 93a, 93b, 120a, 126, 128, 129b, 130b, 133e, 136, 137a, 137b, 138e, 139b, 148b, 150b
pfmtnest: 148a, 148a, 148b
pfnc(): 136
poct(): 148b, 150a, 150a

poplist(): 58b, 61a, 62c, 67c, 70a, 71a, 71c, 73b, 79a, 80, 81b, 82d, 83a, 83d, 84c, 85a, 85d, 86b, 87b, 88b, 91a, 91b, 95b, 96b, 120a, 125c, 126, 128, 129a, 129b
popword(): 58c, 62c, 83d, 91a, 93b
pprompt(): 40a, 40b, 116h
pptr(): 148b, 150d
pquo(): 148b, 150b, 150c
PRD: 31a, 62b, 123a, 124
promptstr: 39f, 39h, 40b
pstr(): 40b, 116h, 118c, 119a, 136, 148b, 149a, 150c
pstrs(): 118c, 119a
psubst(): 116h, 118c
pushlist(): 35e, 36b, 58a, 93b
pushredir(): 68f, 70a, 71a, 71c, 93b, 120a, 121f, 122d, 123a, 124
pushword(): 36a, 57f, 65a, 86b, 93b, 125c
pval(): 148b, 150e
PWR: 62b, 123a, 124
pwrd(): 148b, 150c, 150e
rchr(): 38a, 39b, 116h, 146c, 147a
RCLOSE: 93b, 122c, 122d, 122f, 127a
Rcmain: 103a
rdcmds: 95c, 96a
rdfns: 130b
RDUP: 121f, 122b, 122e, 127a
RDWR: 119c, 119d, 133e
READ: 46b, 62b, 70c, 133e
Read(): 146d
Readdir(): 106e
readhere(): 116g, 116h
reason<>: 141b, 141h, 142b, 143
Redir: 68c, 68d, 68e, 70b, 72a
Redir.from: 68c
Redir.next: 68e
Redir.to: 68c
Redir.type: 68c
redir (typedef):
RESET<>: 141c, 141h, 143
reverse<>(): 141h, 142a
RFCENVG: 128
RFCFDG: 128
RFCNAMEG: 128
RFENVG: 128
RFFDG: 124, 128
RFNAMEG: 128
RFNOMNT: 128
RFNOTEG: 124, 128
RFPROC: 124
ROPEN: 69b, 69c, 70a, 70b, 71a, 71c, 120a, 123a, 124, 127a
Runeerror: 147a
Runeself: 114f, 118c

Runesync: 147a
runq: 28d, 31c, 35b, 35e, 36a, 36b, 36c, 38a, 38c, 39a, 39b, 39h, 40b, 57f, 58b, 58c, 58d, 61a, 61c, 62a, 62b, 62c, 63a, 65b, 66d, 66e, 67c, 68a, 68b, 68f, 70a, 70b, 71a, 71c, 72b, 73b, 73e, 74d, 76a, 76c, 77c, 77d, 77f, 77g, 78c, 79a, 81b, 82d, 83a, 83d, 84c, 85a, 85b, 85d, 86b, 87b, 88b, 91b, 93a, 93b, 95b, 96a, 96b, 115d, 116h, 117b, 120a, 121f, 122d, 123a, 124, 125c, 126, 127a, 128, 129a, 129b, 130b, 138e
rutf(): 123a, 147a
scanflag<>(): 141h, 142b
searchpath(): 62c, 63b, 93b, 126
Seek(): 71c, 152
ser: 117d, 117d, 118b
setstatus(): 58f, 61c, 62a, 67a, 67c, 73e, 74d, 78b, 91a, 91b, 93a, 93b, 126, 128, 129a, 129b, 139c
setvar(): 31b, 58f, 103a, 124, 138e
signame: 101c, 101c
simplemung(): 51i
skipnl(): 43f, 44a, 44d, 45c
skipwhite(): 42a, 43e, 43f
start(): 35b, 36b, 36c, 62b, 76a, 83d, 93b, 96a, 123a, 124, 130b
STDERR: 103a
STDIN: 36b
Stralloc<>: 145g, 145h, 147c
stuffdot(): 66a, 66b, 66c, 72e, 75a, 76b, 77b, 78a, 78d, 79c, 82a, 115c, 122i, 123c
subwords(): 87b, 87c, 87c
syssigname: 101c, 101c
Thread: 28c, 35b, 36c, 62b, 73b, 124
Thread.argv: 28f
Thread.cmdfd: 37a
Thread.cmdfile: 37a
Thread.code: 28c
Thread.eof: 38d
Thread.iflag: 37a
Thread.iflast: 77e
Thread.lineno: 40c
Thread.local: 28g
Thread.pc: 28c
Thread.pid: 74a
Thread.redir: 68d
Thread.ret: 28e
Thread.startredir: 72a
Thread.status: 74b
thread (typedef):
tmp: 117e, 117e, 118b
tok: 43a, 43b, 44a, 44b, 44d, 44e, 46c, 46g, 47c, 49b, 105c, 138e
token(): 46g, 47b, 48a, 118b
trap: 101c
Trapinit(): 103a
Tree: 45a, 148b
Tree.child:
Tree.fd0:
Tree.fd1:

Tree.rtype:
Tree.str:
Tree.type:
tree1(): 26f, 51i
tree2(): 27a
tree3(): 26f, 27a, 27b
tree (typedef):
treenodes: 26b, 26d, 26e
true: 31b, 33a, 38c, 39e, 40e, 41b, 43e, 44b, 46g, 47c, 56g, 59b, 63d, 76a, 76c, 79a, 81b, 82d, 83a, 83d, 84c, 85a, 85b, 93b, 94c, 95b, 96a, 114d, 116h, 129b, 130b
truestatus(): 59b, 66d, 66e, 67a, 76c, 114e, 137a, 139c
turfredir(): 72b, 72c
unicode(): 106b, 109c, 109d
Unlink(): 117b
Updenv(): 61a, 63d, 139c
usage(): 143
UTFmax: 43b, 114f, 123a
Var: 28g, 31b, 45g, 76a, 93b
Var.fnchanged: 116a
Var.next: 121a
var (typedef):
Vinit(): 103a
vlook(): 31b, 31c, 39h, 40b, 59a, 63b, 76a, 84c, 85b, 85d, 86b, 87b, 91b, 118c, 123a, 125c, 126, 129b, 137a
Waitfor(): 61a, 74d, 75b, 96b, 115d
waitpids: 59d, 59e, 60a, 63d, 64a
Word: 63c, 76a, 107b, 148b
Word.next: 27e
Word.word:
word (typedef):
wordchr(): 42b, 46c, 47c, 49b
WRITE: 46a, 69e, 133e
Write(): 145c, 145h
Xappend(): 28b, 71b, 71c, 135c
Xassign(): 37b, 84b, 84c, 135c
Xasync(): 28b, 75a, 135c
Xbackq(): 28b, 123c, 135c
Xbang(): 66f, 67a, 135c
Xcase(): 28b, 79c, 81b, 135c
Xclose(): 28b, 121e, 122d, 135c
Xconc(): 88a, 88b, 135c
Xcount(): 86a, 86b, 135c
Xdelfn(): 82a, 83a, 135c
Xdelhere(): 60g, 117a, 117b, 135c
Xdol(): 35d, 78d, 85c, 85d, 135c
Xdup(): 121e, 121f, 122a, 135c
Xeflag(): 67b, 114b, 114e, 115c, 135c
Xerror(): 61a, 61c, 62b, 70a, 71a, 71c, 93b, 120a, 123a, 124, 135c
Xerror1(): 61b, 62a, 63a, 70a, 71a, 71c, 84c, 85a, 85d, 86b, 87b, 88b, 93b, 95b, 96b, 120a, 125c, 126, 129a
Xexit(): 36b, 65a, 65b, 72e, 75a, 76a, 93a, 114e, 115c, 122i, 123c, 135c

Xfalse(): 28b, 66c, 66e, 135c
Xfn(): 82a, 82b, 82d, 135c
Xfor(): 28b, 78d, 79a, 135c
Xglob(): 69d, 78d, 106c, 135c
Xif(): 76b, 76c, 135c
Xifnot(): 77b, 77c, 135c
Xjump(): 28b, 78a, 78c, 78d, 79c, 130b, 135c
Xlocal(): 78d, 84b, 85a, 95a, 135c
Xmark(): 37b, 58a, 60c, 67b, 69d, 78d, 79c, 82a, 84b, 85c, 86a, 87a, 88a, 95a, 125b, 135c
Xmatch(): 67b, 67c, 135c
Xpipe(): 72e, 73a, 124, 135c
Xpipefd(): 62b, 122a, 122i, 135c
Xpipewait(): 72e, 74d, 135c
Xpopm(): 79c, 80, 130b, 135c
Xpopredir(): 65b, 69d, 70b, 72b, 121e, 135c
Xqdol(): 125b, 125c, 135c
Xrdcmds(): 36c, 95a, 96a, 135c
Xrdfn(): 130b, 135c
Xrdwr(): 28b, 119d, 120a, 135c
Xread(): 28b, 70c, 71a, 135c
Xreturn(): 37c, 56g, 61c, 62a, 72e, 73b, 82a, 95a, 96a, 135c
Xsettrue(): 78a, 78b
Xsimple(): 33b, 60c, 61a, 135c
Xsub(): 87a, 87b, 135c
Xsubshell(): 28b, 115c, 135c
Xtrue(): 28b, 66a, 66d, 78a, 135c
Xunlocal(): 78d, 82a, 84b, 85b, 95a, 135c
Xwastrue(): 76b, 77a, 135c
Xword(): 35a, 35d, 57f, 60f, 60g, 78d, 95a, 135c
Xwrite(): 28b, 69e, 70a, 135c
yyerror(): 77d, 79c, 105c, 116h, 118b, 120b, 138e
yylex(): 42a
__anon_enum_6: 145g
__anon_enum_8:
__anon_struct_7.f: 135c
__anon_struct_7.name: 135c
__anon_struct_7: 135c, 135c

Bibliography

[BK89] Morris Bolsky and David Korn. *The KornShell Command and Programming Language*. Prentice Hall, 1989. cited page(s) 9

[BKM86] Jon Bentley, Donald Knuth, and Doug McIlroy. Programming pearls: A literate program. *Communication of the ACM*, 29(6):471–483, June 1986. The end of the article contains a one-line shell script solving a problem that required Donald Knuth a hundred lines of Pascal code. cited page(s) 7

[BLM92] Doug Brown, John Levine, and Tony Mason. *Lex and Yacc*. O'Reilly, 1992. cited page(s) 10

[Bou79] Stephen R. Bourne. An introduction to the unix shell. In *Unix Programmer's Manual Vol 2a*, 1979. Also available at `shell/docs/sh.pdf`. cited page(s) 8, 10

[Bou15] Stephen R. Bourne. Early days of unix and design of sh. In *BSDCan - The BSD Conference*, 2015. `https://www.bsdcan.org/2015/schedule/attachments/306_srbBSDCan2015.pdf`, also in `shells/docs/early-days-unix-sh-bourne.pdf`. cited page(s) 11

[Duf90] Tom Duff. Rc – a shell for plan 9 and unix. In *Unix Research System: Papers (Vol 2) 10th ed.* Saunders College, 1990. cited page(s) 8

[Duf00] Tom Duff. Rc – the plan 9 shell. Technical report, Bell Labs, 2000. Also available at `shells/docs/rc.pdf`. cited page(s) 11

[Joh79] Stephen C. Johnson. Yacc: Yet another compiler-compiler. In *Unix Programmer's Manual Vol 2b*, 1979. Also available at `generators/docs/yacc.pdf`. cited page(s) 8

[Joy86] William Joy. An introduction to the c shell. In *UNIX Programmer's Supplementary Documents 1*, 1986. Available at `https://docs.freebsd.org/44doc/usd/04.csh/paper.pdf`. cited page(s) 8

[Kid05] Oliver Kiddle. *From Bash to Z Shell*. Apress, 2005. cited page(s) 9, 10

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 11

[Kor94] David G. Korn. ksh - an extensible high level language. In *Proceedings of the USENIX 1994 Very High Level Languages Symposium*, 1994. cited page(s) 9

[KP84] Brian W. Kernighan and Rob Pike. *The UNIX Programming Environment*. Prentice Hall, 1984. cited page(s) 10

[KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 10

[Mic08] Randal K. Michael. *Master UNIX Shell Scripting*. Wiley, 2008. cited page(s) 10

[New95] Cameron Newham. *Learning the bash Shell*. O'Reilly, 1995. cited page(s) 8, 10

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 11

[Pad14] Yoann Padioleau. *Principia Softwarica: The Plan 9 Kernel 9*. 2014. cited page(s) 10, 14, 19

[Pad16a] Yoann Padioleau. *Principia Softwarica: The Plan 9 C Compiler 5c*. 2016. cited page(s) 10, 21

[Pad16b] Yoann Padioleau. *Principia Softwarica: The Plan 9 Windowing System rio*. 2016. cited page(s) 8, 93

[Pou00] Louis Pouzin. The origin of the shell, 2000. http://multicians.org/shell.html. cited page(s) 12, 13

[Ram94] Chet Ramey. Bash, the bourne-again shell, 1994. Available in bash/doc/rose94.pdf. cited page(s) 8

[Roc04] Marc J. Rochkind. *Advanced UNIX Programming*. Addison-Wesley, 2004. cited page(s) 7, 10

[Ste94] Richard W. Stevens. *Advanced Programming in the UNIX Environment*. Addison-Wesley, 1994. cited page(s) 10